

Line-by-Line Mathematical Guide for HAETAE_Algebra.ec

Generated HAETAE EasyCrypt proof guide

2026-05-11

File Role

Deterministic algebraic model and algebra lemmas.

How To Read This Script

Read this file as the pure mathematical foundation over finite lists, polynomials, vectors, matrices, keys, and signatures.

Line-by-Line Mathematical Reading

Each row preserves one EasyCrypt source line and gives the intended mathematical or proof-script reading. Blank rows are kept because they mark proof structure in long EasyCrypt developments.

Line	EasyCrypt source	Mathematical reading
1	<code>require import AllCore IntDiv List.</code>	Imports AllCore IntDiv List. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
2	<code>require import HAETAE_Params HAETAE_FIPS202.</code>	Imports HAETAE_Params HAETAE_FIPS202. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
3	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4	<code>theory HAETAE_Algebra.</code>	Begins the namespace HAETAE_Algebra, grouping the declarations as one checked theory.
5	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6	<code>import HAETAE_Params.</code>	Opens HAETAE_Params so its names can be used without qualification in the following proof.
7	<code>import HAETAE_FIPS202.</code>	Opens HAETAE_FIPS202 so its names can be used without qualification in the following proof.
8	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
9	<code>type byte = int.</code>	Declares carrier set byte. EasyCrypt will type-check all later functions and games over this mathematical domain.
10	<code>type seed = byte list.</code>	Declares carrier set seed. EasyCrypt will type-check all later functions and games over this mathematical domain.
11	<code>type crh = byte list.</code>	Declares carrier set crh. EasyCrypt will type-check all later functions and games over this mathematical domain.
12	<code>type random_coins = byte list.</code>	Declares carrier set random_coins. EasyCrypt will type-check all later functions and games over this mathematical domain.
13	<code>type xof256_state = byte list.</code>	Declares carrier set xof256_state. EasyCrypt will type-check all later functions and games over this mathematical domain.
14	<code>type message = byte list.</code>	Declares carrier set message. EasyCrypt will type-check all later functions and games over this mathematical domain.

Line	EasyCrypt source	Mathematical reading
15	<code>type context = byte list.</code>	Declares carrier set context. EasyCrypt will type-check all later functions and games over this mathematical domain.
16	<code>type coeff = int.</code>	Declares carrier set coeff. EasyCrypt will type-check all later functions and games over this mathematical domain.
17	<code>type poly = coeff list.</code>	Declares carrier set poly. EasyCrypt will type-check all later functions and games over this mathematical domain.
18	<code>type challenge = poly.</code>	Declares carrier set challenge. EasyCrypt will type-check all later functions and games over this mathematical domain.
19	<code>type polyveck = poly list.</code>	Declares carrier set polyveck. EasyCrypt will type-check all later functions and games over this mathematical domain.
20	<code>type polyvecl = poly list.</code>	Declares carrier set polyvecl. EasyCrypt will type-check all later functions and games over this mathematical domain.
21	<code>type polyvecm = poly list.</code>	Declares carrier set polyvecm. EasyCrypt will type-check all later functions and games over this mathematical domain.
22	<code>type matrix = poly list list.</code>	Declares carrier set matrix. EasyCrypt will type-check all later functions and games over this mathematical domain.
23	<code>type hint_t = polyveck.</code>	Declares carrier set hint_t. EasyCrypt will type-check all later functions and games over this mathematical domain.
24	<code>type sig_token = polyvecl * polyveck * hint_t.</code>	Declares carrier set sig_token. EasyCrypt will type-check all later functions and games over this mathematical domain.
25	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
26	<code>type pkey = seed * polyveck.</code>	Declares carrier set pkey. EasyCrypt will type-check all later functions and games over this mathematical domain.
27	<code>type encoded_pkey = byte list.</code>	Declares carrier set encoded_pkey. EasyCrypt will type-check all later functions and games over this mathematical domain.
28	<code>type skey = seed * int.</code>	Declares carrier set skey. EasyCrypt will type-check all later functions and games over this mathematical domain.
29	<code>type signature = polyveck * poly * crh * challenge * sig_token * int * int.</code>	Declares carrier set signature. EasyCrypt will type-check all later functions and games over this mathematical domain.
30	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
31	<code>op secret_key_of_seed (_ : mode) (sd : seed) : skey = (sd, 0).</code>	Defines operation secret_key_of_seed, a total mathematical function or abbreviation used by later declarations.
32	<code>op poly_zero : poly = nseq n 0.</code>	Defines operation poly_zero, a total mathematical function or abbreviation used by later declarations.
33	<code>op polyveck_zero (md : mode) : polyveck = nseq (mode_k md) poly_zero.</code>	Defines operation polyveck_zero, a total mathematical function or abbreviation used by later declarations.
34	<code>op polyvecl_zero (md : mode) : polyvecl = nseq (mode_l md) poly_zero.</code>	Defines operation polyvecl_zero, a total mathematical function or abbreviation used by later declarations.
35	<code>op polyvecm_zero (md : mode) : polyvecm = nseq (mode_m md) poly_zero.</code>	Defines operation polyvecm_zero, a total mathematical function or abbreviation used by later declarations.
36	<code>op matrix_zero (md : mode) : matrix =</code>	Defines operation matrix_zero, a total mathematical function or abbreviation used by later declarations.
37	<code>nseq (mode_k md) (polyvecl_zero md).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
38	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
39	<code>lemma poly_zero_size :</code>	States checked lemma poly_zero_size. The following proof script must derive this proposition from prior definitions and lemmas.
40	<code>size poly_zero = n.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
41	<code>proof. by rewrite /poly_zero size_nseq /n. qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
42	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
43	<code>lemma polyveck_zero_size md :</code>	States checked lemma polyveck_zero_size. The following proof script must derive this proposition from prior definitions and lemmas.
44	<code>size (polyveck_zero md) = mode_k md.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
45	<code>proof. by rewrite /polyveck_zero size_nseq; case md. qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
46	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
47	<code>lemma polyvecl_zero_size md :</code>	States checked lemma polyvecl_zero_size. The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
48	<code>size (polyvecl_zero md) = mode_l md.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
49	<code>proof. by rewrite /polyvecl_zero size_nseq; case md. qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
50	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
51	<code>lemma polyvecm_zero_size md :</code>	States checked lemma <code>polyvecm_zero_size</code> . The following proof script must derive this proposition from prior definitions and lemmas.
52	<code>size (polyvecm_zero md) = mode_m md.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
53	<code>proof. by rewrite /polyvecm_zero /mode_m size_nseq; case md. qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
54	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
55	<code>op poly_wf (p : poly) : bool = size p = n.</code>	Defines predicate <code>poly_wf</code> , an event or well-formedness condition used in later implications and game proofs.
56	<code>op polyveck_wf (md : mode) (xs : polyveck) : bool =</code>	Defines predicate <code>polyveck_wf</code> , an event or well-formedness condition used in later implications and game proofs.
57	<code>size xs = mode_k md /\ all poly_wf xs.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
58	<code>op polyvecl_wf (md : mode) (xs : polyvecl) : bool =</code>	Defines predicate <code>polyvecl_wf</code> , an event or well-formedness condition used in later implications and game proofs.
59	<code>size xs = mode_l md /\ all poly_wf xs.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
60	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
61	<code>lemma polyveck_wf_nth md b row :</code>	States checked lemma <code>polyveck_wf_nth</code> . The following proof script must derive this proposition from prior definitions and lemmas.
62	<code>polyveck_wf md b =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
63	<code>0 <= row < mode_k md =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
64	<code>poly_wf (nth poly_zero b row).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
65	<code>proof.</code>	Begins the proof script for the preceding proposition.
66	<code>rewrite /polyveck_wf.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
67	<code>move=> [b_sz b_all] row_rng.</code>	Introduces universally quantified variables or hypotheses into the proof context.
68	<code>rewrite /poly_wf.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
69	<code>by smt(allP mem_nth).</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
70	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
71	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
72	<code>op crh_wf (mu : crh) : bool = size mu = crhbytes.</code>	Defines predicate <code>crh_wf</code> , an event or well-formedness condition used in later implications and game proofs.
73	<code>op challenge_coeff_ok (x : coeff) : bool = x = -1 \/ x = 0 \/ x = 1.</code>	Defines predicate <code>challenge_coeff_ok</code> , an event or well-formedness condition used in later implications and game proofs.
74	<code>op challenge_wf (ch : challenge) : bool =</code>	Defines predicate <code>challenge_wf</code> , an event or well-formedness condition used in later implications and game proofs.
75	<code>poly_wf ch /\ all challenge_coeff_ok ch.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
76	<code>op unit_coeff_ok (x : coeff) : bool = x = -1 \/ x = 1.</code>	Defines predicate <code>unit_coeff_ok</code> , an event or well-formedness condition used in later implications and game proofs.
77	<code>op challenge_sparse_at (md : mode) (i : int) (x : coeff) : bool =</code>	Defines predicate <code>challenge_sparse_at</code> , an event or well-formedness condition used in later implications and game proofs.
78	<code>if i < mode_tau md then unit_coeff_ok x else x = 0.</code>	Splits proof obligations according to a conditional branch in the program or formula.
79	<code>op challenge_sparse (md : mode) (ch : challenge) : bool =</code>	Defines predicate <code>challenge_sparse</code> , an event or well-formedness condition used in later implications and game proofs.
80	<code>size ch = n /\</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
81	<code>forall (i : int), 0 <= i /\ i < n =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.

Line	EasyCrypt source	Mathematical reading
82	<code>challenge_sparse_at md i (nth 0 ch i).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
83	<code>op poly_unit (p : poly) : bool = poly_wf p /\ all unit_coeff_ok p.</code>	Defines predicate <code>poly_unit</code> , an event or well-formedness condition used in later implications and game proofs.
84	<code>op polyveck_unit (md : mode) (xs : polyveck) : bool =</code>	Defines predicate <code>polyveck_unit</code> , an event or well-formedness condition used in later implications and game proofs.
85	<code>polyveck_wf md xs /\ all poly_unit xs.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
86	<code>op polyvecl_unit (md : mode) (xs : polyvecl) : bool =</code>	Defines predicate <code>polyvecl_unit</code> , an event or well-formedness condition used in later implications and game proofs.
87	<code>polyvecl_wf md xs /\ all poly_unit xs.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
88	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
89	<code>lemma poly_zero_wf : poly_wf poly_zero.</code>	States checked lemma <code>poly_zero_wf</code> . The following proof script must derive this proposition from prior definitions and lemmas.
90	<code>proof. by rewrite /poly_wf poly_zero_size. qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
91	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
92	<code>lemma polyveck_zero_wf md : polyveck_wf md (polyveck_zero md).</code>	States checked lemma <code>polyveck_zero_wf</code> . The following proof script must derive this proposition from prior definitions and lemmas.
93	<code>proof.</code>	Begins the proof script for the preceding proposition.
94	<code>rewrite /polyveck_wf polyveck_zero_size.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
95	<code>split=> //.</code>	Splits a conjunction or structured goal into independent proof obligations.
96	<code>rewrite /polyveck_zero all_nseq.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
97	<code>by right; apply poly_zero_wf.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
98	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
99	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
100	<code>lemma polyvecl_zero_wf md : polyvecl_wf md (polyvecl_zero md).</code>	States checked lemma <code>polyvecl_zero_wf</code> . The following proof script must derive this proposition from prior definitions and lemmas.
101	<code>proof.</code>	Begins the proof script for the preceding proposition.
102	<code>rewrite /polyvecl_wf polyvecl_zero_size.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
103	<code>split=> //.</code>	Splits a conjunction or structured goal into independent proof obligations.
104	<code>rewrite /polyvecl_zero all_nseq.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
105	<code>by right; apply poly_zero_wf.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
106	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
107	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
108	<code>op coeff_mod (x : coeff) : coeff = x %% q.</code>	Defines operation <code>coeff_mod</code> , a total mathematical function or abbreviation used by later declarations.
109	<code>op coeff_add (x y : coeff) : coeff = coeff_mod (x + y).</code>	Defines operation <code>coeff_add</code> , a total mathematical function or abbreviation used by later declarations.
110	<code>op coeff_neg (x : coeff) : coeff = coeff_mod (-x).</code>	Defines operation <code>coeff_neg</code> , a total mathematical function or abbreviation used by later declarations.
111	<code>op coeff_sub (x y : coeff) : coeff = coeff_add x (coeff_neg y).</code>	Defines operation <code>coeff_sub</code> , a total mathematical function or abbreviation used by later declarations.
112	<code>op coeff_mul (x y : coeff) : coeff = coeff_mod (x * y).</code>	Defines operation <code>coeff_mul</code> , a total mathematical function or abbreviation used by later declarations.
113	<code>op coeff_double (x : coeff) : coeff = coeff_add x x.</code>	Defines operation <code>coeff_double</code> , a total mathematical function or abbreviation used by later declarations.
114	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
115	<code>op poly_coeff (p : poly) (i : int) : coeff = nth 0 p i.</code>	Defines integer-valued parameter <code>poly_coeff</code> , used for dimensions, budgets, support sizes, or bounds.
116	<code>op poly_normalize (p : poly) : poly =</code>	Defines operation <code>poly_normalize</code> , a total mathematical function or abbreviation used by later declarations.
117	<code>mkseq (fun i => coeff_mod (poly_coeff p i)) n.</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.

Line	EasyCrypt source	Mathematical reading
118	<code>op poly_add (p r : poly) : poly =</code>	Defines operation <code>poly_add</code> , a total mathematical function or abbreviation used by later declarations.
119	<code>if p = poly_zero /\ r = poly_zero then poly_zero</code>	Splits proof obligations according to a conditional branch in the program or formula.
120	<code>else mkseq</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
121	<code>(fun i => coeff_add (poly_coeff p i) (poly_coeff r i)) n.</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
122	<code>op poly_neg (p : poly) : poly =</code>	Defines operation <code>poly_neg</code> , a total mathematical function or abbreviation used by later declarations.
123	<code>if p = poly_zero then poly_zero</code>	Splits proof obligations according to a conditional branch in the program or formula.
124	<code>else mkseq (fun i => coeff_neg (poly_coeff p i)) n.</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
125	<code>op poly_sub (p r : poly) : poly = poly_add p (poly_neg r).</code>	Defines operation <code>poly_sub</code> , a total mathematical function or abbreviation used by later declarations.
126	<code>op poly_double (p : poly) : poly =</code>	Defines operation <code>poly_double</code> , a total mathematical function or abbreviation used by later declarations.
127	<code>if p = poly_zero then poly_zero</code>	Splits proof obligations according to a conditional branch in the program or formula.
128	<code>else mkseq (fun i => coeff_double (poly_coeff p i)) n.</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
129	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
130	<code>op poly_mul_term (p r : poly) (i j : int) : coeff =</code>	Defines integer-valued parameter <code>poly_mul_term</code> , used for dimensions, budgets, support sizes, or bounds.
131	<code>if j <= i then poly_coeff p j * poly_coeff r (i - j)</code>	Splits proof obligations according to a conditional branch in the program or formula.
132	<code>else - (poly_coeff p j * poly_coeff r (n + i - j)).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
133	<code>op poly_mul (p r : poly) : poly =</code>	Defines operation <code>poly_mul</code> , a total mathematical function or abbreviation used by later declarations.
134	<code>if p = poly_zero \/ r = poly_zero then poly_zero</code>	Splits proof obligations according to a conditional branch in the program or formula.
135	<code>else mkseq</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
136	<code>(fun i =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
137	<code>coeff_mod</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
138	<code>(sumz (map (fun j => poly_mul_term p r i j) (range 0 n)))) n.</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
139	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
140	<code>op poly_dot (row : poly list) (v : poly list) : poly =</code>	Defines operation <code>poly_dot</code> , a total mathematical function or abbreviation used by later declarations.
141	<code>if row = [] \/ v = [] then poly_zero</code>	Splits proof obligations according to a conditional branch in the program or formula.
142	<code>else foldl</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
143	<code>(fun acc (xy : poly * poly) => poly_add acc (poly_mul xy.‘1 xy.‘2))</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
144	<code>poly_zero</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
145	<code>(zip row v).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
146	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
147	<code>op matrix_vec_mul (md : mode) (a : matrix) (v : polyvec1) : polyveck =</code>	Defines operation <code>matrix_vec_mul</code> , a total mathematical function or abbreviation used by later declarations.
148	<code>if a = [] \/ a = matrix_zero md then polyveck_zero md</code>	Splits proof obligations according to a conditional branch in the program or formula.
149	<code>else mkseq (fun i => poly_dot (nth [] a i) v) (mode_k md).</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
150	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
151	<code>op polyveck_add (md : mode) (xs ys : polyveck) : polyveck =</code>	Defines operation <code>polyveck_add</code> , a total mathematical function or abbreviation used by later declarations.
152	<code>if xs = polyveck_zero md /\ ys = polyveck_zero md then polyveck_zero md</code>	Splits proof obligations according to a conditional branch in the program or formula.
153	<code>else mkseq</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
154	<code>(fun i => poly_add (nth poly_zero xs i) (nth poly_zero ys i))</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
155	<code>(mode_k md).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
156	<code>op polyveck_neg (md : mode) (xs : polyveck) : polyveck =</code>	Defines operation <code>polyveck_neg</code> , a total mathematical function or abbreviation used by later declarations.
157	<code>if xs = polyveck_zero md then polyveck_zero md</code>	Splits proof obligations according to a conditional branch in the program or formula.
158	<code>else mkseq (fun i => poly_neg (nth poly_zero xs i)) (mode_k md).</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
159	<code>op polyveck_sub (md : mode) (xs ys : polyveck) : polyveck =</code>	Defines operation <code>polyveck_sub</code> , a total mathematical function or abbreviation used by later declarations.
160	<code>polyveck_add md xs (polyveck_neg md ys).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
161	<code>op polyveck_double (md : mode) (xs : polyveck) : polyveck =</code>	Defines operation <code>polyveck_double</code> , a total mathematical function or abbreviation used by later declarations.
162	<code>if xs = polyveck_zero md then polyveck_zero md</code>	Splits proof obligations according to a conditional branch in the program or formula.
163	<code>else mkseq (fun i => poly_double (nth poly_zero xs i)) (mode_k md).</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
164	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
165	<code>op polyvecl_sub (md : mode) (xs ys : polyvecl) : polyvecl =</code>	Defines operation <code>polyvecl_sub</code> , a total mathematical function or abbreviation used by later declarations.
166	<code>mkseq</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
167	<code>(fun i => poly_sub (nth poly_zero xs i) (nth poly_zero ys i))</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
168	<code>(mode_l md).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
169	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
170	<code>lemma polyvecl_sub_size md xs ys :</code>	States checked lemma <code>polyvecl_sub_size</code> . The following proof script must derive this proposition from prior definitions and lemmas.
171	<code>size (polyvecl_sub md xs ys) = mode_l md.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
172	<code>proof. by rewrite /polyvecl_sub size_mkseq; case md. qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
173	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
174	<code>lemma poly_add_wf p r : poly_wf (poly_add p r).</code>	States checked lemma <code>poly_add_wf</code> . The following proof script must derive this proposition from prior definitions and lemmas.
175	<code>proof.</code>	Begins the proof script for the preceding proposition.
176	<code>rewrite /poly_wf /poly_add.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
177	<code>case: (p = poly_zero /\ r = poly_zero) => _.</code>	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
178	<code>+ by apply poly_zero_size.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
179	<code>by rewrite size_mkseq /n.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
180	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
181	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
182	<code>lemma poly_normalize_wf p : poly_wf (poly_normalize p).</code>	States checked lemma <code>poly_normalize_wf</code> . The following proof script must derive this proposition from prior definitions and lemmas.
183	<code>proof. by rewrite /poly_wf /poly_normalize size_mkseq /n. qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
184	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
185	lemma poly_neg_wf p : poly_wf (poly_neg p).	States checked lemma poly_neg_wf. The following proof script must derive this proposition from prior definitions and lemmas.
186	proof.	Begins the proof script for the preceding proposition.
187	rewrite /poly_wf /poly_neg.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
188	case: (p = poly_zero) => _.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
189	+ by apply poly_zero_size.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
190	by rewrite size_mkseq /n.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
191	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
192	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
193	lemma poly_sub_wf p r : poly_wf (poly_sub p r).	States checked lemma poly_sub_wf. The following proof script must derive this proposition from prior definitions and lemmas.
194	proof. by rewrite /poly_sub; apply poly_add_wf. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
195	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
196	lemma poly_double_wf p : poly_wf (poly_double p).	States checked lemma poly_double_wf. The following proof script must derive this proposition from prior definitions and lemmas.
197	proof.	Begins the proof script for the preceding proposition.
198	rewrite /poly_wf /poly_double.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
199	case: (p = poly_zero) => _.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
200	+ by apply poly_zero_size.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
201	by rewrite size_mkseq /n.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
202	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
203	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
204	lemma poly_mul_wf p r : poly_wf (poly_mul p r).	States checked lemma poly_mul_wf. The following proof script must derive this proposition from prior definitions and lemmas.
205	proof.	Begins the proof script for the preceding proposition.
206	rewrite /poly_wf /poly_mul.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
207	case: (p = poly_zero $\vee$ r = poly_zero) => _.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
208	+ by apply poly_zero_size.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
209	by rewrite size_mkseq /n.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
210	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
211	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
212	lemma poly_dot_wf row v : poly_wf (poly_dot row v).	States checked lemma poly_dot_wf. The following proof script must derive this proposition from prior definitions and lemmas.
213	proof.	Begins the proof script for the preceding proposition.
214	rewrite /poly_dot.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
215	case: (row = [] $\vee$ v = []) => _.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
216	+ by apply poly_zero_wf.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
217	have h :	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
218	forall zs acc,	Part of a universal mathematical condition that must hold for all listed values.
219	poly_wf acc =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
220	poly_wf	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
221	(foldl	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
222	(fun (acc0 : poly) (xy : poly * poly) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
223	poly_add acc0 (poly_mul xy.‘1 xy.‘2))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
224	acc zs).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
225	+ elim=> [xy zs ih] acc acc_wf //=.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
226	apply ih.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
227	by apply poly_add_wf.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
228	by apply h; apply poly_zero_wf.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
229	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
230	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
231	lemma matrix_vec_mul_wf md a v : polyveck_wf md (matrix_vec_mul md a v).	States checked lemma matrix_vec_mul_wf. The following proof script must derive this proposition from prior definitions and lemmas.
232	proof.	Begins the proof script for the preceding proposition.
233	rewrite /matrix_vec_mul.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
234	case: (a = [] \ / a = matrix_zero md) => _.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
235	+ by apply polyveck_zero_wf.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
236	rewrite /polyveck_wf size_mkseq.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
237	split.	Splits a conjunction or structured goal into independent proof obligations.
238	+ by smt(mode_k_gt0).	Solves a proof branch produced by a previous split, transitivity, or case analysis.
239	apply/allP=> p /mkseqP [i [_ ->]].	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
240	by apply poly_dot_wf.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
241	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
242	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
243	lemma polyveck_add_wf md xs ys : polyveck_wf md (polyveck_add md xs ys).	States checked lemma polyveck_add_wf. The following proof script must derive this proposition from prior definitions and lemmas.
244	proof.	Begins the proof script for the preceding proposition.
245	rewrite /polyveck_wf /polyveck_add.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
246	case: (xs = polyveck_zero md /\ ys = polyveck_zero md) => _.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
247	+ by apply polyveck_zero_wf.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
248	rewrite size_mkseq.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
249	split.	Splits a conjunction or structured goal into independent proof obligations.
250	+ by smt(mode_k_gt0).	Solves a proof branch produced by a previous split, transitivity, or case analysis.
251	apply/allP=> p /mkseqP [i [_ ->]].	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
252	by apply poly_add_wf.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
253	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
254	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
255	lemma polyveck_neg_wf md xs : polyveck_wf md (polyveck_neg md xs).	States checked lemma polyveck_neg_wf. The following proof script must derive this proposition from prior definitions and lemmas.
256	proof.	Begins the proof script for the preceding proposition.
257	rewrite /polyveck_wf /polyveck_neg.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
258	case: (xs = polyveck_zero md) => _.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.

Line	EasyCrypt source	Mathematical reading
259	+ by apply polyveck_zero_wf.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
260	rewrite size_mkseq.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
261	split.	Splits a conjunction or structured goal into independent proof obligations.
262	+ by smt(mode_k_gt0).	Solves a proof branch produced by a previous split, transitivity, or case analysis.
263	apply/allP=> p /mkseqP [i [_ ->]].	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
264	by apply poly_neg_wf.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
265	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
266	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
267	lemma polyveck_sub_wf md xs ys : polyveck_wf md (polyveck_sub md xs ys).	States checked lemma polyveck_sub_wf. The following proof script must derive this proposition from prior definitions and lemmas.
268	proof. by rewrite /polyveck_sub; apply polyveck_add_wf. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
269	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
270	lemma polyveck_double_wf md xs : polyveck_wf md (polyveck_double md xs).	States checked lemma polyveck_double_wf. The following proof script must derive this proposition from prior definitions and lemmas.
271	proof.	Begins the proof script for the preceding proposition.
272	rewrite /polyveck_wf /polyveck_double.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
273	case: (xs = polyveck_zero md) => _.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
274	+ by apply polyveck_zero_wf.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
275	rewrite size_mkseq.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
276	split.	Splits a conjunction or structured goal into independent proof obligations.
277	+ by smt(mode_k_gt0).	Solves a proof branch produced by a previous split, transitivity, or case analysis.
278	apply/allP=> p /mkseqP [i [_ ->]].	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
279	by apply poly_double_wf.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
280	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
281	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
282	lemma polyvecl_sub_wf md xs ys : polyvecl_wf md (polyvecl_sub md xs ys).	States checked lemma polyvecl_sub_wf. The following proof script must derive this proposition from prior definitions and lemmas.
283	proof.	Begins the proof script for the preceding proposition.
284	rewrite /polyvecl_wf /polyvecl_sub size_mkseq.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
285	split.	Splits a conjunction or structured goal into independent proof obligations.
286	+ by case md.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
287	apply/allP=> p /mkseqP [i [_ ->]].	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
288	by apply poly_sub_wf.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
289	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
290	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
291	op coeff_q_bound (x : coeff) : bool = 0 <= x < q.	Defines predicate coeff_q_bound, an event or well-formedness condition used in later implications and game proofs.
292	op poly_coeffs_q_bound (p : poly) : bool =	Defines predicate poly_coeffs_q_bound, an event or well-formedness condition used in later implications and game proofs.
293	forall i, 0 <= i < n => coeff_q_bound (poly_coeff p i).	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
294	op polyveck_coeffs_q_bound (md : mode) (xs : polyveck) : bool =	Defines predicate polyveck_coeffs_q_bound, an event or well-formedness condition used in later implications and game proofs.
295	forall row, 0 <= row < mode_k md =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.

Line	EasyCrypt source	Mathematical reading
296	<code>poly_coeffs_q_bound (nth poly_zero xs row).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
297	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
298	<code>lemma coeff_mod_q_bound x : coeff_q_bound (coeff_mod x).</code>	States checked lemma <code>coeff_mod_q_bound</code> . The following proof script must derive this proposition from prior definitions and lemmas.
299	<code>proof. by rewrite /coeff_q_bound /coeff_mod; smt(q_gt0). qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
300	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
301	<code>lemma poly_zero_coeffs_q_bound : poly_coeffs_q_bound poly_zero.</code>	States checked lemma <code>poly_zero_coeffs_q_bound</code> . The following proof script must derive this proposition from prior definitions and lemmas.
302	<code>proof.</code>	Begins the proof script for the preceding proposition.
303	<code>rewrite /poly_coeffs_q_bound /poly_coeff /poly_zero.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
304	<code>by smt(q_gt0 nth_nseq).</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
305	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
306	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
307	<code>lemma poly_add_coeffs_q_bound p r :</code>	States checked lemma <code>poly_add_coeffs_q_bound</code> . The following proof script must derive this proposition from prior definitions and lemmas.
308	<code>poly_coeffs_q_bound (poly_add p r).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
309	<code>proof.</code>	Begins the proof script for the preceding proposition.
310	<code>rewrite /poly_coeffs_q_bound /poly_add => i i_rng.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
311	<code>case: (p = poly_zero /\ r = poly_zero) => _.</code>	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
312	<code>+ by apply poly_zero_coeffs_q_bound.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
313	<code>rewrite /poly_coeff nth_mkseq; first by smt().</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
314	<code>by apply coeff_mod_q_bound.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
315	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
316	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
317	<code>lemma poly_neg_coeffs_q_bound p :</code>	States checked lemma <code>poly_neg_coeffs_q_bound</code> . The following proof script must derive this proposition from prior definitions and lemmas.
318	<code>poly_coeffs_q_bound (poly_neg p).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
319	<code>proof.</code>	Begins the proof script for the preceding proposition.
320	<code>rewrite /poly_coeffs_q_bound /poly_neg => i i_rng.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
321	<code>case: (p = poly_zero) => _.</code>	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
322	<code>+ by apply poly_zero_coeffs_q_bound.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
323	<code>rewrite /poly_coeff nth_mkseq; first by smt().</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
324	<code>by apply coeff_mod_q_bound.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
325	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
326	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
327	<code>lemma poly_sub_coeffs_q_bound p r :</code>	States checked lemma <code>poly_sub_coeffs_q_bound</code> . The following proof script must derive this proposition from prior definitions and lemmas.
328	<code>poly_coeffs_q_bound (poly_sub p r).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
329	<code>proof. by rewrite /poly_sub; apply poly_add_coeffs_q_bound. qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
330	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
331	<code>lemma poly_double_coeffs_q_bound p :</code>	States checked lemma <code>poly_double_coeffs_q_bound</code> . The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
332	<code>poly_coeffs_q_bound (poly_double p).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
333	<code>proof.</code>	Begins the proof script for the preceding proposition.
334	<code>rewrite /poly_coeffs_q_bound /poly_double => i i_rng.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
335	<code>case: (p = poly_zero) => _.</code>	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
336	<code>+ by apply poly_zero_coeffs_q_bound.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
337	<code>rewrite /poly_coeff_nth_mkseq; first by smt().</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
338	<code>by apply coeff_mod_q_bound.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
339	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
340	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
341	<code>lemma poly_mul_coeffs_q_bound p r :</code>	States checked lemma <code>poly_mul_coeffs_q_bound</code> . The following proof script must derive this proposition from prior definitions and lemmas.
342	<code>poly_coeffs_q_bound (poly_mul p r).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
343	<code>proof.</code>	Begins the proof script for the preceding proposition.
344	<code>rewrite /poly_coeffs_q_bound /poly_mul => i i_rng.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
345	<code>case: (p = poly_zero \ / r = poly_zero) => _.</code>	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
346	<code>+ by apply poly_zero_coeffs_q_bound.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
347	<code>rewrite /poly_coeff_nth_mkseq; first by smt().</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
348	<code>by apply coeff_mod_q_bound.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
349	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
350	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
351	<code>lemma poly_dot_coeffs_q_bound row v :</code>	States checked lemma <code>poly_dot_coeffs_q_bound</code> . The following proof script must derive this proposition from prior definitions and lemmas.
352	<code>poly_coeffs_q_bound (poly_dot row v).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
353	<code>proof.</code>	Begins the proof script for the preceding proposition.
354	<code>rewrite /poly_dot.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
355	<code>case: (row = [] \ / v = []) => _.</code>	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
356	<code>+ by apply poly_zero_coeffs_q_bound.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
357	<code>have h :</code>	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
358	<code>forall zs acc,</code>	Part of a universal mathematical condition that must hold for all listed values.
359	<code>poly_coeffs_q_bound acc =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
360	<code>poly_coeffs_q_bound</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
361	<code>(foldl</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
362	<code>(fun (acc0 : poly) (xy : poly * poly) =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
363	<code>poly_add acc0 (poly_mul xy.‘1 xy.‘2))</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
364	<code>acc zs).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
365	<code>+ elim=> [xy zs ih] acc acc_bd //=.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
366	<code>apply ih.</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
367	<code>by apply poly_add_coeffs_q_bound.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.

Line	EasyCrypt source	Mathematical reading
368	by apply h; apply poly_zero_coeffs_q_bound.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
369	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
370	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
371	lemma matrix_vec_mul_coeffs_q_bound md a v :	States checked lemma <code>matrix_vec_mul_coeffs_q_bound</code> . The following proof script must derive this proposition from prior definitions and lemmas.
372	polyveck_coeffs_q_bound md (matrix_vec_mul md a v).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
373	proof.	Begins the proof script for the preceding proposition.
374	rewrite /polyveck_coeffs_q_bound /matrix_vec_mul => row row_rng.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
375	case: (a = [] \ / a = matrix_zero md) => _.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
376	+ rewrite /polyveck_zero.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
377	have -> : nth poly_zero (nseq (mode_k md) poly_zero) row = poly_zero	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
378	by smt(nth_nseq).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
379	by apply poly_zero_coeffs_q_bound.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
380	rewrite nth_mkseq; first by smt().	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
381	by apply poly_dot_coeffs_q_bound.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
382	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
383	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
384	lemma polyveck_zero_coeffs_q_bound md :	States checked lemma <code>polyveck_zero_coeffs_q_bound</code> . The following proof script must derive this proposition from prior definitions and lemmas.
385	polyveck_coeffs_q_bound md (polyveck_zero md).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
386	proof.	Begins the proof script for the preceding proposition.
387	rewrite /polyveck_coeffs_q_bound /polyveck_zero => row row_rng.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
388	have -> : nth poly_zero (nseq (mode_k md) poly_zero) row = poly_zero	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
389	by smt(nth_nseq).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
390	by apply poly_zero_coeffs_q_bound.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
391	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
392	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
393	lemma polyveck_add_coeffs_q_bound md xs ys :	States checked lemma <code>polyveck_add_coeffs_q_bound</code> . The following proof script must derive this proposition from prior definitions and lemmas.
394	polyveck_coeffs_q_bound md (polyveck_add md xs ys).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
395	proof.	Begins the proof script for the preceding proposition.
396	rewrite /polyveck_coeffs_q_bound /polyveck_add => row row_rng.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
397	case: (xs = polyveck_zero md /\ ys = polyveck_zero md) => _.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
398	+ by apply polyveck_zero_coeffs_q_bound.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
399	rewrite nth_mkseq; first by smt().	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
400	by apply poly_add_coeffs_q_bound.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
401	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
402	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
403	lemma polyveck_neg_coeffs_q_bound md xs :	States checked lemma <code>polyveck_neg_coeffs_q_bound</code> . The following proof script must derive this proposition from prior definitions and lemmas.
404	polyveck_coeffs_q_bound md (polyveck_neg md xs).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
405	proof.	Begins the proof script for the preceding proposition.
406	rewrite /polyveck_coeffs_q_bound /polyveck_neg => row row_rng.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
407	case: (xs = polyveck_zero md) => _.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
408	+ by apply polyveck_zero_coeffs_q_bound.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
409	rewrite nth_mkseq; first by smt().	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
410	by apply poly_neg_coeffs_q_bound.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
411	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
412	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
413	lemma polyveck_sub_coeffs_q_bound md xs ys :	States checked lemma polyveck_sub_coeffs_q_bound. The following proof script must derive this proposition from prior definitions and lemmas.
414	polyveck_coeffs_q_bound md (polyveck_sub md xs ys).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
415	proof. by rewrite /polyveck_sub; apply polyveck_add_coeffs_q_bound. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
416	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
417	lemma polyveck_double_coeffs_q_bound md xs :	States checked lemma polyveck_double_coeffs_q_bound. The following proof script must derive this proposition from prior definitions and lemmas.
418	polyveck_coeffs_q_bound md (polyveck_double md xs).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
419	proof.	Begins the proof script for the preceding proposition.
420	rewrite /polyveck_coeffs_q_bound /polyveck_double => row row_rng.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
421	case: (xs = polyveck_zero md) => _.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
422	+ by apply polyveck_zero_coeffs_q_bound.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
423	rewrite nth_mkseq; first by smt().	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
424	by apply poly_double_coeffs_q_bound.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
425	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
426	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
427	op z1_alpha : int = 256.	Defines integer-valued parameter z1_alpha, used for dimensions, budgets, support sizes, or bounds.
428	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
429	op coeff_decompose_z1_low (x : coeff) : coeff =	Defines operation coeff_decompose_z1_low, a total mathematical function or abbreviation used by later declarations.
430	let lb = x %% z1_alpha in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
431	if z1_alpha %/ 2 <= lb then lb - z1_alpha else lb.	Splits proof obligations according to a conditional branch in the program or formula.
432	op coeff_decompose_z1_high (x : coeff) : coeff =	Defines operation coeff_decompose_z1_high, a total mathematical function or abbreviation used by later declarations.
433	(x + (z1_alpha %/ 2)) %/ z1_alpha.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
434	op coeff_lsb (x : coeff) : coeff = x %% 2.	Defines operation coeff_lsb, a total mathematical function or abbreviation used by later declarations.
435	op coeff_decompose_vk_low (x : coeff) : coeff =	Defines operation coeff_decompose_vk_low, a total mathematical function or abbreviation used by later declarations.
436	if x %% 2 = 0 then 0	Splits proof obligations according to a conditional branch in the program or formula.
437	else if (x %/ 2) %% 2 = 0 then 1	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
438	else -1.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
439	op coeff_decompose_vk_high (x : coeff) : coeff =	Defines operation coeff_decompose_vk_high, a total mathematical function or abbreviation used by later declarations.

Line	EasyCrypt source	Mathematical reading
440	<code>(x - coeff_decompose_vk_low x) %/ 2.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
441	<code>op coeff_decompose_hint_high (md : mode) (x : coeff) : coeff =</code>	Defines operation <code>coeff_decompose_hint_high</code> , a total mathematical function or abbreviation used by later declarations.
442	<code>let alpha = mode_alpha_hint md in</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
443	<code>let hb = (x + (alpha %/ 2)) %/ alpha in</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
444	<code>let top = (dq - 2) %/ alpha in</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
445	<code>if top <= hb then hb - top else hb.</code>	Splits proof obligations according to a conditional branch in the program or formula.
446	<code>op coeff_compose_z1 (hi lo : coeff) : coeff = hi * z1_alpha + lo.</code>	Defines operation <code>coeff_compose_z1</code> , a total mathematical function or abbreviation used by later declarations.
447	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
448	<code>op poly_highbits (p : poly) : poly =</code>	Defines operation <code>poly_highbits</code> , a total mathematical function or abbreviation used by later declarations.
449	<code>mkseq (fun i => coeff_decompose_z1_high (poly_coeff p i)) n.</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
450	<code>op poly_lowbits (p : poly) : poly =</code>	Defines operation <code>poly_lowbits</code> , a total mathematical function or abbreviation used by later declarations.
451	<code>mkseq (fun i => coeff_decompose_z1_low (poly_coeff p i)) n.</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
452	<code>op poly_lsb (p : poly) : poly =</code>	Defines operation <code>poly_lsb</code> , a total mathematical function or abbreviation used by later declarations.
453	<code>mkseq (fun i => coeff_lsb (poly_coeff p i)) n.</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
454	<code>op poly_vk_lowbits (p : poly) : poly =</code>	Defines operation <code>poly_vk_lowbits</code> , a total mathematical function or abbreviation used by later declarations.
455	<code>mkseq (fun i => coeff_decompose_vk_low (poly_coeff p i)) n.</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
456	<code>op poly_vk_highbits (p : poly) : poly =</code>	Defines operation <code>poly_vk_highbits</code> , a total mathematical function or abbreviation used by later declarations.
457	<code>mkseq (fun i => coeff_decompose_vk_high (poly_coeff p i)) n.</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
458	<code>op poly_compose_z1 (hi lo : poly) : poly =</code>	Defines operation <code>poly_compose_z1</code> , a total mathematical function or abbreviation used by later declarations.
459	<code>mkseq</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
460	<code>(fun i =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
461	<code>coeff_compose_z1 (poly_coeff hi i) (poly_coeff lo i)) n.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
462	<code>op poly_hint_highbits (md : mode) (p : poly) : poly =</code>	Defines operation <code>poly_hint_highbits</code> , a total mathematical function or abbreviation used by later declarations.
463	<code>mkseq (fun i => coeff_decompose_hint_high md (poly_coeff p i)) n.</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
464	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
465	<code>op polyvecl_highbits (md : mode) (xs : polyvecl) : polyvecl =</code>	Defines operation <code>polyvecl_highbits</code> , a total mathematical function or abbreviation used by later declarations.
466	<code>mkseq (fun i => poly_highbits (nth poly_zero xs i)) (mode_l md).</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
467	<code>op polyvecl_lowbits (md : mode) (xs : polyvecl) : polyvecl =</code>	Defines operation <code>polyvecl_lowbits</code> , a total mathematical function or abbreviation used by later declarations.
468	<code>mkseq (fun i => poly_lowbits (nth poly_zero xs i)) (mode_l md).</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
469	<code>op polyveck_highbits_hint (md : mode) (xs : polyveck) : polyveck =</code>	Defines operation <code>polyveck_highbits_hint</code> , a total mathematical function or abbreviation used by later declarations.
470	<code>mkseq (fun i => poly_hint_highbits md (nth poly_zero xs i)) (mode_k md).</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
471	<code>op polyveck_vk_lowbits (md : mode) (xs : polyveck) : polyveck =</code>	Defines operation <code>polyveck_vk_lowbits</code> , a total mathematical function or abbreviation used by later declarations.
472	<code>mkseq (fun i => poly_vk_lowbits (nth poly_zero xs i)) (mode_k md).</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.

Line	EasyCrypt source	Mathematical reading
473	op polyveck_vk_highbits (md : mode) (xs : polyveck) : polyveck =	Defines operation polyveck_vk_highbits, a total mathematical function or abbreviation used by later declarations.
474	mkseq (fun i => poly_vk_highbits (nth poly_zero xs i)) (mode_k md).	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
475	op polyveck_mul_alpha (md : mode) (xs : polyveck) : polyveck =	Defines operation polyveck_mul_alpha, a total mathematical function or abbreviation used by later declarations.
476	mkseq	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
477	(fun i =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
478	mkseq	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
479	(fun j => mode_alpha_hint md * poly_coeff (nth poly_zero xs i) j)	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
480	n)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
481	(mode_k md).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
482	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
483	lemma poly_highbits_wf p : poly_wf (poly_highbits p).	States checked lemma poly_highbits_wf. The following proof script must derive this proposition from prior definitions and lemmas.
484	proof. by rewrite /poly_wf /poly_highbits size_mkseq /n. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
485	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
486	lemma poly_lowbits_wf p : poly_wf (poly_lowbits p).	States checked lemma poly_lowbits_wf. The following proof script must derive this proposition from prior definitions and lemmas.
487	proof. by rewrite /poly_wf /poly_lowbits size_mkseq /n. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
488	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
489	lemma poly_lsb_wf p : poly_wf (poly_lsb p).	States checked lemma poly_lsb_wf. The following proof script must derive this proposition from prior definitions and lemmas.
490	proof. by rewrite /poly_wf /poly_lsb size_mkseq /n. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
491	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
492	lemma poly_vk_lowbits_wf p : poly_wf (poly_vk_lowbits p).	States checked lemma poly_vk_lowbits_wf. The following proof script must derive this proposition from prior definitions and lemmas.
493	proof. by rewrite /poly_wf /poly_vk_lowbits size_mkseq /n. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
494	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
495	lemma poly_vk_highbits_wf p : poly_wf (poly_vk_highbits p).	States checked lemma poly_vk_highbits_wf. The following proof script must derive this proposition from prior definitions and lemmas.
496	proof. by rewrite /poly_wf /poly_vk_highbits size_mkseq /n. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
497	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
498	lemma poly_compose_z1_wf hi lo : poly_wf (poly_compose_z1 hi lo).	States checked lemma poly_compose_z1_wf. The following proof script must derive this proposition from prior definitions and lemmas.
499	proof. by rewrite /poly_wf /poly_compose_z1 size_mkseq /n. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
500	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
501	lemma poly_hint_highbits_wf md p : poly_wf (poly_hint_highbits md p).	States checked lemma poly_hint_highbits_wf. The following proof script must derive this proposition from prior definitions and lemmas.
502	proof. by rewrite /poly_wf /poly_hint_highbits size_mkseq /n. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
503	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
504	lemma polyvecl_highbits_wf md xs :	States checked lemma polyvecl_highbits_wf. The following proof script must derive this proposition from prior definitions and lemmas.
505	polyvecl_wf md (polyvecl_highbits md xs).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
506	<code>proof.</code>	Begins the proof script for the preceding proposition.
507	<code>rewrite /polyvecl_wf /polyvecl_highbits size_mkseq.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
508	<code>split.</code>	Splits a conjunction or structured goal into independent proof obligations.
509	<code>+ by case md.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
510	<code>apply/allP=> p /mkseqP [i [_ ->]].</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
511	<code>by apply poly_highbits_wf.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
512	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
513	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
514	<code>lemma polyvecl_lowbits_wf md xs :</code>	States checked lemma <code>polyvecl_lowbits_wf</code> . The following proof script must derive this proposition from prior definitions and lemmas.
515	<code>polyvecl_wf md (polyvecl_lowbits md xs).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
516	<code>proof.</code>	Begins the proof script for the preceding proposition.
517	<code>rewrite /polyvecl_wf /polyvecl_lowbits size_mkseq.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
518	<code>split.</code>	Splits a conjunction or structured goal into independent proof obligations.
519	<code>+ by case md.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
520	<code>apply/allP=> p /mkseqP [i [_ ->]].</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
521	<code>by apply poly_lowbits_wf.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
522	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
523	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
524	<code>lemma polyveck_highbits_hint_wf md xs :</code>	States checked lemma <code>polyveck_highbits_hint_wf</code> . The following proof script must derive this proposition from prior definitions and lemmas.
525	<code>polyveck_wf md (polyveck_highbits_hint md xs).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
526	<code>proof.</code>	Begins the proof script for the preceding proposition.
527	<code>rewrite /polyveck_wf /polyveck_highbits_hint size_mkseq.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
528	<code>split.</code>	Splits a conjunction or structured goal into independent proof obligations.
529	<code>+ by case md.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
530	<code>apply/allP=> p /mkseqP [i [_ ->]].</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
531	<code>by apply poly_hint_highbits_wf.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
532	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
533	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
534	<code>lemma polyveck_vk_lowbits_wf md xs :</code>	States checked lemma <code>polyveck_vk_lowbits_wf</code> . The following proof script must derive this proposition from prior definitions and lemmas.
535	<code>polyveck_wf md (polyveck_vk_lowbits md xs).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
536	<code>proof.</code>	Begins the proof script for the preceding proposition.
537	<code>rewrite /polyveck_wf /polyveck_vk_lowbits size_mkseq.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
538	<code>split.</code>	Splits a conjunction or structured goal into independent proof obligations.
539	<code>+ by case md.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
540	<code>apply/allP=> p /mkseqP [i [_ ->]].</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
541	<code>by apply poly_vk_lowbits_wf.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
542	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
543	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
544	<code>lemma polyveck_vk_highbits_wf md xs :</code>	States checked lemma <code>polyveck_vk_highbits_wf</code> . The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
545	<code>polyveck_wf md (polyveck_vk_highbits md xs).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
546	<code>proof.</code>	Begins the proof script for the preceding proposition.
547	<code>rewrite /polyveck_wf /polyveck_vk_highbits size_mkseq.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
548	<code>split.</code>	Splits a conjunction or structured goal into independent proof obligations.
549	<code>+ by case md.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
550	<code>apply/allP=> p /mkseqP [i [_ ->]].</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
551	<code>by apply poly_vk_highbits_wf.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
552	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
553	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
554	<code>lemma polyveck_mul_alpha_wf md xs :</code>	States checked lemma <code>polyveck_mul_alpha_wf</code> . The following proof script must derive this proposition from prior definitions and lemmas.
555	<code>polyveck_wf md (polyveck_mul_alpha md xs).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
556	<code>proof.</code>	Begins the proof script for the preceding proposition.
557	<code>rewrite /polyveck_wf /polyveck_mul_alpha size_mkseq.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
558	<code>split.</code>	Splits a conjunction or structured goal into independent proof obligations.
559	<code>+ by case md.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
560	<code>apply/allP=> p /mkseqP [i [_ ->]].</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
561	<code>by rewrite /poly_wf size_mkseq /n.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
562	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
563	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
564	<code>op public_key_seed_poly (tag : int) (src : byte list) : poly =</code>	Defines integer-valued parameter <code>public_key_seed_poly</code> , used for dimensions, budgets, support sizes, or bounds.
565	<code>mkseq (fun i => coeff_mod (tag + i + nth 0 src i)) n.</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
566	<code>op public_key_seed_polyvecl</code>	Defines operation <code>public_key_seed_polyvecl</code> , a total mathematical function or abbreviation used by later declarations.
567	<code>(md : mode) (tag : int) (src : byte list) : polyvecl =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
568	<code>mkseq</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
569	<code>(fun i => public_key_seed_poly (tag + i) (i :: src))</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
570	<code>(mode_l md).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
571	<code>op public_key_seed_polyveck</code>	Defines operation <code>public_key_seed_polyveck</code> , a total mathematical function or abbreviation used by later declarations.
572	<code>(md : mode) (tag : int) (src : byte list) : polyveck =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
573	<code>mkseq</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
574	<code>(fun i => public_key_seed_poly (tag + i) (i :: src))</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
575	<code>(mode_k md).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
576	<code>op haetae_keygen_seedbuf_bytes : int = 2 * seedbytes + crhbytes.</code>	Defines integer-valued parameter <code>haetae_keygen_seedbuf_bytes</code> , used for dimensions, budgets, support sizes, or bounds.
577	<code>op haetae_reference_keygen_seedbuf_bytes : int =</code>	Defines integer-valued parameter <code>haetae_reference_keygen_seedbuf_bytes</code> , used for dimensions, budgets, support sizes, or bounds.
578	<code>2 * seedbytes + crhbytes.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
579	<code>op haetae_reference_keygen_xof_absorb_bytes : int = seedbytes.</code>	Defines integer-valued parameter <code>haetae_reference_keygen_xof_absorb_bytes</code> , used for dimensions, budgets, support sizes, or bounds.

Line	EasyCrypt source	Mathematical reading
580	op haetae_reference_keygen_xof_squeeze_bytes : int =	Defines integer-valued parameter haetae_reference_keygen_xof_squeeze_bytes, used for dimensions, budgets, support sizes, or bounds.
581	haetae_reference_keygen_seedbuf_bytes.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
582	op haetae_keygen_rhopprime_offset : int = 0.	Defines integer-valued parameter haetae_keygen_rhopprime_offset, used for dimensions, budgets, support sizes, or bounds.
583	op haetae_keygen_sigma_offset : int = seedbytes.	Defines integer-valued parameter haetae_keygen_sigma_offset, used for dimensions, budgets, support sizes, or bounds.
584	op haetae_keygen_key_offset : int = seedbytes + crhbytes.	Defines integer-valued parameter haetae_keygen_key_offset, used for dimensions, budgets, support sizes, or bounds.
585	op haetae_shake256_rate_bytes : int = shake256_rate_bytes.	Defines integer-valued parameter haetae_shake256_rate_bytes, used for dimensions, budgets, support sizes, or bounds.
586	op haetae_shake256_domain_separator : byte = shake256_domain_separator.	Defines operation haetae_shake256_domain_separator, a total mathematical function or abbreviation used by later declarations.
587	op haetae_shake256_bytes (input : byte list) (outlen : int) : byte list =	Defines integer-valued parameter haetae_shake256_bytes, used for dimensions, budgets, support sizes, or bounds.
588	shake256 input (size input) outlen.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
589	op haetae_seed_slice (src : byte list) (offset len : int) : byte list =	Defines integer-valued parameter haetae_seed_slice, used for dimensions, budgets, support sizes, or bounds.
590	mkseq (fun i => nth 0 src (offset + i)) len.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
591	op haetae_xof256_absorb_input	Defines operation haetae_xof256_absorb_input, a total mathematical function or abbreviation used by later declarations.
592	(input : byte list) (inlen : int) : byte list =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
593	haetae_seed_slice input 0 inlen.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
594	op haetae_xof256_absorb_once	Defines operation haetae_xof256_absorb_once, a total mathematical function or abbreviation used by later declarations.
595	(input : byte list) (inlen : int) : xof256_state =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
596	shake256_absorb_once (haetae_xof256_absorb_input input inlen) inlen.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
597	op haetae_xof256_squeeze	Defines operation haetae_xof256_squeeze, a total mathematical function or abbreviation used by later declarations.
598	(st : xof256_state) (outlen : int) : byte list =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
599	shake256_squeeze st outlen.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
600	op haetae_xof256_absorb_once_squeeze	Defines operation haetae_xof256_absorb_once_squeeze, a total mathematical function or abbreviation used by later declarations.
601	(input : byte list) (inlen outlen : int) : byte list =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
602	haetae_xof256_squeeze	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
603	(haetae_xof256_absorb_once input inlen)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
604	outlen.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
605	op haetae_reference_keygen_seedbuf_after_memcpy (sd : seed) : byte list =	Defines operation haetae_reference_keygen_seedbuf_after_memcpy, a total mathematical function or abbreviation used by later declarations.
606	mkseq	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
607	(fun i => if i < seedbytes then nth 0 sd i else 0)	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
608	haetae_reference_keygen_seedbuf_bytes.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
609	op haetae_reference_keygen_xof_input (sd : seed) : byte list =	Defines operation haetae_reference_keygen_xof_input, a total mathematical function or abbreviation used by later declarations.
610	haetae_seed_slice	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
611	(haetae_reference_keygen_seedbuf_after_memcpy sd)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
612	0 haetae_reference_keygen_xof_absorb_bytes.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
613	op haetae_keygen_xof_seedbuf (sd : seed) : byte list =	Defines operation haetae_keygen_xof_seedbuf, a total mathematical function or abbreviation used by later declarations.
614	haetae_xof256_absorb_once_squeeze	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
615	(haetae_reference_keygen_seedbuf_after_memcpy sd)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
616	haetae_reference_keygen_xof_absorb_bytes	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
617	haetae_reference_keygen_xof_squeeze_bytes.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
618	op haetae_reference_keygen_xof256_seedbuf (sd : seed) : byte list =	Defines operation haetae_reference_keygen_xof256_seedbuf, a total mathematical function or abbreviation used by later declarations.
619	haetae_keygen_xof_seedbuf sd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
620	op haetae_keygen_seedbuf_rhoprime (seedbuf : byte list) : seed =	Defines operation haetae_keygen_seedbuf_rhoprime, a total mathematical function or abbreviation used by later declarations.
621	haetae_seed_slice seedbuf haetae_keygen_rhoprime_offset seedbytes.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
622	op haetae_keygen_seedbuf_sigma (seedbuf : byte list) : crh =	Defines operation haetae_keygen_seedbuf_sigma, a total mathematical function or abbreviation used by later declarations.
623	haetae_seed_slice seedbuf haetae_keygen_sigma_offset crhbytes.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
624	op haetae_keygen_seedbuf_key (seedbuf : byte list) : seed =	Defines operation haetae_keygen_seedbuf_key, a total mathematical function or abbreviation used by later declarations.
625	haetae_seed_slice seedbuf haetae_keygen_key_offset seedbytes.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
626	op haetae_keygen_seedbuf_layout_wf (seedbuf : byte list) : bool =	Defines predicate haetae_keygen_seedbuf_layout_wf, an event or well-formedness condition used in later implications and game proofs.
627	size seedbuf = haetae_reference_keygen_seedbuf_bytes /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
628	haetae_keygen_rhoprime_offset = 0 /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
629	haetae_keygen_sigma_offset = seedbytes /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
630	haetae_keygen_key_offset = seedbytes + crhbytes /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
631	size (haetae_keygen_seedbuf_rhoprime seedbuf) = seedbytes /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
632	size (haetae_keygen_seedbuf_sigma seedbuf) = crhbytes /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
633	size (haetae_keygen_seedbuf_key seedbuf) = seedbytes.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
634	op haetae_reference_keygen_xof_call_wf	Defines operation haetae_reference_keygen_xof_call_wf, a total mathematical function or abbreviation used by later declarations.
635	(sd : seed) (seedbuf : byte list) : bool =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
636	size sd = haetae_reference_keygen_xof_absorb_bytes /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
637	size (haetae_reference_keygen_seedbuf_after_memcpy sd) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
638	haetae_reference_keygen_seedbuf_bytes /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
639	haetae_xof256_absorb_input	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
640	(haetae_reference_keygen_seedbuf_after_memcpy sd)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
641	haetae_reference_keygen_xof_absorb_bytes =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
642	haetae_reference_keygen_xof_input sd /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
643	shake256_absorb_once_short_domain	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
644	(haetae_reference_keygen_xof_input sd)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
645	haetae_reference_keygen_xof_absorb_bytes /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
646	haetae_reference_keygen_xof_squeeze_bytes < haetae_shake256_rate_bytes /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
647	haetae_xof256_absorb_once	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
648	(haetae_reference_keygen_seedbuf_after_memcpy sd)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
649	haetae_reference_keygen_xof_absorb_bytes =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
650	shake256_absorb_once	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
651	(haetae_reference_keygen_xof_input sd)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
652	haetae_reference_keygen_xof_absorb_bytes /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
653	seedbuf =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
654	shake256_squeeze	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
655	(haetae_xof256_absorb_once	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
656	(haetae_reference_keygen_seedbuf_after_memcpy sd)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
657	haetae_reference_keygen_xof_absorb_bytes)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
658	haetae_reference_keygen_xof_squeeze_bytes /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
659	size seedbuf = haetae_reference_keygen_xof_squeeze_bytes /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
660	haetae_reference_keygen_xof_absorb_bytes = seedbytes /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
661	haetae_reference_keygen_xof_squeeze_bytes =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
662	haetae_reference_keygen_seedbuf_bytes.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
663	op haetae_keygen_rho_prime (sd : seed) : seed =	Defines operation haetae_keygen_rho_prime, a total mathematical function or abbreviation used by later declarations.
664	haetae_keygen_seedbuf_rho_prime	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
665	(haetae_reference_keygen_xof256_seedbuf sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
666	op haetae_keygen_sigma (sd : seed) : crh =	Defines operation haetae_keygen_sigma, a total mathematical function or abbreviation used by later declarations.
667	haetae_keygen_seedbuf_sigma	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
668	(haetae_reference_keygen_xof256_seedbuf sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
669	op haetae_keygen_key (sd : seed) : seed =	Defines operation haetae_keygen_key, a total mathematical function or abbreviation used by later declarations.
670	haetae_keygen_seedbuf_key	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
671	(haetae_reference_keygen_xof256_seedbuf sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
672	op haetae_keygen_counter_next (md : mode) (counter : int) : int =	Defines integer-valued parameter haetae_keygen_counter_next, used for dimensions, budgets, support sizes, or bounds.
673	counter + mode_m md + mode_k md.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
674	op haetae_keygen_first_accept_counter (_ : mode) (_ : seed) : int = 0.	Defines integer-valued parameter haetae_keygen_first_accept_counter, used for dimensions, budgets, support sizes, or bounds.
675	op haetae_keygen_sampler_seed (sigma : crh) (counter : int) : seed =	Defines integer-valued parameter haetae_keygen_sampler_seed, used for dimensions, budgets, support sizes, or bounds.
676	mkseq	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
677	(fun i => nth 0 sigma (i %% crhbytes) + counter + i)	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
678	seedbytes.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
679	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
680	<code>op public_matrix_seed (md : mode) (sd : seed) : matrix =</code>	Defines operation <code>public_matrix_seed</code> , a total mathematical function or abbreviation used by later declarations.
681	<code>mkseq</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
682	<code>(fun i => public_key_seed_polyvecl md (101 + i) (i :: sd))</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
683	<code>(mode_k md).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
684	<code>op public_secret_vector_seed (md : mode) (sd : seed) : polyvecl =</code>	Defines operation <code>public_secret_vector_seed</code> , a total mathematical function or abbreviation used by later declarations.
685	<code>public_key_seed_polyvecl md 211 sd.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
686	<code>op public_error_vector_seed (md : mode) (sd : seed) : polyveck =</code>	Defines operation <code>public_error_vector_seed</code> , a total mathematical function or abbreviation used by later declarations.
687	<code>public_key_seed_polyveck md 307 sd.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
688	<code>op public_rounding_vector_seed (md : mode) (sd : seed) : polyveck =</code>	Defines operation <code>public_rounding_vector_seed</code> , a total mathematical function or abbreviation used by later declarations.
689	<code>public_key_seed_polyveck md 401 sd.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
690	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
691	<code>op haetae_keygen_a0_matrix (md : mode) (sd : seed) : matrix =</code>	Defines operation <code>haetae_keygen_a0_matrix</code> , a total mathematical function or abbreviation used by later declarations.
692	<code>public_matrix_seed md sd.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
693	<code>op haetae_keygen_secret_vector (md : mode) (sd : seed) : polyvecl =</code>	Defines operation <code>haetae_keygen_secret_vector</code> , a total mathematical function or abbreviation used by later declarations.
694	<code>public_secret_vector_seed md sd.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
695	<code>op haetae_keygen_error_vector (md : mode) (sd : seed) : polyveck =</code>	Defines operation <code>haetae_keygen_error_vector</code> , a total mathematical function or abbreviation used by later declarations.
696	<code>public_error_vector_seed md sd.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
697	<code>op haetae_keygen_raw_public_vector (md : mode) (sd : seed) : polyveck =</code>	Defines operation <code>haetae_keygen_raw_public_vector</code> , a total mathematical function or abbreviation used by later declarations.
698	<code>polyveck_add md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
699	<code>(matrix_vec_mul md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
700	<code>(haetae_keygen_a0_matrix md sd)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
701	<code>(haetae_keygen_secret_vector md sd))</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
702	<code>(haetae_keygen_error_vector md sd).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
703	<code>op haetae_keygen_mode23_predecompose_vector</code>	Defines operation <code>haetae_keygen_mode23_predecompose_vector</code> , a total mathematical function or abbreviation used by later declarations.
704	<code>(md : mode) (sd : seed) : polyveck =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
705	<code>polyveck_add md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
706	<code>(public_rounding_vector_seed md sd)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
707	<code>(haetae_keygen_raw_public_vector md sd).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
708	<code>op haetae_keygen_mode23_public_vector</code>	Defines operation <code>haetae_keygen_mode23_public_vector</code> , a total mathematical function or abbreviation used by later declarations.
709	<code>(md : mode) (sd : seed) : polyveck =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
710	<code>polyveck_vk_highbits md (haetae_keygen_mode23_predecompose_vector md sd).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
711	<code>op haetae_keygen_mode23_rounding_lowbits</code>	Defines operation <code>haetae_keygen_mode23_rounding_lowbits</code> , a total mathematical function or abbreviation used by later declarations.

Line	EasyCrypt source	Mathematical reading
712	(md : mode) (sd : seed) : polyveck =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
713	polyveck_vk_lowbits md (haetae_keygen_mode23_predecompose_vector md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
714	op haetae_keygen_mode23_adjusted_error_vector	Defines operation haetae_keygen_mode23_adjusted_error_vector, a total mathematical function or abbreviation used by later declarations.
715	(md : mode) (sd : seed) : polyveck =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
716	polyveck_sub md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
717	(haetae_keygen_error_vector md sd)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
718	(haetae_keygen_mode23_rounding_lowbits md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
719	op haetae_keygen_mode5_public_vector	Defines operation haetae_keygen_mode5_public_vector, a total mathematical function or abbreviation used by later declarations.
720	(md : mode) (sd : seed) : polyveck =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
721	polyveck_neg md (polyveck_double md (haetae_keygen_raw_public_vector md sd)).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
722	op haetae_public_key_vector_from_relation	Defines operation haetae_public_key_vector_from_relation, a total mathematical function or abbreviation used by later declarations.
723	(md : mode) (sd : seed) (s : polyveck1) (e : polyveck) : polyveck =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
724	with md = Mode2 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
725	let raw =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
726	polyveck_add md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
727	(matrix_vec_mul md (public_matrix_seed md sd) s)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
728	e in	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
729	polyveck_vk_highbits md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
730	(polyveck_add md (public_rounding_vector_seed md sd) raw)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
731	with md = Mode3 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
732	let raw =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
733	polyveck_add md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
734	(matrix_vec_mul md (public_matrix_seed md sd) s)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
735	e in	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
736	polyveck_vk_highbits md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
737	(polyveck_add md (public_rounding_vector_seed md sd) raw)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
738	with md = Mode5 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
739	let raw =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
740	polyveck_add md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
741	(matrix_vec_mul md (public_matrix_seed md sd) s)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
742	e in	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
743	polyveck_neg md (polyveck_double md raw).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
744	op haetae_keygen_public_vector (md : mode) (sd : seed) : polyveck =	Defines operation haetae_keygen_public_vector, a total mathematical function or abbreviation used by later declarations.

Line	EasyCrypt source	Mathematical reading
745	haetae_public_key_vector_from_relation md sd	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
746	(haetae_keygen_secret_vector md sd)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
747	(haetae_keygen_error_vector md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
748	op public_key_vector_seed (md : mode) (sd : seed) : polyveck =	Defines operation public_key_vector_seed, a total mathematical function or abbreviation used by later declarations.
749	haetae_keygen_public_vector md sd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
750	op public_key_relation	Defines operation public_key_relation, a total mathematical function or abbreviation used by later declarations.
751	(md : mode) (pk : pkey) (s : polyvecl) (e : polyveck) : bool =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
752	pk.'2 = haetae_public_key_vector_from_relation md pk.'1 s e.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
753	op public_key_equation_holds (md : mode) (pk : pkey) : bool =	Defines predicate public_key_equation_holds, an event or well-formedness condition used in later implications and game proofs.
754	public_key_relation md pk	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
755	(public_secret_vector_seed md pk.'1)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
756	(public_error_vector_seed md pk.'1).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
757	op haetae_reference_polymatkm_expand_matA	Defines operation haetae_reference_polymatkm_expand_matA, a total mathematical function or abbreviation used by later declarations.
758	(md : mode) (rho_prime : seed) : matrix =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
759	public_matrix_seed md rho_prime.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
760	op haetae_reference_polyveck_expand_vecA	Defines operation haetae_reference_polyveck_expand_vecA, a total mathematical function or abbreviation used by later declarations.
761	(md : mode) (rho_prime : seed) : polyveck =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
762	public_rounding_vector_seed md rho_prime.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
763	op haetae_reference_polyvecmk_expand_S_s1	Defines operation haetae_reference_polyvecmk_expand_S_s1, a total mathematical function or abbreviation used by later declarations.
764	(md : mode) (sigma : crh) (counter : int) : polyvecl =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
765	public_secret_vector_seed md (haetae_keygen_sampler_seed sigma counter).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
766	op haetae_reference_polyvecmk_expand_S_s2	Defines operation haetae_reference_polyvecmk_expand_S_s2, a total mathematical function or abbreviation used by later declarations.
767	(md : mode) (sigma : crh) (counter : int) : polyveck =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
768	public_error_vector_seed md (haetae_keygen_sampler_seed sigma counter).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
769	op haetae_reference_keygen_secret_vector	Defines operation haetae_reference_keygen_secret_vector, a total mathematical function or abbreviation used by later declarations.
770	(md : mode) (sd : seed) : polyvecl =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
771	haetae_reference_polyvecmk_expand_S_s1 md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
772	(haetae_keygen_sigma sd)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
773	(haetae_keygen_first_accept_counter md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
774	op haetae_reference_keygen_error_vector	Defines operation haetae_reference_keygen_error_vector, a total mathematical function or abbreviation used by later declarations.
775	(md : mode) (sd : seed) : polyveck =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
776	haetae_reference_polyvecmk_expand_S_s2 md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
777	(haetae_keygen_sigma sd)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
778	(haetae_keygen_first_accept_counter md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
779	op haetae_reference_keygen_a0_matrix	Defines operation haetae_reference_keygen_a0_matrix, a total mathematical function or abbreviation used by later declarations.
780	(md : mode) (sd : seed) : matrix =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
781	haetae_reference_polymatkm_expand_matA md (haetae_keygen_rhoprime sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
782	op haetae_reference_keygen_raw_public_vector	Defines operation haetae_reference_keygen_raw_public_vector, a total mathematical function or abbreviation used by later declarations.
783	(md : mode) (sd : seed) : polyveck =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
784	polyveck_add md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
785	(matrix_vec_mul md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
786	(haetae_reference_keygen_a0_matrix md sd)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
787	(haetae_reference_keygen_secret_vector md sd))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
788	(haetae_reference_keygen_error_vector md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
789	op haetae_reference_keygen_mode23_predecompose_vector	Defines operation haetae_reference_keygen_mode23_predecompose_vector, a total mathematical function or abbreviation used by later declarations.
790	(md : mode) (sd : seed) : polyveck =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
791	polyveck_add md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
792	(haetae_reference_polyveck_expand_vecA md (haetae_keygen_rhoprime sd))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
793	(haetae_reference_keygen_raw_public_vector md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
794	op haetae_reference_keygen_mode23_public_vector	Defines operation haetae_reference_keygen_mode23_public_vector, a total mathematical function or abbreviation used by later declarations.
795	(md : mode) (sd : seed) : polyveck =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
796	polyveck_vk_highbits md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
797	(haetae_reference_keygen_mode23_predecompose_vector md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
798	op haetae_reference_keygen_mode23_rounding_lowbits	Defines operation haetae_reference_keygen_mode23_rounding_lowbits, a total mathematical function or abbreviation used by later declarations.
799	(md : mode) (sd : seed) : polyveck =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
800	polyveck_vk_lowbits md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
801	(haetae_reference_keygen_mode23_predecompose_vector md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
802	op haetae_reference_keygen_mode23_adjusted_error_vector	Defines operation haetae_reference_keygen_mode23_adjusted_error_vector, a total mathematical function or abbreviation used by later declarations.
803	(md : mode) (sd : seed) : polyveck =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
804	polyveck_sub md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
805	(haetae_reference_keygen_error_vector md sd)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
806	(haetae_reference_keygen_mode23_rounding_lowbits md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
807	op haetae_reference_keygen_mode5_public_vector	Defines operation haetae_reference_keygen_mode5_public_vector, a total mathematical function or abbreviation used by later declarations.
808	(md : mode) (sd : seed) : polyveck =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
809	polyveck_neg md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
810	(polyveck_double md (haetae_reference_keygen_raw_public_vector md sd)).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
811	op haetae_reference_keygen_public_vector	Defines operation <code>haetae_reference_keygen_public_vector</code> , a total mathematical function or abbreviation used by later declarations.
812	(md : mode) (sd : seed) : polyveck =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
813	haetae_public_key_vector_from_relation md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
814	(haetae_keygen_rho_prime sd)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
815	(haetae_reference_keygen_secret_vector md sd)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
816	(haetae_reference_keygen_error_vector md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
817	op packed_haetae_reference_public_key	Defines operation <code>packed_haetae_reference_public_key</code> , a total mathematical function or abbreviation used by later declarations.
818	(md : mode) (sd : seed) : pkey =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
819	(haetae_keygen_rho_prime sd, haetae_reference_keygen_public_vector md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
820	op haetae_reference_keygen_seed_flow_wf (md : mode) (sd : seed) : bool =	Defines predicate <code>haetae_reference_keygen_seed_flow_wf</code> , an event or well-formedness condition used in later implications and game proofs.
821	haetae_reference_keygen_a0_matrix md sd =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
822	haetae_reference_polymatkm_expand_matA md (haetae_keygen_rho_prime sd) /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
823	haetae_reference_keygen_secret_vector md sd =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
824	haetae_reference_polyvecmk_expand_S_s1 md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
825	(haetae_keygen_sigma sd)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
826	(haetae_keygen_first_accept_counter md sd) /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
827	haetae_reference_keygen_error_vector md sd =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
828	haetae_reference_polyvecmk_expand_S_s2 md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
829	(haetae_keygen_sigma sd)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
830	(haetae_keygen_first_accept_counter md sd) /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
831	haetae_reference_keygen_public_vector md sd =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
832	haetae_public_key_vector_from_relation md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
833	(haetae_keygen_rho_prime sd)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
834	(haetae_reference_keygen_secret_vector md sd)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
835	(haetae_reference_keygen_error_vector md sd) /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
836	(packed_haetae_reference_public_key md sd).‘1 =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
837	haetae_keygen_rho_prime sd /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
838	(packed_haetae_reference_public_key md sd).‘2 =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
839	haetae_reference_keygen_public_vector md sd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
840	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
841	op haetae_mode23_qj_vector (md : mode) (sd : seed) : polyveck =	Defines operation <code>haetae_mode23_qj_vector</code> , a total mathematical function or abbreviation used by later declarations.
842	public_rounding_vector_seed md sd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
843	op haetae_mode23_verification_column	Defines operation <code>haetae_mode23_verification_column</code> , a total mathematical function or abbreviation used by later declarations.

Line	EasyCrypt source	Mathematical reading
844	(md : mode) (b qj : polyveck) : polyveck =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
845	polyveck_double md (polyveck_sub md qj (polyveck_double md b)).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
846	op haetae_mode5_verification_column	Defines operation haetae_mode5_verification_column, a total mathematical function or abbreviation used by later declarations.
847	(md : mode) (b : polyveck) : polyveck =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
848	mkseq (fun i => poly_normalize (nth poly_zero b i)) (mode_k md).	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
849	op public_key_mode23_column (md : mode) (pk : pkey) : polyveck =	Defines operation public_key_mode23_column, a total mathematical function or abbreviation used by later declarations.
850	haetae_mode23_verification_column md pk.'2	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
851	(haetae_mode23_qj_vector md pk.'1).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
852	op public_key_mode5_column (md : mode) (pk : pkey) : polyveck =	Defines operation public_key_mode5_column, a total mathematical function or abbreviation used by later declarations.
853	haetae_mode5_verification_column md pk.'2.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
854	op public_key_verification_column (md : mode) (pk : pkey) : polyveck =	Defines operation public_key_verification_column, a total mathematical function or abbreviation used by later declarations.
855	with md = Mode2 => public_key_mode23_column md pk	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
856	with md = Mode3 => public_key_mode23_column md pk	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
857	with md = Mode5 => public_key_mode5_column md pk.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
858	op haetae_verification_column_from_unpacked	Defines operation haetae_verification_column_from_unpacked, a total mathematical function or abbreviation used by later declarations.
859	(md : mode) (sd : seed) (b : polyveck) : polyveck =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
860	with md = Mode2 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
861	haetae_mode23_verification_column md b	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
862	(haetae_mode23_qj_vector md sd)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
863	with md = Mode3 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
864	haetae_mode23_verification_column md b	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
865	(haetae_mode23_qj_vector md sd)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
866	with md = Mode5 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
867	haetae_mode5_verification_column md b.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
868	op haetae_verification_matrix_from_unpacked	Defines operation haetae_verification_matrix_from_unpacked, a total mathematical function or abbreviation used by later declarations.
869	(md : mode) (sd : seed) (b : polyveck) : matrix =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
870	mkseq	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
871	(fun i =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
872	mkseq	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
873	(fun j =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
874	if j = 0	Splits proof obligations according to a conditional branch in the program or formula.
875	then nth poly_zero	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
876	(haetae_verification_column_from_unpacked md sd b) i	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
877	else poly_double	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
878	(nth poly_zero	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
879	(nth [] (haetae_keygen_a0_matrix md sd) i)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
880	(j - 1)))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
881	(mode_l md))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
882	(mode_k md).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
883	op packed_haetae_public_key (md : mode) (sd : seed) : pkey =	Defines operation packed_haetae_public_key, a total mathematical function or abbreviation used by later declarations.
884	(sd, haetae_keygen_public_vector md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
885	op packed_haetae_verification_matrix (md : mode) (pk : pkey) : matrix =	Defines operation packed_haetae_verification_matrix, a total mathematical function or abbreviation used by later declarations.
886	haetae_verification_matrix_from_unpacked md pk.'1 pk.'2.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
887	op packed_haetae_keygen_verification_matrix (md : mode) (sd : seed) :	Defines operation packed_haetae_keygen_verification_matrix, a total mathematical function or abbreviation used by later declarations.
888	matrix =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
889	packed_haetae_verification_matrix md (packed_haetae_public_key md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
890	op public_verification_matrix (md : mode) (pk : pkey) : matrix =	Defines operation public_verification_matrix, a total mathematical function or abbreviation used by later declarations.
891	packed_haetae_verification_matrix md pk.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
892	op challenge_embedding (md : mode) (ch : challenge) : polyvec1 =	Defines operation challenge_embedding, a total mathematical function or abbreviation used by later declarations.
893	mkseq (fun i => if i = 0 then ch else poly_zero) (mode_l md).	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
894	op public_key_challenge_term	Defines operation public_key_challenge_term, a total mathematical function or abbreviation used by later declarations.
895	(md : mode) (pk : pkey) (ch : challenge) : polyveck =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
896	matrix_vec_mul md (public_verification_matrix md pk)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
897	(challenge_embedding md ch).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
898	op reconstructed_highbits	Defines operation reconstructed_highbits, a total mathematical function or abbreviation used by later declarations.
899	(md : mode) (aux_pk_ch : polyveck) : polyveck =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
900	polyveck_add md aux_pk_ch.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
901	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
902	op public_key_of_secret (md : mode) (sk : skey) : pkey =	Defines operation public_key_of_secret, a total mathematical function or abbreviation used by later declarations.
903	(sk.'1, public_key_vector_seed md sk.'1).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
904	op haetae_public_key_body_bytes (md : mode) : int =	Defines integer-valued parameter haetae_public_key_body_bytes, used for dimensions, budgets, support sizes, or bounds.
905	mode_k md * mode_polyq_packedbytes md.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
906	op haetae_public_key_bytes (md : mode) : int =	Defines integer-valued parameter haetae_public_key_bytes, used for dimensions, budgets, support sizes, or bounds.
907	mode_publickeybytes md.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
908	op haetae_public_key_encoding_wf (md : mode) (enc : encoded_pkey) : bool =	Defines predicate haetae_public_key_encoding_wf, an event or well-formedness condition used in later implications and game proofs.
909	size enc = haetae_public_key_bytes md.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.

Line	EasyCrypt source	Mathematical reading
910	op haetae_public_key_unpacked_wf (md : mode) (pk : pkey) : bool =	Defines predicate haetae_public_key_unpacked_wf, an event or well-formedness condition used in later implications and game proofs.
911	size pk.'1 = seedbytes /\ polyveck_wf md pk.'2.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
912	op haetae_pack_seed (sd : seed) : byte list =	Defines operation haetae_pack_seed, a total mathematical function or abbreviation used by later declarations.
913	mkseq (fun i => nth 0 sd i) seedbytes.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
914	op haetae_unpack_seed (enc : encoded_pkey) : seed =	Defines operation haetae_unpack_seed, a total mathematical function or abbreviation used by later declarations.
915	mkseq (fun i => nth 0 enc i) seedbytes.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
916	op haetae_pack_poly_q (md : mode) (p : poly) : byte list =	Defines operation haetae_pack_poly_q, a total mathematical function or abbreviation used by later declarations.
917	mkseq	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
918	(fun i => if i < n then poly_coeff p i else 0)	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
919	(mode_polyq_packedbytes md).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
920	op haetae_unpack_poly_q_at	Defines operation haetae_unpack_poly_q_at, a total mathematical function or abbreviation used by later declarations.
921	(md : mode) (enc : encoded_pkey) (offset : int) : poly =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
922	mkseq (fun i => nth 0 enc (offset + i)) n.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
923	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
924	op haetae_byte_modulus : int = 256.	Defines integer-valued parameter haetae_byte_modulus, used for dimensions, budgets, support sizes, or bounds.
925	op haetae_polyq_mode23_bound : int = 32768.	Defines integer-valued parameter haetae_polyq_mode23_bound, used for dimensions, budgets, support sizes, or bounds.
926	op haetae_polyq_mode5_bound : int = 65536.	Defines integer-valued parameter haetae_polyq_mode5_bound, used for dimensions, budgets, support sizes, or bounds.
927	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
928	op haetae_byte_norm (x : int) : byte = x %% haetae_byte_modulus.	Defines integer-valued parameter haetae_byte_norm, used for dimensions, budgets, support sizes, or bounds.
929	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
930	op haetae_polyq_coeff_bound (md : mode) : int =	Defines integer-valued parameter haetae_polyq_coeff_bound, used for dimensions, budgets, support sizes, or bounds.
931	with md = Mode2 => haetae_polyq_mode23_bound	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
932	with md = Mode3 => haetae_polyq_mode23_bound	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
933	with md = Mode5 => haetae_polyq_mode5_bound.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
934	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
935	op haetae_polyq_coeffs_wf (md : mode) (p : poly) : bool =	Defines predicate haetae_polyq_coeffs_wf, an event or well-formedness condition used in later implications and game proofs.
936	poly_wf p /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
937	forall i, 0 <= i < n =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
938	0 <= poly_coeff p i < haetae_polyq_coeff_bound md.	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
939	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
940	op haetae_polyveck_polyq_coeffs_wf	Defines operation haetae_polyveck_polyq_coeffs_wf, a total mathematical function or abbreviation used by later declarations.
941	(md : mode) (b : polyveck) : bool =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
942	polyveck_wf md b /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
943	<code>forall row, 0 <= row < mode_k md =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
944	<code>haetae_polyq_coeffs_wf md (nth poly_zero b row).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
945	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
946	<code>lemma coeff_decompose_hint_high_polyq_bound md x :</code>	States checked lemma <code>coeff_decompose_hint_high_polyq_bound</code> . The following proof script must derive this proposition from prior definitions and lemmas.
947	<code>coeff_q_bound x =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
948	<code>0 <= coeff_decompose_hint_high md x < haetae_polyq_coeff_bound md.</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
949	<code>proof.</code>	Begins the proof script for the preceding proposition.
950	<code>case: md; rewrite /coeff_q_bound /coeff_decompose_hint_high</code>	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
951	<code>/mode_alpha_hint /haetae_polyq_coeff_bound</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
952	<code>/haetae_polyq_mode23_bound /haetae_polyq_mode5_bound</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
953	<code>/dq /q; smt().</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
954	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
955	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
956	<code>lemma poly_hint_highbits_polyq_wf md p :</code>	States checked lemma <code>poly_hint_highbits_polyq_wf</code> . The following proof script must derive this proposition from prior definitions and lemmas.
957	<code>poly_coeffs_q_bound p =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
958	<code>haetae_polyq_coeffs_wf md (poly_hint_highbits md p).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
959	<code>proof.</code>	Begins the proof script for the preceding proposition.
960	<code>move=> p_bd.</code>	Introduces universally quantified variables or hypotheses into the proof context.
961	<code>rewrite /haetae_polyq_coeffs_wf.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
962	<code>split.</code>	Splits a conjunction or structured goal into independent proof obligations.
963	<code>+ by apply poly_hint_highbits_wf.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
964	<code>move=> i i_rng.</code>	Introduces universally quantified variables or hypotheses into the proof context.
965	<code>rewrite /poly_hint_highbits /poly_coeff_nth_mkseq; first by smt().</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
966	<code>apply coeff_decompose_hint_high_polyq_bound.</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
967	<code>by apply p_bd.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
968	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
969	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
970	<code>lemma polyveck_highbits_hint_polyq_wf md xs :</code>	States checked lemma <code>polyveck_highbits_hint_polyq_wf</code> . The following proof script must derive this proposition from prior definitions and lemmas.
971	<code>polyveck_coeffs_q_bound md xs =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
972	<code>haetae_polyveck_polyq_coeffs_wf md (polyveck_highbits_hint md xs).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
973	<code>proof.</code>	Begins the proof script for the preceding proposition.
974	<code>move=> xs_bd.</code>	Introduces universally quantified variables or hypotheses into the proof context.
975	<code>rewrite /haetae_polyveck_polyq_coeffs_wf.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
976	<code>split.</code>	Splits a conjunction or structured goal into independent proof obligations.
977	<code>+ by apply polyveck_highbits_hint_wf.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
978	<code>move=> row row_rng.</code>	Introduces universally quantified variables or hypotheses into the proof context.

Line	EasyCrypt source	Mathematical reading
979	rewrite /polyveck_highbits_hint nth_mkseq; first by smt().	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
980	apply poly_hint_highbits_polyq_wf.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
981	by apply xs_bd.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
982	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
983	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
984	lemma coeff_decompose_vk_high_mode23_polyq_bound md x :	States checked lemma coeff_decompose_vk_high_mode23_polyq_bound. The following proof script must derive this proposition from prior definitions and lemmas.
985	(md = Mode2 \ / md = Mode3) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
986	coeff_q_bound x =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
987	0 <= coeff_decompose_vk_high x < haetae_polyq_coeff_bound md.	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
988	proof.	Begins the proof script for the preceding proposition.
989	case: md; rewrite /coeff_q_bound /coeff_decompose_vk_high	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
990	/coeff_decompose_vk_low /haetae_polyq_coeff_bound	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
991	/haetae_polyq_mode23_bound /haetae_polyq_mode5_bound	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
992	/q; smt().	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
993	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
994	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
995	lemma poly_vk_highbits_mode23_polyq_wf md p :	States checked lemma poly_vk_highbits_mode23_polyq_wf. The following proof script must derive this proposition from prior definitions and lemmas.
996	(md = Mode2 \ / md = Mode3) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
997	poly_coeffs_q_bound p =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
998	haetae_polyq_coeffs_wf md (poly_vk_highbits p).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
999	proof.	Begins the proof script for the preceding proposition.
1000	move=> md23 p_bd.	Introduces universally quantified variables or hypotheses into the proof context.
1001	rewrite /haetae_polyq_coeffs_wf.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1002	split.	Splits a conjunction or structured goal into independent proof obligations.
1003	+ by apply poly_vk_highbits_wf.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1004	move=> i i_rng.	Introduces universally quantified variables or hypotheses into the proof context.
1005	rewrite /poly_vk_highbits /poly_coeff nth_mkseq; first by smt().	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1006	apply coeff_decompose_vk_high_mode23_polyq_bound; first by apply md23.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1007	by apply p_bd.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1008	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1009	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1010	lemma polyveck_vk_highbits_mode23_polyq_wf md xs :	States checked lemma polyveck_vk_highbits_mode23_polyq_wf. The following proof script must derive this proposition from prior definitions and lemmas.
1011	(md = Mode2 \ / md = Mode3) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1012	polyveck_coeffs_q_bound md xs =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1013	haetae_polyveck_polyq_coeffs_wf md (polyveck_vk_highbits md xs).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
1014	<code>proof.</code>	Begins the proof script for the preceding proposition.
1015	<code>move=> md23 xs_bd.</code>	Introduces universally quantified variables or hypotheses into the proof context.
1016	<code>rewrite /haetae_polyveck_polyq_coeffs_wf.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1017	<code>split.</code>	Splits a conjunction or structured goal into independent proof obligations.
1018	<code>+ by apply polyveck_vk_highbits_wf.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1019	<code>move=> row row_rng.</code>	Introduces universally quantified variables or hypotheses into the proof context.
1020	<code>rewrite /polyveck_vk_highbits_nth_mkseq; first by smt().</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1021	<code>apply poly_vk_highbits_mode23_polyq_wf; first by apply md23.</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1022	<code>by apply xs_bd.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1023	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1024	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1025	<code>lemma coeff_q_bound_mode5_polyq_bound x :</code>	States checked lemma <code>coeff_q_bound_mode5_polyq_bound</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1026	<code>coeff_q_bound x =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1027	<code>0 <= x < haetae_polyq_coeff_bound Mode5.</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1028	<code>proof.</code>	Begins the proof script for the preceding proposition.
1029	<code>by rewrite /coeff_q_bound /haetae_polyq_coeff_bound</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1030	<code>/haetae_polyq_mode5_bound /q; smt().</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1031	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1032	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1033	<code>lemma poly_q_bound_mode5_polyq_wf p :</code>	States checked lemma <code>poly_q_bound_mode5_polyq_wf</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1034	<code>poly_wf p =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1035	<code>poly_coeffs_q_bound p =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1036	<code>haetae_polyq_coeffs_wf Mode5 p.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1037	<code>proof.</code>	Begins the proof script for the preceding proposition.
1038	<code>move=> p_wf p_bd.</code>	Introduces universally quantified variables or hypotheses into the proof context.
1039	<code>rewrite /haetae_polyq_coeffs_wf.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1040	<code>split; first by apply p_wf.</code>	Splits a conjunction or structured goal into independent proof obligations.
1041	<code>move=> i i_rng.</code>	Introduces universally quantified variables or hypotheses into the proof context.
1042	<code>by apply coeff_q_bound_mode5_polyq_bound; apply p_bd.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1043	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1044	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1045	<code>lemma polyveck_q_bound_mode5_polyq_wf xs :</code>	States checked lemma <code>polyveck_q_bound_mode5_polyq_wf</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1046	<code>polyveck_wf Mode5 xs =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1047	<code>polyveck_coeffs_q_bound Mode5 xs =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1048	<code>haetae_polyveck_polyq_coeffs_wf Mode5 xs.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1049	<code>proof.</code>	Begins the proof script for the preceding proposition.
1050	<code>move=> xs_wf xs_bd.</code>	Introduces universally quantified variables or hypotheses into the proof context.
1051	<code>rewrite /haetae_polyveck_polyq_coeffs_wf.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.

Line	EasyCrypt source	Mathematical reading
1052	split; first by apply xs_wf.	Splits a conjunction or structured goal into independent proof obligations.
1053	move=> row row_rng.	Introduces universally quantified variables or hypotheses into the proof context.
1054	apply poly_q_bound_mode5_polyq_wf.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1055	+ by apply (polyveck_wf_nth Mode5 xs row).	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1056	by apply (xs_bd row row_rng).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1057	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1058	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1059	op haetae_pack_poly_q_mode23_byte	Defines operation haetae_pack_poly_q_mode23_byte, a total mathematical function or abbreviation used by later declarations.
1060	(p : poly) (blk ofs : int) : byte =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1061	let c0 = poly_coeff p (8 * blk + 0) in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1062	let c1 = poly_coeff p (8 * blk + 1) in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1063	let c2 = poly_coeff p (8 * blk + 2) in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1064	let c3 = poly_coeff p (8 * blk + 3) in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1065	let c4 = poly_coeff p (8 * blk + 4) in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1066	let c5 = poly_coeff p (8 * blk + 5) in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1067	let c6 = poly_coeff p (8 * blk + 6) in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1068	let c7 = poly_coeff p (8 * blk + 7) in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1069	if ofs = 0 then haetae_byte_norm c0	Splits proof obligations according to a conditional branch in the program or formula.
1070	else if ofs = 1 then	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1071	haetae_byte_norm ((c0 %/ 256) %% 128 + (c1 %% 2) * 128)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1072	else if ofs = 2 then haetae_byte_norm (c1 %/ 2)	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1073	else if ofs = 3 then	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1074	haetae_byte_norm ((c1 %/ 512) %% 64 + (c2 %% 4) * 64)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1075	else if ofs = 4 then haetae_byte_norm (c2 %/ 4)	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1076	else if ofs = 5 then	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1077	haetae_byte_norm ((c2 %/ 1024) %% 32 + (c3 %% 8) * 32)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1078	else if ofs = 6 then haetae_byte_norm (c3 %/ 8)	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1079	else if ofs = 7 then	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1080	haetae_byte_norm ((c3 %/ 2048) %% 16 + (c4 %% 16) * 16)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1081	else if ofs = 8 then haetae_byte_norm (c4 %/ 16)	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1082	else if ofs = 9 then	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1083	haetae_byte_norm ((c4 %/ 4096) %% 8 + (c5 %% 32) * 8)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1084	else if ofs = 10 then haetae_byte_norm (c5 %/ 32)	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1085	else if ofs = 11 then	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.

Line	EasyCrypt source	Mathematical reading
1086	haetae_byte_norm ((c5 %/ 8192) %% 4 + (c6 %% 64) * 4)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1087	else if ofs = 12 then haetae_byte_norm (c6 %/ 64)	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1088	else if ofs = 13 then	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1089	haetae_byte_norm ((c6 %/ 16384) %% 2 + (c7 %% 128) * 2)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1090	else haetae_byte_norm (c7 %/ 128).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1091	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1092	op haetae_pack_poly_q_mode5_byte (p : poly) (j : int) : byte =	Defines integer-valued parameter haetae_pack_poly_q_mode5_byte, used for dimensions, budgets, support sizes, or bounds.
1093	let c = poly_coeff p (j %/ 2) in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1094	if j %% 2 = 0 then haetae_byte_norm c	Splits proof obligations according to a conditional branch in the program or formula.
1095	else haetae_byte_norm (c %/ 256).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1096	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1097	op haetae_pack_poly_q_ref (md : mode) (p : poly) : byte list =	Defines operation haetae_pack_poly_q_ref, a total mathematical function or abbreviation used by later declarations.
1098	with md = Mode2 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1099	mkseq	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1100	(fun j => haetae_pack_poly_q_mode23_byte p (j %/ 15) (j %% 15))	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1101	(mode_polyq_packedbytes Mode2)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1102	with md = Mode3 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1103	mkseq	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1104	(fun j => haetae_pack_poly_q_mode23_byte p (j %/ 15) (j %% 15))	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1105	(mode_polyq_packedbytes Mode3)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1106	with md = Mode5 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1107	mkseq	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1108	(fun j => haetae_pack_poly_q_mode5_byte p j)	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1109	(mode_polyq_packedbytes Mode5).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1110	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1111	op haetae_unpack_poly_q_mode23_coeff_at	Defines operation haetae_unpack_poly_q_mode23_coeff_at, a total mathematical function or abbreviation used by later declarations.
1112	(enc : encoded_pkey) (base blk ofs : int) : coeff =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1113	let a0 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 0)) in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1114	let a1 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 1)) in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1115	let a2 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 2)) in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1116	let a3 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 3)) in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1117	let a4 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 4)) in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1118	let a5 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 5)) in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.

Line	EasyCrypt source	Mathematical reading
1119	let a6 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 6)) in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1120	let a7 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 7)) in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1121	let a8 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 8)) in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1122	let a9 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 9)) in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1123	let a10 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 10)) in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1124	let a11 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 11)) in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1125	let a12 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 12)) in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1126	let a13 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 13)) in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1127	let a14 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 14)) in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1128	if ofs = 0 then a0 + 256 * (a1 %% 128)	Splits proof obligations according to a conditional branch in the program or formula.
1129	else if ofs = 1 then	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1130	((a1 %/ 128) %% 2) + 2 * a2 + 512 * (a3 %% 64)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1131	else if ofs = 2 then	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1132	((a3 %/ 64) %% 4) + 4 * a4 + 1024 * (a5 %% 32)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1133	else if ofs = 3 then	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1134	((a5 %/ 32) %% 8) + 8 * a6 + 2048 * (a7 %% 16)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1135	else if ofs = 4 then	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1136	((a7 %/ 16) %% 16) + 16 * a8 + 4096 * (a9 %% 8)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1137	else if ofs = 5 then	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1138	((a9 %/ 8) %% 32) + 32 * a10 + 8192 * (a11 %% 4)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1139	else if ofs = 6 then	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1140	((a11 %/ 4) %% 64) + 64 * a12 + 16384 * (a13 %% 2)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1141	else ((a13 %/ 2) %% 128) + 128 * a14.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1142	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1143	op haetae_unpack_poly_q_mode5_coeff_at	Defines operation haetae_unpack_poly_q_mode5_coeff_at, a total mathematical function or abbreviation used by later declarations.
1144	(enc : encoded_pkey) (base i : int) : coeff =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1145	haetae_byte_norm (nth 0 enc (base + 2 * i)) +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1146	256 * haetae_byte_norm (nth 0 enc (base + 2 * i + 1)).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1147	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1148	op haetae_unpack_poly_q_ref_at	Defines operation haetae_unpack_poly_q_ref_at, a total mathematical function or abbreviation used by later declarations.
1149	(md : mode) (enc : encoded_pkey) (offset : int) : poly =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1150	with md = Mode2 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1151	mkseq	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
1152	(fun i =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1153	haetae_unpack_poly_q_mode23_coeff_at	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1154	enc offset (i %/ 8) (i %% 8))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1155	n	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1156	with md = Mode3 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1157	mkseq	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1158	(fun i =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1159	haetae_unpack_poly_q_mode23_coeff_at	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1160	enc offset (i %/ 8) (i %% 8))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1161	n	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1162	with md = Mode5 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1163	mkseq	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1164	(fun i => haetae_unpack_poly_q_mode5_coeff_at enc offset i)	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1165	n.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1166	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1167	op haetae_pack_polyveck_body_ref (md : mode) (b : polyveck) : byte list =	Defines operation <code>haetae_pack_polyveck_body_ref</code> , a total mathematical function or abbreviation used by later declarations.
1168	mkseq	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1169	(fun i =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1170	nth 0	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1171	(haetae_pack_poly_q_ref md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1172	(nth poly_zero b (i %/ mode_polyq_packedbytes md)))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1173	(i %% mode_polyq_packedbytes md))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1174	(haetae_public_key_body_bytes md).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1175	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1176	op haetae_unpack_polyveck_body_ref	Defines operation <code>haetae_unpack_polyveck_body_ref</code> , a total mathematical function or abbreviation used by later declarations.
1177	(md : mode) (enc : encoded_pkey) (offset : int) : polyveck =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1178	mkseq	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1179	(fun row =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1180	haetae_unpack_poly_q_ref_at md enc	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1181	(offset + row * mode_polyq_packedbytes md))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1182	(mode_k md).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1183	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1184	op haetae_pack_vk_ref (md : mode) (b : polyveck) (sd : seed) : encoded_pkey =	Defines operation <code>haetae_pack_vk_ref</code> , a total mathematical function or abbreviation used by later declarations.

Line	EasyCrypt source	Mathematical reading
1185	haetae_pack_seed sd ++ haetae_pack_polyveck_body_ref md b.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1186	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1187	op haetae_unpack_vk_public_key_ref	Defines operation haetae_unpack_vk_public_key_ref, a total mathematical function or abbreviation used by later declarations.
1188	(md : mode) (enc : encoded_pkey) : pkey =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1189	(haetae_unpack_seed enc,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1190	haetae_unpack_polyveck_body_ref md enc seedbytes).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1191	op haetae_pack_polyveck_body (md : mode) (b : polyveck) : byte list =	Defines operation haetae_pack_polyveck_body, a total mathematical function or abbreviation used by later declarations.
1192	mkseq	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1193	(fun i =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1194	if i %% mode_polyq_packedbytes md < n then	Splits proof obligations according to a conditional branch in the program or formula.
1195	poly_coeff	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1196	(nth poly_zero b (i %% mode_polyq_packedbytes md))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1197	(i %% mode_polyq_packedbytes md)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1198	else 0)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1199	(haetae_public_key_body_bytes md).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1200	op haetae_unpack_polyveck_body	Defines operation haetae_unpack_polyveck_body, a total mathematical function or abbreviation used by later declarations.
1201	(md : mode) (enc : encoded_pkey) (offset : int) : polyveck =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1202	mkseq	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1203	(fun row =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1204	haetae_unpack_poly_q_at md enc	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1205	(offset + row * mode_polyq_packedbytes md))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1206	(mode_k md).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1207	op haetae_pack_vk (md : mode) (b : polyveck) (sd : seed) : encoded_pkey =	Defines operation haetae_pack_vk, a total mathematical function or abbreviation used by later declarations.
1208	haetae_pack_seed sd ++ haetae_pack_polyveck_body md b.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1209	op haetae_unpack_vk_public_key	Defines operation haetae_unpack_vk_public_key, a total mathematical function or abbreviation used by later declarations.
1210	(md : mode) (enc : encoded_pkey) : pkey =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1211	(haetae_unpack_seed enc,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1212	haetae_unpack_polyveck_body md enc seedbytes).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1213	op haetae_unpack_vk_matrix (md : mode) (enc : encoded_pkey) : matrix =	Defines operation haetae_unpack_vk_matrix, a total mathematical function or abbreviation used by later declarations.
1214	packed_haetae_verification_matrix md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1215	(haetae_unpack_vk_public_key md enc).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1216	op haetae_pack_public_key_bytes (md : mode) (pk : pkey) : encoded_pkey =	Defines operation haetae_pack_public_key_bytes, a total mathematical function or abbreviation used by later declarations.
1217	haetae_pack_vk md pk.'2 pk.'1.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
1218	op haetae_unpack_public_key_bytes	Defines operation <code>haetae_unpack_public_key_bytes</code> , a total mathematical function or abbreviation used by later declarations.
1219	(md : mode) (enc : encoded_pkey) : pkey =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1220	haetae_unpack_vk_public_key md enc.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1221	op concrete_haetae_keygen_packed_public_key	Defines operation <code>concrete_haetae_keygen_packed_public_key</code> , a total mathematical function or abbreviation used by later declarations.
1222	(md : mode) (sd : seed) : encoded_pkey =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1223	haetae_pack_public_key_bytes md (packed_haetae_public_key md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1224	op concrete_haetae_keygen_unpacked_public_key	Defines operation <code>concrete_haetae_keygen_unpacked_public_key</code> , a total mathematical function or abbreviation used by later declarations.
1225	(md : mode) (sd : seed) : pkey =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1226	haetae_unpack_public_key_bytes md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1227	(concrete_haetae_keygen_packed_public_key md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1228	op concrete_haetae_keygen_verification_matrix (md : mode) (sd : seed) :	Defines operation <code>concrete_haetae_keygen_verification_matrix</code> , a total mathematical function or abbreviation used by later declarations.
1229	matrix =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1230	packed_haetae_verification_matrix md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1231	(concrete_haetae_keygen_unpacked_public_key md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1232	op haetae_pack_public_key_bytes_ref (md : mode) (pk : pkey) :	Defines operation <code>haetae_pack_public_key_bytes_ref</code> , a total mathematical function or abbreviation used by later declarations.
1233	encoded_pkey =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1234	haetae_pack_vk_ref md pk.'2 pk.'1.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1235	op haetae_unpack_public_key_bytes_ref	Defines operation <code>haetae_unpack_public_key_bytes_ref</code> , a total mathematical function or abbreviation used by later declarations.
1236	(md : mode) (enc : encoded_pkey) : pkey =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1237	haetae_unpack_vk_public_key_ref md enc.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1238	op concrete_haetae_keygen_packed_public_key_ref	Defines operation <code>concrete_haetae_keygen_packed_public_key_ref</code> , a total mathematical function or abbreviation used by later declarations.
1239	(md : mode) (sd : seed) : encoded_pkey =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1240	haetae_pack_public_key_bytes_ref md (packed_haetae_public_key md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1241	op concrete_haetae_keygen_unpacked_public_key_ref	Defines operation <code>concrete_haetae_keygen_unpacked_public_key_ref</code> , a total mathematical function or abbreviation used by later declarations.
1242	(md : mode) (sd : seed) : pkey =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1243	haetae_unpack_public_key_bytes_ref md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1244	(concrete_haetae_keygen_packed_public_key_ref md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1245	op concrete_haetae_keygen_verification_matrix_ref	Defines operation <code>concrete_haetae_keygen_verification_matrix_ref</code> , a total mathematical function or abbreviation used by later declarations.
1246	(md : mode) (sd : seed) : matrix =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1247	packed_haetae_verification_matrix md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1248	(concrete_haetae_keygen_unpacked_public_key_ref md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1249	op concrete_haetae_reference_keygen_packed_public_key_ref	Defines operation <code>concrete_haetae_reference_keygen_packed_public_key_ref</code> , a total mathematical function or abbreviation used by later declarations.
1250	(md : mode) (sd : seed) : encoded_pkey =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.

Line	EasyCrypt source	Mathematical reading
1251	haetae_pack_public_key_bytes_ref md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1252	(packed_haetae_reference_public_key md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1253	op concrete_haetae_reference_keygen_unpacked_public_key_ref	Defines operation concrete_haetae_reference_keygen_unpacked_public_key_ref, a total mathematical function or abbreviation used by later declarations.
1254	(md : mode) (sd : seed) : pkey =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1255	haetae_unpack_public_key_bytes_ref md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1256	(concrete_haetae_reference_keygen_packed_public_key_ref md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1257	op concrete_haetae_reference_keygen_verification_matrix_ref	Defines operation concrete_haetae_reference_keygen_verification_matrix_ref, a total mathematical function or abbreviation used by later declarations.
1258	(md : mode) (sd : seed) : matrix =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1259	packed_haetae_verification_matrix md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1260	(concrete_haetae_reference_keygen_unpacked_public_key_ref md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1261	op haetae_public_key_roundtrip_ok (md : mode) (pk : pkey) : bool =	Defines predicate haetae_public_key_roundtrip_ok, an event or well-formedness condition used in later implications and game proofs.
1262	haetae_unpack_public_key_bytes md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1263	(haetae_pack_public_key_bytes md pk) = pk.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1264	op haetae_keygen_public_key_roundtrip_ok (md : mode) (sd : seed) : bool =	Defines predicate haetae_keygen_public_key_roundtrip_ok, an event or well-formedness condition used in later implications and game proofs.
1265	haetae_public_key_roundtrip_ok md (packed_haetae_public_key md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1266	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1267	lemma public_key_seed_poly_wf tag src :	States checked lemma public_key_seed_poly_wf. The following proof script must derive this proposition from prior definitions and lemmas.
1268	poly_wf (public_key_seed_poly tag src).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1269	proof. by rewrite /poly_wf /public_key_seed_poly size_mkseq /n. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1270	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1271	lemma public_key_seed_polyvecl_wf md tag src :	States checked lemma public_key_seed_polyvecl_wf. The following proof script must derive this proposition from prior definitions and lemmas.
1272	polyvecl_wf md (public_key_seed_polyvecl md tag src).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1273	proof.	Begins the proof script for the preceding proposition.
1274	rewrite /polyvecl_wf /public_key_seed_polyvecl size_mkseq.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1275	split.	Splits a conjunction or structured goal into independent proof obligations.
1276	+ by case md.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1277	apply/allP=> p /mkseqP [i [_ ->]].	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1278	by apply public_key_seed_poly_wf.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1279	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1280	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1281	lemma public_key_seed_polyveck_wf md tag src :	States checked lemma public_key_seed_polyveck_wf. The following proof script must derive this proposition from prior definitions and lemmas.
1282	polyveck_wf md (public_key_seed_polyveck md tag src).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1283	proof.	Begins the proof script for the preceding proposition.
1284	rewrite /polyveck_wf /public_key_seed_polyveck size_mkseq.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.

Line	EasyCrypt source	Mathematical reading
1285	<code>split.</code>	Splits a conjunction or structured goal into independent proof obligations.
1286	<code>+ by case md.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1287	<code>apply/allP=> p /mkseqP [i [_ ->]].</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1288	<code>by apply public_key_seed_poly_wf.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1289	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1290	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1291	<code>lemma haetae_keygen_seedbuf_bytes_gt0 :</code>	States checked lemma <code>haetae_keygen_seedbuf_bytes_gt0</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1292	<code>0 < haetae_keygen_seedbuf_bytes.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1293	<code>proof. by rewrite /haetae_keygen_seedbuf_bytes /seedbytes /crhbytes. qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1294	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1295	<code>lemma haetae_reference_keygen_seedbuf_bytesE :</code>	States checked lemma <code>haetae_reference_keygen_seedbuf_bytesE</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1296	<code>haetae_reference_keygen_seedbuf_bytes = haetae_keygen_seedbuf_bytes.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1297	<code>proof.</code>	Begins the proof script for the preceding proposition.
1298	<code>by rewrite /haetae_reference_keygen_seedbuf_bytes</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1299	<code>/haetae_keygen_seedbuf_bytes.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1300	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1301	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1302	<code>lemma haetae_reference_keygen_xof_absorb_bytesE :</code>	States checked lemma <code>haetae_reference_keygen_xof_absorb_bytesE</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1303	<code>haetae_reference_keygen_xof_absorb_bytes = seedbytes.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1304	<code>proof. by rewrite /haetae_reference_keygen_xof_absorb_bytes. qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1305	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1306	<code>lemma haetae_reference_keygen_xof_squeeze_bytesE :</code>	States checked lemma <code>haetae_reference_keygen_xof_squeeze_bytesE</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1307	<code>haetae_reference_keygen_xof_squeeze_bytes =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1308	<code>haetae_reference_keygen_seedbuf_bytes.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1309	<code>proof. by rewrite /haetae_reference_keygen_xof_squeeze_bytes. qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1310	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1311	<code>lemma haetae_keygen_rhoprime_offsetE :</code>	States checked lemma <code>haetae_keygen_rhoprime_offsetE</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1312	<code>haetae_keygen_rhoprime_offset = 0.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1313	<code>proof. by rewrite /haetae_keygen_rhoprime_offset. qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1314	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1315	<code>lemma haetae_keygen_sigma_offsetE :</code>	States checked lemma <code>haetae_keygen_sigma_offsetE</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1316	<code>haetae_keygen_sigma_offset = seedbytes.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1317	<code>proof. by rewrite /haetae_keygen_sigma_offset. qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1318	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
1319	lemma haetae_keygen_key_offsetE :	States checked lemma haetae_keygen_key_offsetE. The following proof script must derive this proposition from prior definitions and lemmas.
1320	haetae_keygen_key_offset = seedbytes + crhbytes.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1321	proof. by rewrite /haetae_keygen_key_offset. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1322	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1323	lemma haetae_shake256_rate_bytesE :	States checked lemma haetae_shake256_rate_bytesE. The following proof script must derive this proposition from prior definitions and lemmas.
1324	haetae_shake256_rate_bytes = 136.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1325	proof.	Begins the proof script for the preceding proposition.
1326	by rewrite /haetae_shake256_rate_bytes shake256_rate_bytesE.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1327	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1328	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1329	lemma haetae_shake256_domain_separatorE :	States checked lemma haetae_shake256_domain_separatorE. The following proof script must derive this proposition from prior definitions and lemmas.
1330	haetae_shake256_domain_separator = 31.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1331	proof.	Begins the proof script for the preceding proposition.
1332	by rewrite /haetae_shake256_domain_separator shake256_domain_separatorE.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1333	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1334	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1335	lemma haetae_shake256_bytes_size input outlen :	States checked lemma haetae_shake256_bytes_size. The following proof script must derive this proposition from prior definitions and lemmas.
1336	0 <= outlen =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1337	size (haetae_shake256_bytes input outlen) = outlen.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1338	proof.	Begins the proof script for the preceding proposition.
1339	move=> ge0_outlen.	Introduces universally quantified variables or hypotheses into the proof context.
1340	by rewrite /haetae_shake256_bytes shake256_size.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1341	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1342	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1343	lemma haetae_xof256_squeeze_size st outlen :	States checked lemma haetae_xof256_squeeze_size. The following proof script must derive this proposition from prior definitions and lemmas.
1344	0 <= outlen =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1345	size (haetae_xof256_squeeze st outlen) = outlen.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1346	proof.	Begins the proof script for the preceding proposition.
1347	move=> ge0_outlen.	Introduces universally quantified variables or hypotheses into the proof context.
1348	by rewrite /haetae_xof256_squeeze	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1349	shake256_squeeze_size.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1350	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1351	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1352	lemma haetae_xof256_absorb_once_squeeze_size input inlen outlen :	States checked lemma haetae_xof256_absorb_once_squeeze_size. The following proof script must derive this proposition from prior definitions and lemmas.
1353	0 <= outlen =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1354	size (haetae_xof256_absorb_once_squeeze input inlen outlen) = outlen.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1355	proof.	Begins the proof script for the preceding proposition.

Line	EasyCrypt source	Mathematical reading
1356	<code>move=> ge0_outlen.</code>	Introduces universally quantified variables or hypotheses into the proof context.
1357	<code>by rewrite /haetae_xof256_absorb_once_squeeze</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1358	<code>haetae_xof256_squeeze_size.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1359	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1360	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1361	<code>lemma haetae_reference_keygen_seedbuf_after_memcpy_size sd :</code>	States checked lemma <code>haetae_reference_keygen_seedbuf_after_memcpy_size</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1362	<code>size (haetae_reference_keygen_seedbuf_after_memcpy sd) =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1363	<code>haetae_reference_keygen_seedbuf_bytes.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1364	<code>proof.</code>	Begins the proof script for the preceding proposition.
1365	<code>by rewrite /haetae_reference_keygen_seedbuf_after_memcpy size_mkseq.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1366	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1367	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1368	<code>lemma haetae_keygen_xof_seedbuf_size sd :</code>	States checked lemma <code>haetae_keygen_xof_seedbuf_size</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1369	<code>size (haetae_keygen_xof_seedbuf sd) = haetae_keygen_seedbuf_bytes.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1370	<code>proof.</code>	Begins the proof script for the preceding proposition.
1371	<code>by rewrite /haetae_keygen_xof_seedbuf</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1372	<code>/haetae_xof256_absorb_once_squeeze</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1373	<code>/haetae_xof256_squeeze /shake256_squeeze</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1374	<code>/haetae_reference_keygen_xof_squeeze_bytes</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1375	<code>/haetae_reference_keygen_seedbuf_bytes</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1376	<code>/haetae_keygen_seedbuf_bytes size_mkseq.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1377	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1378	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1379	<code>lemma haetae_reference_keygen_xof256_seedbuf_size sd :</code>	States checked lemma <code>haetae_reference_keygen_xof256_seedbuf_size</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1380	<code>size (haetae_reference_keygen_xof256_seedbuf sd) =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1381	<code>haetae_reference_keygen_seedbuf_bytes.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1382	<code>proof.</code>	Begins the proof script for the preceding proposition.
1383	<code>by rewrite /haetae_reference_keygen_xof256_seedbuf</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1384	<code>haetae_keygen_xof_seedbuf_size</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1385	<code>haetae_reference_keygen_seedbuf_bytesE.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1386	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1387	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1388	<code>lemma haetae_seed_slice_size src offset len :</code>	States checked lemma <code>haetae_seed_slice_size</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1389	<code>0 <= len =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1390	<code>size (haetae_seed_slice src offset len) = len.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1391	<code>proof.</code>	Begins the proof script for the preceding proposition.

Line	EasyCrypt source	Mathematical reading
1392	<code>move=> ge0_len.</code>	Introduces universally quantified variables or hypotheses into the proof context.
1393	<code>rewrite /haetae_seed_slice_size_mkseq.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1394	<code>by smt().</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1395	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1396	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1397	<code>lemma haetae_reference_keygen_xof_input_size sd :</code>	States checked lemma <code>haetae_reference_keygen_xof_input_size</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1398	<code>size (haetae_reference_keygen_xof_input sd) =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1399	<code>haetae_reference_keygen_xof_absorb_bytes.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1400	<code>proof.</code>	Begins the proof script for the preceding proposition.
1401	<code>rewrite /haetae_reference_keygen_xof_input.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1402	<code>apply haetae_seed_slice_size.</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1403	<code>by rewrite /haetae_reference_keygen_xof_absorb_bytes /seedbytes.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1404	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1405	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1406	<code>lemma haetae_xof256_absorb_input_size input inlen :</code>	States checked lemma <code>haetae_xof256_absorb_input_size</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1407	<code>0 <= inlen =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1408	<code>size (haetae_xof256_absorb_input input inlen) = inlen.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1409	<code>proof.</code>	Begins the proof script for the preceding proposition.
1410	<code>move=> ge0_inlen.</code>	Introduces universally quantified variables or hypotheses into the proof context.
1411	<code>rewrite /haetae_xof256_absorb_input.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1412	<code>by apply haetae_seed_slice_size.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1413	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1414	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1415	<code>lemma haetae_reference_keygen_xof_absorb_inputE sd :</code>	States checked lemma <code>haetae_reference_keygen_xof_absorb_inputE</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1416	<code>haetae_xof256_absorb_input</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1417	<code>(haetae_reference_keygen_seedbuf_after_memcpy sd)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1418	<code>haetae_reference_keygen_xof_absorb_bytes =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1419	<code>haetae_reference_keygen_xof_input sd.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1420	<code>proof.</code>	Begins the proof script for the preceding proposition.
1421	<code>by rewrite /haetae_xof256_absorb_input</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1422	<code>/haetae_reference_keygen_xof_input.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1423	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1424	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1425	<code>lemma haetae_reference_keygen_shake256_domain sd :</code>	States checked lemma <code>haetae_reference_keygen_shake256_domain</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1426	<code>shake256_absorb_once_short_domain</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
1427	(haetae_reference_keygen_xof_input sd)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1428	haetae_reference_keygen_xof_absorb_bytes.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1429	proof.	Begins the proof script for the preceding proposition.
1430	by rewrite /shake256_absorb_once_short_domain	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1431	/haetae_reference_keygen_xof_absorb_bytes	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1432	shake256_rate_bytesE /seedbytes.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1433	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1434	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1435	lemma haetae_reference_keygen_xof_squeeze_bytes_lt_rate :	States checked lemma
1436	haetae_reference_keygen_xof_squeeze_bytes < haetae_shake256_rate_bytes.	haetae_reference_keygen_xof_squeeze_bytes_lt_rate. The following proof script must derive this proposition from prior definitions and lemmas.
1437	proof.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1438	by rewrite /haetae_reference_keygen_xof_squeeze_bytes	Begins the proof script for the preceding proposition.
1439	/haetae_reference_keygen_seedbuf_bytes	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1440	/haetae_shake256_rate_bytes	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1441	shake256_rate_bytesE /seedbytes /crhbytes.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1442	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1443	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1444	lemma haetae_reference_keygen_xof_absorb_once_stateE sd :	States checked lemma haetae_reference_keygen_xof_absorb_once_stateE. The following proof script must derive this proposition from prior definitions and lemmas.
1445	haetae_xof256_absorb_once	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1446	(haetae_reference_keygen_seedbuf_after_memcpy sd)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1447	haetae_reference_keygen_xof_absorb_bytes =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1448	shake256_absorb_once	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1449	(haetae_reference_keygen_xof_input sd)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1450	haetae_reference_keygen_xof_absorb_bytes.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1451	proof.	Begins the proof script for the preceding proposition.
1452	by rewrite /haetae_xof256_absorb_once	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1453	haetae_reference_keygen_xof_absorb_inputE.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1454	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1455	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1456	lemma haetae_keygen_xof_seedbuf_incrementalE sd :	States checked lemma haetae_keygen_xof_seedbuf_incrementalE. The following proof script must derive this proposition from prior definitions and lemmas.
1457	haetae_keygen_xof_seedbuf sd =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1458	haetae_xof256_squeeze	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1459	(haetae_xof256_absorb_once	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1460	(haetae_reference_keygen_seedbuf_after_memcpy sd)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1461	haetae_reference_keygen_xof_absorb_bytes)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
1462	haetae_reference_keygen_xof_squeeze_bytes.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1463	proof. by rewrite /haetae_keygen_xof_seedbuf	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1464	/haetae_xof256_absorb_once_squeeze. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1465	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1466	lemma haetae_keygen_xof_seedbuf_shake256E sd :	States checked lemma haetae_keygen_xof_seedbuf_shake256E. The following proof script must derive this proposition from prior definitions and lemmas.
1467	haetae_keygen_xof_seedbuf sd =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1468	haetae_shake256_bytes	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1469	(haetae_reference_keygen_xof_input sd)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1470	haetae_reference_keygen_xof_squeeze_bytes.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1471	proof.	Begins the proof script for the preceding proposition.
1472	by rewrite haetae_keygen_xof_seedbuf_incrementalE	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1473	haetae_reference_keygen_xof_absorb_once_stateE	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1474	/haetae_shake256_bytes	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1475	/shake256 haetae_reference_keygen_xof_input_size.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1476	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1477	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1478	lemma haetae_keygen_seedbuf_rho_prime_size seedbuf :	States checked lemma haetae_keygen_seedbuf_rho_prime_size. The following proof script must derive this proposition from prior definitions and lemmas.
1479	size (haetae_keygen_seedbuf_rho_prime seedbuf) = seedbytes.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1480	proof.	Begins the proof script for the preceding proposition.
1481	rewrite /haetae_keygen_seedbuf_rho_prime.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1482	apply haetae_seed_slice_size.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1483	by rewrite /seedbytes.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1484	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1485	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1486	lemma haetae_keygen_seedbuf_sigma_size seedbuf :	States checked lemma haetae_keygen_seedbuf_sigma_size. The following proof script must derive this proposition from prior definitions and lemmas.
1487	size (haetae_keygen_seedbuf_sigma seedbuf) = crhbytes.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1488	proof.	Begins the proof script for the preceding proposition.
1489	rewrite /haetae_keygen_seedbuf_sigma.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1490	apply haetae_seed_slice_size.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1491	by rewrite /crhbytes.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1492	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1493	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1494	lemma haetae_keygen_seedbuf_key_size seedbuf :	States checked lemma haetae_keygen_seedbuf_key_size. The following proof script must derive this proposition from prior definitions and lemmas.
1495	size (haetae_keygen_seedbuf_key seedbuf) = seedbytes.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1496	proof.	Begins the proof script for the preceding proposition.
1497	rewrite /haetae_keygen_seedbuf_key.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.

Line	EasyCrypt source	Mathematical reading
1498	apply haetae_seed_slice_size.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1499	by rewrite /seedbytes.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1500	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1501	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1502	lemma haetae_reference_keygen_xof_seedbuf_layout_wf sd :	States checked lemma haetae_reference_keygen_xof_seedbuf_layout_wf. The following proof script must derive this proposition from prior definitions and lemmas.
1503	haetae_keygen_seedbuf_layout_wf	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1504	(haetae_reference_keygen_xof256_seedbuf sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1505	proof.	Begins the proof script for the preceding proposition.
1506	by rewrite /haetae_keygen_seedbuf_layout_wf	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1507	haetae_reference_keygen_xof256_seedbuf_size	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1508	haetae_keygen_rhopprime_offsetE	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1509	haetae_keygen_sigma_offsetE	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1510	haetae_keygen_key_offsetE	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1511	haetae_keygen_seedbuf_rhopprime_size	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1512	haetae_keygen_seedbuf_sigma_size	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1513	haetae_keygen_seedbuf_key_size.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1514	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1515	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1516	lemma haetae_reference_keygen_xof_call_wf_ok sd :	States checked lemma haetae_reference_keygen_xof_call_wf_ok. The following proof script must derive this proposition from prior definitions and lemmas.
1517	size sd = seedbytes =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1518	haetae_reference_keygen_xof_call_wf sd	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1519	(haetae_reference_keygen_xof256_seedbuf sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1520	proof.	Begins the proof script for the preceding proposition.
1521	move=> sd_sz.	Introduces universally quantified variables or hypotheses into the proof context.
1522	by rewrite /haetae_reference_keygen_xof_call_wf	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1523	sd_sz	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1524	haetae_reference_keygen_seedbuf_after_memcpy_size	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1525	haetae_reference_keygen_xof_absorb_inputE	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1526	haetae_reference_keygen_shake256_domain	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1527	haetae_reference_keygen_xof_squeeze_bytes_lt_rate	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1528	haetae_reference_keygen_xof_absorb_once_stateE	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1529	haetae_reference_keygen_xof256_seedbuf_size	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1530	/haetae_reference_keygen_xof256_seedbuf	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1531	haetae_keygen_xof_seedbuf_incrementalE	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
1532	/haetae_xof256_squeeze	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1533	haetae_reference_keygen_xof_absorb_bytesE	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1534	haetae_reference_keygen_xof_squeeze_bytesE.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1535	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1536	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1537	lemma haetae_keygen_rho_prime_ref_sliceE sd :	States checked lemma haetae_keygen_rho_prime_ref_sliceE. The following proof script must derive this proposition from prior definitions and lemmas.
1538	haetae_keygen_rho_prime sd =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1539	haetae_keygen_seedbuf_rho_prime	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1540	(haetae_reference_keygen_xof256_seedbuf sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1541	proof. by rewrite /haetae_keygen_rho_prime. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1542	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1543	lemma haetae_keygen_sigma_ref_sliceE sd :	States checked lemma haetae_keygen_sigma_ref_sliceE. The following proof script must derive this proposition from prior definitions and lemmas.
1544	haetae_keygen_sigma sd =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1545	haetae_keygen_seedbuf_sigma	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1546	(haetae_reference_keygen_xof256_seedbuf sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1547	proof. by rewrite /haetae_keygen_sigma. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1548	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1549	lemma haetae_keygen_key_ref_sliceE sd :	States checked lemma haetae_keygen_key_ref_sliceE. The following proof script must derive this proposition from prior definitions and lemmas.
1550	haetae_keygen_key sd =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1551	haetae_keygen_seedbuf_key	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1552	(haetae_reference_keygen_xof256_seedbuf sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1553	proof. by rewrite /haetae_keygen_key. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1554	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1555	lemma haetae_keygen_rho_prime_size sd :	States checked lemma haetae_keygen_rho_prime_size. The following proof script must derive this proposition from prior definitions and lemmas.
1556	size (haetae_keygen_rho_prime sd) = seedbytes.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1557	proof.	Begins the proof script for the preceding proposition.
1558	by rewrite /haetae_keygen_rho_prime haetae_keygen_seedbuf_rho_prime_size.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1559	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1560	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1561	lemma haetae_keygen_sigma_size sd :	States checked lemma haetae_keygen_sigma_size. The following proof script must derive this proposition from prior definitions and lemmas.
1562	size (haetae_keygen_sigma sd) = crhbytes.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1563	proof.	Begins the proof script for the preceding proposition.
1564	by rewrite /haetae_keygen_sigma haetae_keygen_seedbuf_sigma_size.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1565	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1566	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1567	lemma haetae_keygen_key_size sd :	States checked lemma haetae_keygen_key_size. The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
1568	size (haetae_keygen_key sd) = seedbytes.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1569	proof.	Begins the proof script for the preceding proposition.
1570	by rewrite /haetae_keygen_key haetae_keygen_seedbuf_key_size.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1571	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1572	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1573	lemma haetae_keygen_sampler_seed_size sigma counter :	States checked lemma haetae_keygen_sampler_seed_size. The following proof script must derive this proposition from prior definitions and lemmas.
1574	size (haetae_keygen_sampler_seed sigma counter) = seedbytes.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1575	proof. by rewrite /haetae_keygen_sampler_seed size_mkseq. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1576	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1577	lemma haetae_keygen_counter_next_gt counter md :	States checked lemma haetae_keygen_counter_next_gt. The following proof script must derive this proposition from prior definitions and lemmas.
1578	counter < haetae_keygen_counter_next md counter.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1579	proof. by rewrite /haetae_keygen_counter_next; smt(mode_m_gt0 mode_k_gt0). qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1580	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1581	lemma public_matrix_seed_size md sd :	States checked lemma public_matrix_seed_size. The following proof script must derive this proposition from prior definitions and lemmas.
1582	size (public_matrix_seed md sd) = mode_k md.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1583	proof. by rewrite /public_matrix_seed size_mkseq; case md. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1584	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1585	lemma public_matrix_seed_rows_wf md sd :	States checked lemma public_matrix_seed_rows_wf. The following proof script must derive this proposition from prior definitions and lemmas.
1586	all (polyvecl_wf md) (public_matrix_seed md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1587	proof.	Begins the proof script for the preceding proposition.
1588	rewrite /public_matrix_seed.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1589	apply/allP=> row /mkseqP [i [i_rng ->]].	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1590	by apply public_key_seed_polyvecl_wf.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1591	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1592	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1593	lemma public_secret_vector_seed_wf md sd :	States checked lemma public_secret_vector_seed_wf. The following proof script must derive this proposition from prior definitions and lemmas.
1594	polyvecl_wf md (public_secret_vector_seed md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1595	proof. by rewrite /public_secret_vector_seed; apply public_key_seed_polyvecl_wf. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1596	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1597	lemma public_error_vector_seed_wf md sd :	States checked lemma public_error_vector_seed_wf. The following proof script must derive this proposition from prior definitions and lemmas.
1598	polyveck_wf md (public_error_vector_seed md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1599	proof. by rewrite /public_error_vector_seed; apply public_key_seed_polyveck_wf. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1600	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1601	lemma public_rounding_vector_seed_wf md sd :	States checked lemma public_rounding_vector_seed_wf. The following proof script must derive this proposition from prior definitions and lemmas.
1602	polyveck_wf md (public_rounding_vector_seed md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
1603	proof. by rewrite /public_rounding_vector_seed; apply public_key_seed_polyveck_wf. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1604	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1605	lemma haetae_reference_polymatkm_expand_matA_size md rhoprime :	States checked lemma haetae_reference_polymatkm_expand_matA_size. The following proof script must derive this proposition from prior definitions and lemmas.
1606	size (haetae_reference_polymatkm_expand_matA md rhoprime) = mode_k md.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1607	proof.	Begins the proof script for the preceding proposition.
1608	by rewrite /haetae_reference_polymatkm_expand_matA_public_matrix_seed_size.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1609	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1610	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1611	lemma haetae_reference_polymatkm_expand_matA_rows_wf md rhoprime :	States checked lemma haetae_reference_polymatkm_expand_matA_rows_wf. The following proof script must derive this proposition from prior definitions and lemmas.
1612	all (polyvecl_wf md)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1613	(haetae_reference_polymatkm_expand_matA md rhoprime).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1614	proof.	Begins the proof script for the preceding proposition.
1615	by rewrite /haetae_reference_polymatkm_expand_matA;	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1616	apply public_matrix_seed_rows_wf.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1617	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1618	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1619	lemma haetae_reference_polyveck_expand_vecA_wf md rhoprime :	States checked lemma haetae_reference_polyveck_expand_vecA_wf. The following proof script must derive this proposition from prior definitions and lemmas.
1620	polyveck_wf md (haetae_reference_polyveck_expand_vecA md rhoprime).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1621	proof.	Begins the proof script for the preceding proposition.
1622	by rewrite /haetae_reference_polyveck_expand_vecA;	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1623	apply public_rounding_vector_seed_wf.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1624	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1625	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1626	lemma haetae_reference_polyvecmk_expand_S_s1_wf md sigma counter :	States checked lemma haetae_reference_polyvecmk_expand_S_s1_wf. The following proof script must derive this proposition from prior definitions and lemmas.
1627	polyvecl_wf md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1628	(haetae_reference_polyvecmk_expand_S_s1 md sigma counter).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1629	proof.	Begins the proof script for the preceding proposition.
1630	by rewrite /haetae_reference_polyvecmk_expand_S_s1;	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1631	apply public_secret_vector_seed_wf.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1632	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1633	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1634	lemma haetae_reference_polyvecmk_expand_S_s2_wf md sigma counter :	States checked lemma haetae_reference_polyvecmk_expand_S_s2_wf. The following proof script must derive this proposition from prior definitions and lemmas.
1635	polyveck_wf md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1636	(haetae_reference_polyvecmk_expand_S_s2 md sigma counter).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1637	proof.	Begins the proof script for the preceding proposition.

Line	EasyCrypt source	Mathematical reading
1638	by rewrite /haetae_reference_polyvecmk_expand_S_s2;	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1639	apply public_error_vector_seed_wf.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1640	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1641	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1642	lemma haetae_reference_keygen_secret_vector_wf md sd :	States checked lemma <code>haetae_reference_keygen_secret_vector_wf</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1643	polyvecl_wf md (haetae_reference_keygen_secret_vector md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1644	proof.	Begins the proof script for the preceding proposition.
1645	by rewrite /haetae_reference_keygen_secret_vector;	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1646	apply haetae_reference_polyvecmk_expand_S_s1_wf.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1647	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1648	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1649	lemma haetae_reference_keygen_error_vector_wf md sd :	States checked lemma <code>haetae_reference_keygen_error_vector_wf</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1650	polyveck_wf md (haetae_reference_keygen_error_vector md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1651	proof.	Begins the proof script for the preceding proposition.
1652	by rewrite /haetae_reference_keygen_error_vector;	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1653	apply haetae_reference_polyvecmk_expand_S_s2_wf.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1654	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1655	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1656	lemma haetae_reference_keygen_raw_public_vector_wf md sd :	States checked lemma <code>haetae_reference_keygen_raw_public_vector_wf</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1657	polyveck_wf md (haetae_reference_keygen_raw_public_vector md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1658	proof.	Begins the proof script for the preceding proposition.
1659	by rewrite /haetae_reference_keygen_raw_public_vector;	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1660	apply polyveck_add_wf.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1661	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1662	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1663	lemma haetae_reference_keygen_raw_public_vector_coeffs_q_bound md sd :	States checked lemma <code>haetae_reference_keygen_raw_public_vector_coeffs_q_bound</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1664	polyveck_coeffs_q_bound md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1665	(haetae_reference_keygen_raw_public_vector md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1666	proof.	Begins the proof script for the preceding proposition.
1667	by rewrite /haetae_reference_keygen_raw_public_vector;	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1668	apply polyveck_add_coeffs_q_bound.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1669	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1670	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1671	lemma haetae_reference_keygen_mode23_predecompose_vector_wf md sd :	States checked lemma <code>haetae_reference_keygen_mode23_predecompose_vector_wf</code> . The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
1672	<code>polyveck_wf md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1673	<code>(haetae_reference_keygen_mode23_predecompose_vector md sd).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1674	<code>proof.</code>	Begins the proof script for the preceding proposition.
1675	<code>by rewrite /haetae_reference_keygen_mode23_predecompose_vector;</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1676	<code>apply polyveck_add_wf.</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1677	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1678	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1679	<code>lemma haetae_reference_keygen_mode23_predecompose_vector_coeffs_q_bound</code>	States checked lemma
		<code>haetae_reference_keygen_mode23_predecompose_vector_coeffs_q_bound.</code>
		The following proof script must derive this proposition from prior definitions and lemmas.
1680	<code>md sd :</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1681	<code>polyveck_coeffs_q_bound md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1682	<code>(haetae_reference_keygen_mode23_predecompose_vector md sd).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1683	<code>proof.</code>	Begins the proof script for the preceding proposition.
1684	<code>by rewrite /haetae_reference_keygen_mode23_predecompose_vector;</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1685	<code>apply polyveck_add_coeffs_q_bound.</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1686	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1687	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1688	<code>lemma haetae_reference_keygen_mode23_public_vector_polyq_wf md sd :</code>	States checked lemma
		<code>haetae_reference_keygen_mode23_public_vector_polyq_wf.</code> The following proof script must derive this proposition from prior definitions and lemmas.
1689	<code>(md = Mode2 \ / md = Mode3) =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1690	<code>haetae_polyveck_polyq_coeffs_wf md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1691	<code>(haetae_reference_keygen_mode23_public_vector md sd).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1692	<code>proof.</code>	Begins the proof script for the preceding proposition.
1693	<code>move=> md23.</code>	Introduces universally quantified variables or hypotheses into the proof context.
1694	<code>rewrite /haetae_reference_keygen_mode23_public_vector.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1695	<code>apply polyveck_vk_highbits_mode23_polyq_wf; first by apply md23.</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1696	<code>by apply haetae_reference_keygen_mode23_predecompose_vector_coeffs_q_bound.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1697	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1698	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1699	<code>lemma haetae_reference_keygen_mode23_adjusted_error_vector_wf md sd :</code>	States checked lemma
		<code>haetae_reference_keygen_mode23_adjusted_error_vector_wf.</code> The following proof script must derive this proposition from prior definitions and lemmas.
1700	<code>polyveck_wf md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1701	<code>(haetae_reference_keygen_mode23_adjusted_error_vector md sd).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1702	<code>proof.</code>	Begins the proof script for the preceding proposition.
1703	<code>by rewrite /haetae_reference_keygen_mode23_adjusted_error_vector;</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1704	<code>apply polyveck_sub_wf.</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1705	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1706	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
1707	lemma haetae_reference_keygen_mode5_public_vector_wf sd :	States checked lemma haetae_reference_keygen_mode5_public_vector_wf. The following proof script must derive this proposition from prior definitions and lemmas.
1708	polyveck_wf Mode5	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1709	(haetae_reference_keygen_mode5_public_vector Mode5 sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1710	proof.	Begins the proof script for the preceding proposition.
1711	by rewrite /haetae_reference_keygen_mode5_public_vector;	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1712	apply polyveck_neg_wf.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1713	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1714	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1715	lemma haetae_reference_keygen_mode5_public_vector_coeffs_q_bound sd :	States checked lemma haetae_reference_keygen_mode5_public_vector_coeffs_q_bound. The following proof script must derive this proposition from prior definitions and lemmas.
1716	polyveck_coeffs_q_bound Mode5	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1717	(haetae_reference_keygen_mode5_public_vector Mode5 sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1718	proof.	Begins the proof script for the preceding proposition.
1719	by rewrite /haetae_reference_keygen_mode5_public_vector;	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1720	apply polyveck_neg_coeffs_q_bound.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1721	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1722	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1723	lemma haetae_reference_keygen_mode5_public_vector_polyq_wf sd :	States checked lemma haetae_reference_keygen_mode5_public_vector_polyq_wf. The following proof script must derive this proposition from prior definitions and lemmas.
1724	haetae_polyveck_polyq_coeffs_wf Mode5	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1725	(haetae_reference_keygen_mode5_public_vector Mode5 sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1726	proof.	Begins the proof script for the preceding proposition.
1727	apply polyveck_q_bound_mode5_polyq_wf.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1728	+ by apply haetae_reference_keygen_mode5_public_vector_wf.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1729	by apply haetae_reference_keygen_mode5_public_vector_coeffs_q_bound.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1730	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1731	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1732	lemma haetae_keygen_raw_public_vector_wf md sd :	States checked lemma haetae_keygen_raw_public_vector_wf. The following proof script must derive this proposition from prior definitions and lemmas.
1733	polyveck_wf md (haetae_keygen_raw_public_vector md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1734	proof. by rewrite /haetae_keygen_raw_public_vector; apply polyveck_add_wf. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1735	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1736	lemma haetae_keygen_raw_public_vector_coeffs_q_bound md sd :	States checked lemma haetae_keygen_raw_public_vector_coeffs_q_bound. The following proof script must derive this proposition from prior definitions and lemmas.
1737	polyveck_coeffs_q_bound md (haetae_keygen_raw_public_vector md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1738	proof.	Begins the proof script for the preceding proposition.
1739	by rewrite /haetae_keygen_raw_public_vector;	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1740	apply polyveck_add_coeffs_q_bound.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.

Line	EasyCrypt source	Mathematical reading
1741	<i>qed.</i>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1742	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1743	<code>lemma haetae_keygen_mode23_predecompose_vector_wf md sd :</code>	States checked lemma <code>haetae_keygen_mode23_predecompose_vector_wf</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1744	<code>polyveck_wf md (haetae_keygen_mode23_predecompose_vector md sd).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1745	<code>proof.</code>	Begins the proof script for the preceding proposition.
1746	<code>by rewrite /haetae_keygen_mode23_predecompose_vector;</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1747	<code>apply polyveck_add_wf.</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1748	<i>qed.</i>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1749	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1750	<code>lemma haetae_keygen_mode23_predecompose_vector_coeffs_q_bound md sd :</code>	States checked lemma <code>haetae_keygen_mode23_predecompose_vector_coeffs_q_bound</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1751	<code>polyveck_coeffs_q_bound md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1752	<code>(haetae_keygen_mode23_predecompose_vector md sd).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1753	<code>proof.</code>	Begins the proof script for the preceding proposition.
1754	<code>by rewrite /haetae_keygen_mode23_predecompose_vector;</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1755	<code>apply polyveck_add_coeffs_q_bound.</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1756	<i>qed.</i>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1757	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1758	<code>lemma haetae_keygen_mode23_public_vector_polyq_wf md sd :</code>	States checked lemma <code>haetae_keygen_mode23_public_vector_polyq_wf</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1759	<code>(md = Mode2 \ / md = Mode3) =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1760	<code>haetae_polyveck_polyq_coeffs_wf md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1761	<code>(haetae_keygen_mode23_public_vector md sd).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1762	<code>proof.</code>	Begins the proof script for the preceding proposition.
1763	<code>move=> md23.</code>	Introduces universally quantified variables or hypotheses into the proof context.
1764	<code>rewrite /haetae_keygen_mode23_public_vector.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1765	<code>apply polyveck_vk_highbits_mode23_polyq_wf; first by apply md23.</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1766	<code>by apply haetae_keygen_mode23_predecompose_vector_coeffs_q_bound.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1767	<i>qed.</i>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1768	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1769	<code>lemma haetae_keygen_mode23_rounding_lowbits_wf md sd :</code>	States checked lemma <code>haetae_keygen_mode23_rounding_lowbits_wf</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1770	<code>polyveck_wf md (haetae_keygen_mode23_rounding_lowbits md sd).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1771	<code>proof.</code>	Begins the proof script for the preceding proposition.
1772	<code>by rewrite /haetae_keygen_mode23_rounding_lowbits;</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1773	<code>apply polyveck_vk_lowbits_wf.</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1774	<i>qed.</i>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1775	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
1776	lemma haetae_keygen_mode23_adjusted_error_vector_wf md sd :	States checked lemma haetae_keygen_mode23_adjusted_error_vector_wf. The following proof script must derive this proposition from prior definitions and lemmas.
1777	polyveck_wf md (haetae_keygen_mode23_adjusted_error_vector md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1778	proof.	Begins the proof script for the preceding proposition.
1779	by rewrite /haetae_keygen_mode23_adjusted_error_vector;	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1780	apply polyveck_sub_wf.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1781	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1782	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1783	lemma haetae_keygen_mode5_public_vector_wf sd :	States checked lemma haetae_keygen_mode5_public_vector_wf. The following proof script must derive this proposition from prior definitions and lemmas.
1784	polyveck_wf Mode5 (haetae_keygen_mode5_public_vector Mode5 sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1785	proof.	Begins the proof script for the preceding proposition.
1786	by rewrite /haetae_keygen_mode5_public_vector;	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1787	apply polyveck_neg_wf.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1788	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1789	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1790	lemma haetae_keygen_mode5_public_vector_coeffs_q_bound sd :	States checked lemma
1791	polyveck_coeffs_q_bound Mode5	haetae_keygen_mode5_public_vector_coeffs_q_bound. The following proof script must derive this proposition from prior definitions and lemmas.
1792	(haetae_keygen_mode5_public_vector Mode5 sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1793	proof.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1794	by rewrite /haetae_keygen_mode5_public_vector;	Begins the proof script for the preceding proposition.
1795	apply polyveck_neg_coeffs_q_bound.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1796	qed.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1797	blank	Ends the proof; EasyCrypt has accepted all remaining obligations.
1798	lemma haetae_keygen_mode5_public_vector_polyq_wf sd :	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1799	haetae_polyveck_polyq_coeffs_wf Mode5	States checked lemma haetae_keygen_mode5_public_vector_polyq_wf. The following proof script must derive this proposition from prior definitions and lemmas.
1800	(haetae_keygen_mode5_public_vector Mode5 sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1801	proof.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1802	apply polyveck_q_bound_mode5_polyq_wf.	Begins the proof script for the preceding proposition.
1803	+ by apply haetae_keygen_mode5_public_vector_wf.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1804	by apply haetae_keygen_mode5_public_vector_coeffs_q_bound.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1805	qed.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1806	blank	Ends the proof; EasyCrypt has accepted all remaining obligations.
1807	lemma haetae_public_key_vector_from_relation_polyq_wf md sd s e :	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1808	haetae_polyveck_polyq_coeffs_wf md	States checked lemma haetae_public_key_vector_from_relation_polyq_wf. The following proof script must derive this proposition from prior definitions and lemmas.
1809	(haetae_public_key_vector_from_relation md sd s e).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1810	proof.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
1811	rewrite /haetae_public_key_vector_from_relation.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1812	case: md => /=.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
1813	+ apply polyveck_vk_highbits_mode23_polyq_wf.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1814	+ by left.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1815	by apply polyveck_add_coeffs_q_bound.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1816	+ apply polyveck_vk_highbits_mode23_polyq_wf.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1817	+ by right.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1818	by apply polyveck_add_coeffs_q_bound.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1819	apply polyveck_q_bound_mode5_polyq_wf.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1820	+ by apply polyveck_neg_wf.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1821	by apply polyveck_neg_coeffs_q_bound.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1822	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1823	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1824	lemma haetae_reference_keygen_public_vector_polyq_wf md sd :	States checked lemma haetae_reference_keygen_public_vector_polyq_wf. The following proof script must derive this proposition from prior definitions and lemmas.
1825	haetae_polyveck_polyq_coeffs_wf md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1826	(haetae_reference_keygen_public_vector md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1827	proof.	Begins the proof script for the preceding proposition.
1828	by rewrite /haetae_reference_keygen_public_vector;	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1829	apply haetae_public_key_vector_from_relation_polyq_wf.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1830	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1831	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1832	lemma haetae_keygen_public_vector_polyq_wf md sd :	States checked lemma haetae_keygen_public_vector_polyq_wf. The following proof script must derive this proposition from prior definitions and lemmas.
1833	haetae_polyveck_polyq_coeffs_wf md (haetae_keygen_public_vector md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1834	proof.	Begins the proof script for the preceding proposition.
1835	by rewrite /haetae_keygen_public_vector;	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1836	apply haetae_public_key_vector_from_relation_polyq_wf.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1837	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1838	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1839	lemma public_key_vector_seed_wf md sd :	States checked lemma public_key_vector_seed_wf. The following proof script must derive this proposition from prior definitions and lemmas.
1840	polyveck_wf md (public_key_vector_seed md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1841	proof.	Begins the proof script for the preceding proposition.
1842	have h := haetae_keygen_public_vector_polyq_wf md sd.	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
1843	by move: h; rewrite /haetae_polyveck_polyq_coeffs_wf /public_key_vector_seed.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1844	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1845	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1846	lemma public_key_mode23_column_wf md pk :	States checked lemma public_key_mode23_column_wf. The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
1847	<code>polyveck_wf md (public_key_mode23_column md pk).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1848	<code>proof. by rewrite /public_key_mode23_column</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1849	<code>/haetae_mode23_verification_column; apply polyveck_double_wf. qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1850	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1851	<code>lemma haetae_verification_column_from_unpacked_wf md sd b :</code>	States checked lemma <code>haetae_verification_column_from_unpacked_wf</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1852	<code>polyveck_wf md (haetae_verification_column_from_unpacked md sd b).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1853	<code>proof.</code>	Begins the proof script for the preceding proposition.
1854	<code>case md; rewrite /haetae_verification_column_from_unpacked /=.</code>	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
1855	<code>+ by rewrite /haetae_mode23_verification_column; apply polyveck_double_wf.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1856	<code>+ by rewrite /haetae_mode23_verification_column; apply polyveck_double_wf.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1857	<code>rewrite /haetae_mode5_verification_column.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1858	<code>rewrite /polyveck_wf size_mkseq.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1859	<code>split.</code>	Splits a conjunction or structured goal into independent proof obligations.
1860	<code>+ by trivial.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1861	<code>apply/allP=> p /mkseqP [i [_ ->]].</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1862	<code>by apply poly_normalize_wf.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1863	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1864	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1865	<code>lemma public_key_verification_column_wf md pk :</code>	States checked lemma <code>public_key_verification_column_wf</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1866	<code>polyveck_wf md (public_key_verification_column md pk).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1867	<code>proof.</code>	Begins the proof script for the preceding proposition.
1868	<code>case md; rewrite /public_key_verification_column /=.</code>	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
1869	<code>+ by apply public_key_mode23_column_wf.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1870	<code>+ by apply public_key_mode23_column_wf.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1871	<code>rewrite /public_key_mode5_column.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1872	<code>rewrite /polyveck_wf size_mkseq.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1873	<code>split.</code>	Splits a conjunction or structured goal into independent proof obligations.
1874	<code>+ by trivial.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1875	<code>apply/allP=> p /mkseqP [i [_ ->]].</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1876	<code>by apply poly_normalize_wf.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1877	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1878	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1879	<code>lemma public_verification_matrix_size md pk :</code>	States checked lemma <code>public_verification_matrix_size</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1880	<code>size (public_verification_matrix md pk) = mode_k md.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1881	<code>proof.</code>	Begins the proof script for the preceding proposition.
1882	<code>by rewrite /public_verification_matrix /packed_haetae_verification_matrix</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.

Line	EasyCrypt source	Mathematical reading
1883	/haetae_verification_matrix_from_unpacked size_mkseq; case md.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1884	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1885	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1886	lemma public_verification_matrix_rows_wf md pk :	States checked lemma public_verification_matrix_rows_wf. The following proof script must derive this proposition from prior definitions and lemmas.
1887	all (polyvecl_wf md) (public_verification_matrix md pk).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1888	proof.	Begins the proof script for the preceding proposition.
1889	rewrite /public_verification_matrix /packed_haetae_verification_matrix	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1890	/haetae_verification_matrix_from_unpacked.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1891	apply/allP=> row /mkseqP [i [i_rng ->]].	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1892	rewrite /polyvecl_wf size_mkseq.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1893	split.	Splits a conjunction or structured goal into independent proof obligations.
1894	+ by smt(mode_l_gt0).	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1895	apply/allP=> p /mkseqP [j [_ ->]].	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1896	case: (j = 0) => j0; rewrite j0 />.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
1897	+ have col_wf :=	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1898	haetae_verification_column_from_unpacked_wf md pk.'1 pk.'2.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1899	smt(polyveck_wf_nth).	Calls SMT automation for first-order arithmetic, list, or algebraic side conditions.
1900	by apply poly_double_wf.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1901	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1902	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1903	lemma challenge_embedding_wf md ch :	States checked lemma challenge_embedding_wf. The following proof script must derive this proposition from prior definitions and lemmas.
1904	challenge_wf ch => polyvecl_wf md (challenge_embedding md ch).	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1905	proof.	Begins the proof script for the preceding proposition.
1906	move=> ch_wf.	Introduces universally quantified variables or hypotheses into the proof context.
1907	rewrite /polyvecl_wf /challenge_embedding size_mkseq.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1908	split.	Splits a conjunction or structured goal into independent proof obligations.
1909	+ by case md.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1910	apply/allP=> p /mkseqP [i [_ ->]].	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1911	case: (i = 0) => i0; rewrite i0 />.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
1912	+ by rewrite /challenge_wf in ch_wf; smt.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1913	by apply poly_zero_wf.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1914	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1915	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1916	lemma public_key_of_secret_wf md sk :	States checked lemma public_key_of_secret_wf. The following proof script must derive this proposition from prior definitions and lemmas.
1917	polyveck_wf md (public_key_of_secret md sk).'2.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1918	proof. by rewrite /public_key_of_secret />; apply public_key_vector_seed_wf. qed.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1919	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
1920	lemma haetae_pack_seed_size sd :	States checked lemma <code>haetae_pack_seed_size</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1921	size (haetae_pack_seed sd) = seedbytes.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1922	proof. by rewrite /haetae_pack_seed size_mkseq /seedbytes. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1923	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1924	lemma haetae_unpack_seed_size enc :	States checked lemma <code>haetae_unpack_seed_size</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1925	size (haetae_unpack_seed enc) = seedbytes.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1926	proof. by rewrite /haetae_unpack_seed size_mkseq /seedbytes. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1927	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1928	lemma haetae_unpack_seed_pack sd :	States checked lemma <code>haetae_unpack_seed_pack</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1929	size sd = seedbytes =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1930	haetae_unpack_seed (haetae_pack_seed sd) = sd.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1931	proof.	Begins the proof script for the preceding proposition.
1932	move=> sd_sz.	Introduces universally quantified variables or hypotheses into the proof context.
1933	rewrite /haetae_unpack_seed /haetae_pack_seed.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1934	apply/(eq_from_nth 0).	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1935	+ by rewrite size_mkseq /seedbytes sd_sz.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1936	move=> i i_rng.	Introduces universally quantified variables or hypotheses into the proof context.
1937	rewrite size_mkseq /seedbytes in i_rng.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1938	rewrite nth_mkseq; first by smt().	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1939	by smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1940	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1941	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1942	lemma haetae_unpack_seed_pack_cat sd tail :	States checked lemma <code>haetae_unpack_seed_pack_cat</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1943	size sd = seedbytes =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1944	haetae_unpack_seed (haetae_pack_seed sd ++ tail) = sd.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1945	proof.	Begins the proof script for the preceding proposition.
1946	move=> sd_sz.	Introduces universally quantified variables or hypotheses into the proof context.
1947	rewrite /haetae_unpack_seed /haetae_pack_seed.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1948	apply/(eq_from_nth 0).	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1949	+ by rewrite size_mkseq /seedbytes sd_sz.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1950	move=> i i_rng.	Introduces universally quantified variables or hypotheses into the proof context.
1951	rewrite size_mkseq /seedbytes in i_rng.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1952	by smt(nth_cat nth_mkseq size_mkseq).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1953	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1954	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
1955	lemma haetae_polyq_coeff_bound_gt0 md :	States checked lemma haetae_polyq_coeff_bound_gt0. The following proof script must derive this proposition from prior definitions and lemmas.
1956	0 < haetae_polyq_coeff_bound md.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1957	proof.	Begins the proof script for the preceding proposition.
1958	by case md; rewrite /haetae_polyq_coeff_bound	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1959	/haetae_polyq_mode23_bound /haetae_polyq_mode5_bound.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1960	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1961	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1962	lemma haetae_byte_norm_bounds x :	States checked lemma haetae_byte_norm_bounds. The following proof script must derive this proposition from prior definitions and lemmas.
1963	0 <= haetae_byte_norm x < haetae_byte_modulus.	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1964	proof. by rewrite /haetae_byte_norm /haetae_byte_modulus; smt(). qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1965	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1966	lemma haetae_byte_norm_small x :	States checked lemma haetae_byte_norm_small. The following proof script must derive this proposition from prior definitions and lemmas.
1967	0 <= x < haetae_byte_modulus => haetae_byte_norm x = x.	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1968	proof. by rewrite /haetae_byte_norm /haetae_byte_modulus; smt(). qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1969	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1970	lemma haetae_byte_norm_idempotent x :	States checked lemma haetae_byte_norm_idempotent. The following proof script must derive this proposition from prior definitions and lemmas.
1971	haetae_byte_norm (haetae_byte_norm x) = haetae_byte_norm x.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1972	proof. by rewrite /haetae_byte_norm /haetae_byte_modulus; smt(). qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1973	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1974	lemma haetae_unpack15_coeff0 c0 c1 :	States checked lemma haetae_unpack15_coeff0. The following proof script must derive this proposition from prior definitions and lemmas.
1975	0 <= c0 < 32768 =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1976	0 <= c1 < 32768 =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1977	haetae_byte_norm c0 +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1978	256 * (haetae_byte_norm ((c0 %/ 256) %% 128 + (c1 %% 2) * 128) %% 128) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1979	c0.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1980	proof. by rewrite /haetae_byte_norm /haetae_byte_modulus; smt(). qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1981	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1982	lemma haetae_unpack15_coeff1 c0 c1 c2 :	States checked lemma haetae_unpack15_coeff1. The following proof script must derive this proposition from prior definitions and lemmas.
1983	0 <= c0 < 32768 =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1984	0 <= c1 < 32768 =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1985	0 <= c2 < 32768 =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1986	((haetae_byte_norm ((c0 %/ 256) %% 128 + (c1 %% 2) * 128) %/ 128) %% 2) +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1987	2 * haetae_byte_norm (c1 %/ 2) +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1988	512 * (haetae_byte_norm ((c1 %/ 512) %% 64 + (c2 %% 4) * 64) %% 64) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.

Line	EasyCrypt source	Mathematical reading
1989	<code>c1.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1990	<code>proof. by rewrite /haetae_byte_norm /haetae_byte_modulus; smt(). qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1991	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1992	<code>lemma haetae_unpack15_coeff2 c1 c2 c3 :</code>	States checked lemma <code>haetae_unpack15_coeff2</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1993	<code>0 <= c1 < 32768 =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1994	<code>0 <= c2 < 32768 =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1995	<code>0 <= c3 < 32768 =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1996	<code>((haetae_byte_norm ((c1 %/ 512) %% 64 + (c2 %% 4) * 64) %/ 64) %% 4) +</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1997	<code>4 * haetae_byte_norm (c2 %/ 4) +</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1998	<code>1024 * (haetae_byte_norm ((c2 %/ 1024) %% 32 + (c3 %% 8) * 32) %% 32) =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1999	<code>c2.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2000	<code>proof. by rewrite /haetae_byte_norm /haetae_byte_modulus; smt(). qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2001	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2002	<code>lemma haetae_unpack15_coeff3 c2 c3 c4 :</code>	States checked lemma <code>haetae_unpack15_coeff3</code> . The following proof script must derive this proposition from prior definitions and lemmas.
2003	<code>0 <= c2 < 32768 =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2004	<code>0 <= c3 < 32768 =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2005	<code>0 <= c4 < 32768 =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2006	<code>((haetae_byte_norm ((c2 %/ 1024) %% 32 + (c3 %% 8) * 32) %/ 32) %% 8) +</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2007	<code>8 * haetae_byte_norm (c3 %/ 8) +</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2008	<code>2048 * (haetae_byte_norm ((c3 %/ 2048) %% 16 + (c4 %% 16) * 16) %% 16) =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2009	<code>c3.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2010	<code>proof. by rewrite /haetae_byte_norm /haetae_byte_modulus; smt(). qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2011	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2012	<code>lemma haetae_unpack15_coeff4 c3 c4 c5 :</code>	States checked lemma <code>haetae_unpack15_coeff4</code> . The following proof script must derive this proposition from prior definitions and lemmas.
2013	<code>0 <= c3 < 32768 =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2014	<code>0 <= c4 < 32768 =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2015	<code>0 <= c5 < 32768 =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2016	<code>((haetae_byte_norm ((c3 %/ 2048) %% 16 + (c4 %% 16) * 16) %/ 16) %% 16) +</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2017	<code>16 * haetae_byte_norm (c4 %/ 16) +</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2018	<code>4096 * (haetae_byte_norm ((c4 %/ 4096) %% 8 + (c5 %% 32) * 8) %% 8) =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2019	<code>c4.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2020	<code>proof. by rewrite /haetae_byte_norm /haetae_byte_modulus; smt(). qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2021	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
2022	lemma haetae_unpack15_coeff5 c4 c5 c6 :	States checked lemma haetae_unpack15_coeff5. The following proof script must derive this proposition from prior definitions and lemmas.
2023	0 <= c4 < 32768 =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2024	0 <= c5 < 32768 =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2025	0 <= c6 < 32768 =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2026	((haetae_byte_norm ((c4 %/ 4096) %% 8 + (c5 %% 32) * 8) %/ 8) %% 32) +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2027	32 * haetae_byte_norm (c5 %/ 32) +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2028	8192 * (haetae_byte_norm ((c5 %/ 8192) %% 4 + (c6 %% 64) * 4) %% 4) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2029	c5.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2030	proof. by rewrite /haetae_byte_norm /haetae_byte_modulus; smt(). qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2031	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2032	lemma haetae_unpack15_coeff6 c5 c6 c7 :	States checked lemma haetae_unpack15_coeff6. The following proof script must derive this proposition from prior definitions and lemmas.
2033	0 <= c5 < 32768 =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2034	0 <= c6 < 32768 =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2035	0 <= c7 < 32768 =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2036	((haetae_byte_norm ((c5 %/ 8192) %% 4 + (c6 %% 64) * 4) %/ 4) %% 64) +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2037	64 * haetae_byte_norm (c6 %/ 64) +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2038	16384 * (haetae_byte_norm ((c6 %/ 16384) %% 2 + (c7 %% 128) * 2) %% 2) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2039	c6.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2040	proof. by rewrite /haetae_byte_norm /haetae_byte_modulus; smt(). qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2041	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2042	lemma haetae_unpack15_coeff7 c6 c7 :	States checked lemma haetae_unpack15_coeff7. The following proof script must derive this proposition from prior definitions and lemmas.
2043	0 <= c6 < 32768 =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2044	0 <= c7 < 32768 =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2045	((haetae_byte_norm ((c6 %/ 16384) %% 2 + (c7 %% 128) * 2) %/ 2) %% 128) +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2046	128 * haetae_byte_norm (c7 %/ 128) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2047	c7.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2048	proof. by rewrite /haetae_byte_norm /haetae_byte_modulus; smt(). qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2049	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2050	lemma haetae_pack_poly_q_ref_size md p :	States checked lemma haetae_pack_poly_q_ref_size. The following proof script must derive this proposition from prior definitions and lemmas.
2051	size (haetae_pack_poly_q_ref md p) = mode_polyq_packedbytes md.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2052	proof.	Begins the proof script for the preceding proposition.
2053	rewrite /haetae_pack_poly_q_ref.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2054	by case md; smt(size_mkseq).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2055	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.

Line	EasyCrypt source	Mathematical reading
2056	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2057	lemma haetae_unpack_poly_q_ref_at_wf md enc offset :	States checked lemma haetae_unpack_poly_q_ref_at_wf. The following proof script must derive this proposition from prior definitions and lemmas.
2058	poly_wf (haetae_unpack_poly_q_ref_at md enc offset).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2059	proof.	Begins the proof script for the preceding proposition.
2060	rewrite /haetae_unpack_poly_q_ref_at.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2061	case md; rewrite /poly_wf; smt(size_mkseq).	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
2062	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2063	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2064	lemma haetae_pack_polyveck_body_ref_size md b :	States checked lemma haetae_pack_polyveck_body_ref_size. The following proof script must derive this proposition from prior definitions and lemmas.
2065	size (haetae_pack_polyveck_body_ref md b) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2066	haetae_public_key_body_bytes md.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2067	proof.	Begins the proof script for the preceding proposition.
2068	by rewrite /haetae_pack_polyveck_body_ref /haetae_public_key_body_bytes	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2069	size_mkseq; case md.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2070	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2071	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2072	lemma haetae_unpack_polyveck_body_ref_wf md enc offset :	States checked lemma haetae_unpack_polyveck_body_ref_wf. The following proof script must derive this proposition from prior definitions and lemmas.
2073	polyveck_wf md (haetae_unpack_polyveck_body_ref md enc offset).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2074	proof.	Begins the proof script for the preceding proposition.
2075	rewrite /polyveck_wf /haetae_unpack_polyveck_body_ref size_mkseq.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2076	split.	Splits a conjunction or structured goal into independent proof obligations.
2077	+ by case md.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2078	apply/allP=> p /mkseqP [i [_ ->]].	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
2079	by apply haetae_unpack_poly_q_ref_at_wf.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2080	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2081	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2082	lemma haetae_pack_vk_ref_size md b sd :	States checked lemma haetae_pack_vk_ref_size. The following proof script must derive this proposition from prior definitions and lemmas.
2083	size (haetae_pack_vk_ref md b sd) = haetae_public_key_bytes md.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2084	proof.	Begins the proof script for the preceding proposition.
2085	rewrite /haetae_pack_vk_ref size_cat haetae_pack_seed_size	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2086	haetae_pack_polyveck_body_ref_size.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2087	by rewrite /haetae_public_key_bytes /haetae_public_key_body_bytes;	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2088	case md.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
2089	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2090	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2091	lemma haetae_unpack_vk_public_key_ref_wf md enc :	States checked lemma haetae_unpack_vk_public_key_ref_wf. The following proof script must derive this proposition from prior definitions and lemmas.
2092	haetae_public_key_unpacked_wf md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
2093	<code>(haetae_unpack_vk_public_key_ref md enc).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2094	<code>proof.</code>	Begins the proof script for the preceding proposition.
2095	<code>rewrite /haetae_public_key_unpacked_wf</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2096	<code>/haetae_unpack_vk_public_key_ref /=.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2097	<code>by split; [apply haetae_unpack_seed_size apply haetae_unpack_polyveck_body_ref_wf].</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2098	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
2099	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2100	<code>lemma nth_cat_size_plus ['a] (d : 'a) (xs ys : 'a list) j :</code>	States checked lemma <code>nth_cat_size_plus</code> . The following proof script must derive this proposition from prior definitions and lemmas.
2101	<code>0 <= j =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2102	<code>nth d (xs ++ ys) (size xs + j) = nth d ys j.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2103	<code>proof. by smt(nth_cat). qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2104	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2105	<code>lemma haetae_pack_polyveck_body_ref_nth md b row k :</code>	States checked lemma <code>haetae_pack_polyveck_body_ref_nth</code> . The following proof script must derive this proposition from prior definitions and lemmas.
2106	<code>0 <= row < mode_k md =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2107	<code>0 <= k < mode_polyq_packedbytes md =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2108	<code>nth 0 (haetae_pack_polyveck_body_ref md b)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2109	<code>(row * mode_polyq_packedbytes md + k) =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2110	<code>nth 0 (haetae_pack_poly_q_ref md (nth poly_zero b row)) k.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2111	<code>proof.</code>	Begins the proof script for the preceding proposition.
2112	<code>move=> row_rng k_rng.</code>	Introduces universally quantified variables or hypotheses into the proof context.
2113	<code>rewrite /haetae_pack_polyveck_body_ref.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2114	<code>rewrite nth_mkseq.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2115	<code>+ rewrite /haetae_public_key_body_bytes.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2116	<code>by smt(mode_polyq_packedbytes_gt0).</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2117	<code>by smt(nth_mkseq haetae_pack_poly_q_ref_size</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2118	<code>mode_polyq_packedbytes_gt0).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2119	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
2120	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2121	<code>lemma haetae_pack_polyveck_body_ref_mode5_nth b row k :</code>	States checked lemma <code>haetae_pack_polyveck_body_ref_mode5_nth</code> . The following proof script must derive this proposition from prior definitions and lemmas.
2122	<code>0 <= row < mode_k Mode5 =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2123	<code>0 <= k < mode_polyq_packedbytes Mode5 =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2124	<code>nth 0 (haetae_pack_polyveck_body_ref Mode5 b)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2125	<code>(row * mode_polyq_packedbytes Mode5 + k) =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2126	<code>nth 0 (haetae_pack_poly_q_ref Mode5 (nth poly_zero b row)) k.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2127	<code>proof.</code>	Begins the proof script for the preceding proposition.

Line	EasyCrypt source	Mathematical reading
2128	by apply haetae_pack_polyveck_body_ref_nth.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2129	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2130	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2131	lemma haetae_unpack_poly_q_mode23_coeff_at_cat prefix bytes blk ofs :	States checked lemma haetae_unpack_poly_q_mode23_coeff_at_cat. The following proof script must derive this proposition from prior definitions and lemmas.
2132	0 <= blk =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2133	haetae_unpack_poly_q_mode23_coeff_at	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2134	(prefix ++ bytes) (size prefix) blk ofs =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2135	haetae_unpack_poly_q_mode23_coeff_at bytes 0 blk ofs.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2136	proof.	Begins the proof script for the preceding proposition.
2137	move=> blk_ge0.	Introduces universally quantified variables or hypotheses into the proof context.
2138	rewrite /haetae_unpack_poly_q_mode23_coeff_at.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2139	by smt(nth_cat).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2140	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2141	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2142	lemma haetae_polyq_mode23_coeff_rng md p j :	States checked lemma haetae_polyq_mode23_coeff_rng. The following proof script must derive this proposition from prior definitions and lemmas.
2143	(md = Mode2 \/ md = Mode3) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2144	haetae_polyq_coeffs_wf md p =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2145	0 <= j < n =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2146	0 <= poly_coeff p j < 32768.	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2147	proof.	Begins the proof script for the preceding proposition.
2148	move=> md23_p_wf_j_rng.	Introduces universally quantified variables or hypotheses into the proof context.
2149	move: p_wf md23; rewrite /haetae_polyq_coeffs_wf	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2150	/haetae_polyq_coeff_bound	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2151	/haetae_polyq_mode23_bound => -[p_sz p_rng].	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2152	by smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2153	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2154	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2155	lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs0 md p blk :	States checked lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs0. The following proof script must derive this proposition from prior definitions and lemmas.
2156	(md = Mode2 \/ md = Mode3) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2157	haetae_polyq_coeffs_wf md p =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2158	0 <= blk < 32 =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2159	haetae_unpack_poly_q_mode23_coeff_at	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2160	(haetae_pack_poly_q_ref md p) 0 blk 0 =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2161	poly_coeff p (8 * blk + 0).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2162	proof.	Begins the proof script for the preceding proposition.

Line	EasyCrypt source	Mathematical reading
2163	<code>move=> md23 p_wf blk_rng.</code>	Introduces universally quantified variables or hypotheses into the proof context.
2164	<code>have c0_rng : 0 <= poly_coeff p (8 * blk + 0) < 32768</code>	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2165	<code>by smt(haetae_polyq_mode23_coeff_rng).</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2166	<code>have c1_rng : 0 <= poly_coeff p (8 * blk + 1) < 32768</code>	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2167	<code>by smt(haetae_polyq_mode23_coeff_rng).</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2168	<code>move: p_wf md23; case: md => /=.</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2169	<code>+ move=> p_wf2.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2170	<code>rewrite /haetae_unpack_poly_q_mode23_coeff_at</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2171	<code>/haetae_pack_poly_q_ref /haetae_pack_poly_q_mode23_byte /=.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2172	<code>by smt(nth_mkseq size_mkseq haetae_byte_norm_idempotent</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2173	<code>haetae_unpack15_coeff0).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2174	<code>+ move=> p_wf3.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2175	<code>rewrite /haetae_unpack_poly_q_mode23_coeff_at</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2176	<code>/haetae_pack_poly_q_ref /haetae_pack_poly_q_mode23_byte /=.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2177	<code>by smt(nth_mkseq size_mkseq haetae_byte_norm_idempotent</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2178	<code>haetae_unpack15_coeff0).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2179	<code>by smt().</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2180	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
2181	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2182	<code>lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs1 md p blk :</code>	States checked lemma
2183	<code>(md = Mode2 \ / md = Mode3) =></code>	<code>haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs1</code> . The following proof script must derive this proposition from prior definitions and lemmas.
2184	<code>haetae_polyq_coeffs_wf md p =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2185	<code>0 <= blk < 32 =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2186	<code>haetae_unpack_poly_q_mode23_coeff_at</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2187	<code>(haetae_pack_poly_q_ref md p) 0 blk 1 =</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2188	<code>poly_coeff p (8 * blk + 1).</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2189	<code>proof.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2190	<code>move=> md23 p_wf blk_rng.</code>	Begins the proof script for the preceding proposition.
2191	<code>have c0_rng : 0 <= poly_coeff p (8 * blk + 0) < 32768</code>	Introduces universally quantified variables or hypotheses into the proof context.
2192	<code>by smt(haetae_polyq_mode23_coeff_rng).</code>	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2193	<code>have c1_rng : 0 <= poly_coeff p (8 * blk + 1) < 32768</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2194	<code>by smt(haetae_polyq_mode23_coeff_rng).</code>	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2195	<code>have c2_rng : 0 <= poly_coeff p (8 * blk + 2) < 32768</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2196	<code>by smt(haetae_polyq_mode23_coeff_rng).</code>	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.

Line	EasyCrypt source	Mathematical reading
2197	move: p_wf md23; case: md => /=.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2198	+ move=> p_wf2.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2199	rewrite /haetae_unpack_poly_q_mode23_coeff_at	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2200	/haetae_pack_poly_q_ref /haetae_pack_poly_q_mode23_byte /=.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2201	by smt(nth_mkseq size_mkseq haetae_byte_norm_idempotent	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2202	haetae_unpack15_coeff1).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2203	+ move=> p_wf3.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2204	rewrite /haetae_unpack_poly_q_mode23_coeff_at	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2205	/haetae_pack_poly_q_ref /haetae_pack_poly_q_mode23_byte /=.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2206	by smt(nth_mkseq size_mkseq haetae_byte_norm_idempotent	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2207	haetae_unpack15_coeff1).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2208	by smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2209	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2210	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2211	lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs2 md p blk :	States checked lemma
2212	(md = Mode2 \ / md = Mode3) =>	haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs2. The following proof script must derive this proposition from prior definitions and lemmas.
2213	haetae_polyq_coeffs_wf md p =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2214	0 <= blk < 32 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2215	haetae_unpack_poly_q_mode23_coeff_at	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2216	(haetae_pack_poly_q_ref md p) 0 blk 2 =	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2217	poly_coeff p (8 * blk + 2).	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2218	proof.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2219	move=> md23 p_wf blk_rng.	Begins the proof script for the preceding proposition.
2220	have c1_rng : 0 <= poly_coeff p (8 * blk + 1) < 32768	Introduces universally quantified variables or hypotheses into the proof context.
2221	by smt(haetae_polyq_mode23_coeff_rng).	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2222	have c2_rng : 0 <= poly_coeff p (8 * blk + 2) < 32768	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2223	by smt(haetae_polyq_mode23_coeff_rng).	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2224	have c3_rng : 0 <= poly_coeff p (8 * blk + 3) < 32768	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2225	by smt(haetae_polyq_mode23_coeff_rng).	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2226	move: p_wf md23; case: md => /=.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2227	+ move=> p_wf2.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2228	rewrite /haetae_unpack_poly_q_mode23_coeff_at	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2229	/haetae_pack_poly_q_ref /haetae_pack_poly_q_mode23_byte /=.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2230	by smt(nth_mkseq size_mkseq haetae_byte_norm_idempotent	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
		Discharges the current goal in one step using the cited simplification, lemma, or automation.

Line	EasyCrypt source	Mathematical reading
2231	haetae_unpack15_coeff2).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2232	+ move=> p_wf3.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2233	rewrite /haetae_unpack_poly_q_mode23_coeff_at	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2234	/haetae_pack_poly_q_ref /haetae_pack_poly_q_mode23_byte /=.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2235	by smt(nth_mkseq size_mkseq haetae_byte_norm_idempotent	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2236	haetae_unpack15_coeff2).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2237	by smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2238	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2239	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2240	lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs3 md p blk :	States checked lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs3. The following proof script must derive this proposition from prior definitions and lemmas.
2241	(md = Mode2 \ / md = Mode3) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2242	haetae_polyq_coeffs_wf md p =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2243	0 <= blk < 32 =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2244	haetae_unpack_poly_q_mode23_coeff_at	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2245	(haetae_pack_poly_q_ref md p) 0 blk 3 =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2246	poly_coeff p (8 * blk + 3).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2247	proof.	Begins the proof script for the preceding proposition.
2248	move=> md23 p_wf blk_rng.	Introduces universally quantified variables or hypotheses into the proof context.
2249	have c2_rng : 0 <= poly_coeff p (8 * blk + 2) < 32768	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2250	by smt(haetae_polyq_mode23_coeff_rng).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2251	have c3_rng : 0 <= poly_coeff p (8 * blk + 3) < 32768	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2252	by smt(haetae_polyq_mode23_coeff_rng).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2253	have c4_rng : 0 <= poly_coeff p (8 * blk + 4) < 32768	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2254	by smt(haetae_polyq_mode23_coeff_rng).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2255	move: p_wf md23; case: md => /=.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2256	+ move=> p_wf2.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2257	rewrite /haetae_unpack_poly_q_mode23_coeff_at	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2258	/haetae_pack_poly_q_ref /haetae_pack_poly_q_mode23_byte /=.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2259	by smt(nth_mkseq size_mkseq haetae_byte_norm_idempotent	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2260	haetae_unpack15_coeff3).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2261	+ move=> p_wf3.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2262	rewrite /haetae_unpack_poly_q_mode23_coeff_at	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2263	/haetae_pack_poly_q_ref /haetae_pack_poly_q_mode23_byte /=.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2264	by smt(nth_mkseq size_mkseq haetae_byte_norm_idempotent	Discharges the current goal in one step using the cited simplification, lemma, or automation.

Line	EasyCrypt source	Mathematical reading
2265	haetae_unpack15_coeff3).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2266	by smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2267	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2268	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2269	lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs4 md p blk :	States checked lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs4. The following proof script must derive this proposition from prior definitions and lemmas.
2270	(md = Mode2 \ / md = Mode3) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2271	haetae_polyq_coeffs_wf md p =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2272	0 <= blk < 32 ==>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2273	haetae_unpack_poly_q_mode23_coeff_at	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2274	(haetae_pack_poly_q_ref md p) 0 blk 4 =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2275	poly_coeff p (8 * blk + 4).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2276	proof.	Begins the proof script for the preceding proposition.
2277	move=> md23 p_wf blk_rng.	Introduces universally quantified variables or hypotheses into the proof context.
2278	have c3_rng : 0 <= poly_coeff p (8 * blk + 3) < 32768	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2279	by smt(haetae_polyq_mode23_coeff_rng).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2280	have c4_rng : 0 <= poly_coeff p (8 * blk + 4) < 32768	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2281	by smt(haetae_polyq_mode23_coeff_rng).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2282	have c5_rng : 0 <= poly_coeff p (8 * blk + 5) < 32768	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2283	by smt(haetae_polyq_mode23_coeff_rng).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2284	move: p_wf md23; case: md => /=.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2285	+ move=> p_wf2.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2286	rewrite /haetae_unpack_poly_q_mode23_coeff_at	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2287	/haetae_pack_poly_q_ref /haetae_pack_poly_q_mode23_byte /=.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2288	by smt(nth_mkseq size_mkseq haetae_byte_norm_idempotent	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2289	haetae_unpack15_coeff4).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2290	+ move=> p_wf3.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2291	rewrite /haetae_unpack_poly_q_mode23_coeff_at	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2292	/haetae_pack_poly_q_ref /haetae_pack_poly_q_mode23_byte /=.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2293	by smt(nth_mkseq size_mkseq haetae_byte_norm_idempotent	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2294	haetae_unpack15_coeff4).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2295	by smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2296	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2297	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2298	lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs5 md p blk :	States checked lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs5. The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
2299	(md = Mode2 \ / md = Mode3) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2300	haetae_polyq_coeffs_wf md p =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2301	0 <= blk < 32 =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2302	haetae_unpack_poly_q_mode23_coeff_at	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2303	(haetae_pack_poly_q_ref md p) 0 blk 5 =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2304	poly_coeff p (8 * blk + 5).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2305	proof.	Begins the proof script for the preceding proposition.
2306	move=> md23 p_wf blk_rng.	Introduces universally quantified variables or hypotheses into the proof context.
2307	have c4_rng : 0 <= poly_coeff p (8 * blk + 4) < 32768	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2308	by smt(haetae_polyq_mode23_coeff_rng).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2309	have c5_rng : 0 <= poly_coeff p (8 * blk + 5) < 32768	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2310	by smt(haetae_polyq_mode23_coeff_rng).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2311	have c6_rng : 0 <= poly_coeff p (8 * blk + 6) < 32768	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2312	by smt(haetae_polyq_mode23_coeff_rng).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2313	move: p_wf md23; case: md => /=.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2314	+ move=> p_wf2.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2315	rewrite /haetae_unpack_poly_q_mode23_coeff_at	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2316	/haetae_pack_poly_q_ref /haetae_pack_poly_q_mode23_byte /=.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2317	by smt(nth_mkseq size_mkseq haetae_byte_norm_idempotent	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2318	haetae_unpack15_coeff5).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2319	+ move=> p_wf3.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2320	rewrite /haetae_unpack_poly_q_mode23_coeff_at	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2321	/haetae_pack_poly_q_ref /haetae_pack_poly_q_mode23_byte /=.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2322	by smt(nth_mkseq size_mkseq haetae_byte_norm_idempotent	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2323	haetae_unpack15_coeff5).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2324	by smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2325	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2326	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2327	lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs6 md p blk :	States checked lemma
2328	(md = Mode2 \ / md = Mode3) =>	haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs6. The following proof script must derive this proposition from prior definitions and lemmas.
2329	haetae_polyq_coeffs_wf md p =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2330	0 <= blk < 32 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2331	haetae_unpack_poly_q_mode23_coeff_at	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2332	(haetae_pack_poly_q_ref md p) 0 blk 6 =	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
		Part of an equality or definition, identifying two mathematical expressions or assigning a program value.

Line	EasyCrypt source	Mathematical reading
2333	<code>poly_coeff p (8 * blk + 6).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2334	<code>proof.</code>	Begins the proof script for the preceding proposition.
2335	<code>move=> md23 p_wf blk_rng.</code>	Introduces universally quantified variables or hypotheses into the proof context.
2336	<code>have c5_rng : 0 <= poly_coeff p (8 * blk + 5) < 32768</code>	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2337	<code>by smt(haetae_polyq_mode23_coeff_rng).</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2338	<code>have c6_rng : 0 <= poly_coeff p (8 * blk + 6) < 32768</code>	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2339	<code>by smt(haetae_polyq_mode23_coeff_rng).</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2340	<code>have c7_rng : 0 <= poly_coeff p (8 * blk + 7) < 32768</code>	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2341	<code>by smt(haetae_polyq_mode23_coeff_rng).</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2342	<code>move: p_wf md23; case: md => /=.</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2343	<code>+ move=> p_wf2.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2344	<code>rewrite /haetae_unpack_poly_q_mode23_coeff_at</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2345	<code>/haetae_pack_poly_q_ref /haetae_pack_poly_q_mode23_byte /=.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2346	<code>by smt(nth_mkseq size_mkseq haetae_byte_norm_idempotent</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2347	<code>haetae_unpack15_coeff6).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2348	<code>+ move=> p_wf3.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2349	<code>rewrite /haetae_unpack_poly_q_mode23_coeff_at</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2350	<code>/haetae_pack_poly_q_ref /haetae_pack_poly_q_mode23_byte /=.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2351	<code>by smt(nth_mkseq size_mkseq haetae_byte_norm_idempotent</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2352	<code>haetae_unpack15_coeff6).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2353	<code>by smt().</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2354	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
2355	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2356	<code>lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs7 md p blk :</code>	States checked lemma <code>haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs7</code> . The following proof script must derive this proposition from prior definitions and lemmas.
2357	<code>(md = Mode2 \ / md = Mode3) =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2358	<code>haetae_polyq_coeffs_wf md p =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2359	<code>0 <= blk < 32 =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2360	<code>haetae_unpack_poly_q_mode23_coeff_at</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2361	<code>(haetae_pack_poly_q_ref md p) 0 blk 7 =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2362	<code>poly_coeff p (8 * blk + 7).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2363	<code>proof.</code>	Begins the proof script for the preceding proposition.
2364	<code>move=> md23 p_wf blk_rng.</code>	Introduces universally quantified variables or hypotheses into the proof context.
2365	<code>have c6_rng : 0 <= poly_coeff p (8 * blk + 6) < 32768</code>	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2366	<code>by smt(haetae_polyq_mode23_coeff_rng).</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.

Line	EasyCrypt source	Mathematical reading
2367	have c7_rng : 0 <= poly_coeff p (8 * blk + 7) < 32768	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2368	by smt(haetae_polyq_mode23_coeff_rng).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2369	move: p_wf md23; case: md => /=.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2370	+ move=> p_wf2.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2371	rewrite /haetae_unpack_poly_q_mode23_coeff_at	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2372	/haetae_pack_poly_q_ref /haetae_pack_poly_q_mode23_byte /=.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2373	by smt(nth_mkseq size_mkseq haetae_byte_norm_idempotent	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2374	haetae_unpack15_coeff7).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2375	+ move=> p_wf3.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2376	rewrite /haetae_unpack_poly_q_mode23_coeff_at	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2377	/haetae_pack_poly_q_ref /haetae_pack_poly_q_mode23_byte /=.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2378	by smt(nth_mkseq size_mkseq haetae_byte_norm_idempotent	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2379	haetae_unpack15_coeff7).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2380	by smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2381	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2382	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2383	lemma haetae_unpack_poly_q_ref_pack_mode23_coeff md p i :	States checked lemma haetae_unpack_poly_q_ref_pack_mode23_coeff.
2384	(md = Mode2 $\vee$ md = Mode3) =>	The following proof script must derive this proposition from prior definitions and lemmas.
2385	haetae_polyq_coeffs_wf md p =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2386	0 <= i < n =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2387	haetae_unpack_poly_q_mode23_coeff_at	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2388	(haetae_pack_poly_q_ref md p) 0 (i % 8) (i % 8) =	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2389	poly_coeff p i.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2390	proof.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2391	move=> md23 p_wf i_rng.	Begins the proof script for the preceding proposition.
2392	have blk_rng : 0 <= i % 8 < 32 by rewrite /n in i_rng; smt().	Introduces universally quantified variables or hypotheses into the proof context.
2393	have ofs_cases :	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2394	i % 8 = 0 $\vee$ i % 8 = 1 $\vee$ i % 8 = 2 $\vee$ i % 8 = 3 $\vee$	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2395	i % 8 = 4 $\vee$ i % 8 = 5 $\vee$ i % 8 = 6 $\vee$ i % 8 = 7	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2396	by smt().	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2397	case: ofs_cases => [ofs0 ofs_cases].	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2398	+ rewrite ofs0.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
2399	have rhs0 : poly_coeff p i = poly_coeff p (8 * (i % 8) + 0)	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2400	by smt().	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
		Discharges the current goal in one step using the cited simplification, lemma, or automation.

Line	EasyCrypt source	Mathematical reading
2401	rewrite rhs0.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2402	by apply haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs0.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2403	case: ofs_cases => [ofs1 ofs_cases].	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
2404	+ rewrite ofs1.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2405	have rhs1 : poly_coeff p i = poly_coeff p (8 * (i % 8) + 1)	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2406	by smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2407	rewrite rhs1.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2408	by apply haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs1.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2409	case: ofs_cases => [ofs2 ofs_cases].	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
2410	+ rewrite ofs2.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2411	have rhs2 : poly_coeff p i = poly_coeff p (8 * (i % 8) + 2)	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2412	by smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2413	rewrite rhs2.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2414	by apply haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs2.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2415	case: ofs_cases => [ofs3 ofs_cases].	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
2416	+ rewrite ofs3.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2417	have rhs3 : poly_coeff p i = poly_coeff p (8 * (i % 8) + 3)	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2418	by smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2419	rewrite rhs3.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2420	by apply haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs3.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2421	case: ofs_cases => [ofs4 ofs_cases].	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
2422	+ rewrite ofs4.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2423	have rhs4 : poly_coeff p i = poly_coeff p (8 * (i % 8) + 4)	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2424	by smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2425	rewrite rhs4.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2426	by apply haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs4.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2427	case: ofs_cases => [ofs5 ofs_cases].	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
2428	+ rewrite ofs5.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2429	have rhs5 : poly_coeff p i = poly_coeff p (8 * (i % 8) + 5)	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2430	by smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2431	rewrite rhs5.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2432	by apply haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs5.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2433	case: ofs_cases => [ofs6 ofs7].	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.

Line	EasyCrypt source	Mathematical reading
2434	+ rewrite ofs6.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2435	have rhs6 : poly_coeff p i = poly_coeff p (8 * (i %/ 8) + 6)	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2436	by smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2437	rewrite rhs6.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2438	by apply haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs6.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2439	rewrite ofs7.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2440	have rhs7 : poly_coeff p i = poly_coeff p (8 * (i %/ 8) + 7)	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2441	by smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2442	rewrite rhs7.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2443	by apply haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs7.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2444	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2445	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2446	lemma haetae_unpack_poly_q_ref_pack_mode23_at md prefix p :	States checked lemma haetae_unpack_poly_q_ref_pack_mode23_at. The following proof script must derive this proposition from prior definitions and lemmas.
2447	(md = Mode2 \ / md = Mode3) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2448	haetae_polyq_coeffs_wf md p =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2449	haetae_unpack_poly_q_ref_at md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2450	(prefix ++ haetae_pack_poly_q_ref md p) (size prefix) = p.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2451	proof.	Begins the proof script for the preceding proposition.
2452	move=> md23 p_wf.	Introduces universally quantified variables or hypotheses into the proof context.
2453	case: md md23 p_wf => /=.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
2454	+ move=> p_wf2.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2455	rewrite /haetae_unpack_poly_q_ref_at /=.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2456	apply/(eq_from_nth 0).	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
2457	+ rewrite size_mkseq.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2458	move: p_wf2; rewrite /haetae_polyq_coeffs_wf /poly_wf => -[p_sz _].	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2459	by rewrite p_sz.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2460	move=> i_i_rng.	Introduces universally quantified variables or hypotheses into the proof context.
2461	rewrite size_mkseq in i_i_rng.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2462	rewrite nth_mkseq; first by smt().	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2463	by smt(haetae_unpack_poly_q_mode23_coeff_at_cat	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2464	haetae_unpack_poly_q_ref_pack_mode23_coeff).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2465	+ move=> p_wf3.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2466	rewrite /haetae_unpack_poly_q_ref_at /=.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2467	apply/(eq_from_nth 0).	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.

Line	EasyCrypt source	Mathematical reading
2468	+ rewrite size_mkseq.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2469	move: p_wf3; rewrite /haetae_polyq_coeffs_wf /poly_wf => -[p_sz _].	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2470	by rewrite p_sz.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2471	move=> i i_rng.	Introduces universally quantified variables or hypotheses into the proof context.
2472	rewrite size_mkseq in i_rng.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2473	rewrite nth_mkseq; first by smt().	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2474	by smt(haetae_unpack_poly_q_mode23_coeff_at_cat	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2475	haetae_unpack_poly_q_ref_pack_mode23_coeff).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2476	by smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2477	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2478	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2479	lemma haetae_unpack_poly_q_mode23_coeff_at_body md prefix b row blk ofs :	States checked lemma haetae_unpack_poly_q_mode23_coeff_at_body.
		The following proof script must derive this proposition from prior definitions and lemmas.
2480	(md = Mode2 \/ md = Mode3) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2481	0 <= row < mode_k md =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2482	0 <= blk < 32 =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2483	haetae_unpack_poly_q_mode23_coeff_at	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2484	(prefix ++ haetae_pack_polyveck_body_ref md b)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2485	(size prefix + row * mode_polyq_packedbytes md) blk ofs =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2486	haetae_unpack_poly_q_mode23_coeff_at	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2487	(haetae_pack_poly_q_ref md (nth poly_zero b row)) 0 blk ofs.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2488	proof.	Begins the proof script for the preceding proposition.
2489	move=> md23 row_rng blk_rng.	Introduces universally quantified variables or hypotheses into the proof context.
2490	rewrite /haetae_unpack_poly_q_mode23_coeff_at.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2491	by smt(nth_cat haetae_pack_polyveck_body_ref_nth	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2492	mode_polyq_packedbytes_gt0).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2493	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2494	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2495	lemma haetae_unpack_polyveck_body_ref_pack_mode23_coeff md prefix b row i :	States checked lemma
		haetae_unpack_polyveck_body_ref_pack_mode23_coeff. The following proof script must derive this proposition from prior definitions and lemmas.
2496	(md = Mode2 \/ md = Mode3) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2497	haetae_polyveck_polyq_coeffs_wf md b =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2498	0 <= row < mode_k md =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2499	0 <= i < n =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2500	haetae_unpack_poly_q_mode23_coeff_at	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2501	(prefix ++ haetae_pack_polyveck_body_ref md b)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
2502	(size prefix + row * mode_polyq_packedbytes md) (i %/ 8) (i %% 8) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2503	poly_coeff (nth poly_zero b row) i.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2504	proof.	Begins the proof script for the preceding proposition.
2505	move=> md23 b_wf row_rng i_rng.	Introduces universally quantified variables or hypotheses into the proof context.
2506	have row_wf :	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2507	haetae_polyq_coeffs_wf md (nth poly_zero b row).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2508	+ move: b_wf; rewrite /haetae_polyveck_polyq_coeffs_wf => -[_ b_rng].	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2509	by apply b_rng.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2510	rewrite (haetae_unpack_poly_q_mode23_coeff_at_body md prefix b row	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2511	(i %/ 8) (i %% 8)).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2512	+ by smt().	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2513	+ by smt().	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2514	+ by rewrite /n in i_rng; smt().	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2515	by apply haetae_unpack_poly_q_ref_pack_mode23_coeff.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2516	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2517	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2518	lemma haetae_unpack_polyveck_body_ref_pack_mode23 md prefix b :	States checked lemma <code>haetae_unpack_polyveck_body_ref_pack_mode23</code> . The following proof script must derive this proposition from prior definitions and lemmas.
2519	(md = Mode2 \ / md = Mode3) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2520	haetae_polyveck_polyq_coeffs_wf md b =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2521	haetae_unpack_polyveck_body_ref md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2522	(prefix ++ haetae_pack_polyveck_body_ref md b)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2523	(size prefix) = b.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2524	proof.	Begins the proof script for the preceding proposition.
2525	move=> md23 b_wf.	Introduces universally quantified variables or hypotheses into the proof context.
2526	rewrite /haetae_unpack_polyveck_body_ref.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2527	apply/(eq_from_nth poly_zero).	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
2528	+ rewrite size_mkseq.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2529	move: b_wf; rewrite /haetae_polyveck_polyq_coeffs_wf	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2530	/polyveck_wf => -[[b_sz _] _].	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2531	by smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2532	move=> row row_rng.	Introduces universally quantified variables or hypotheses into the proof context.
2533	rewrite size_mkseq in row_rng.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2534	rewrite nth_mkseq; first by smt().	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2535	case: md md23 b_wf row_rng => /=.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.

Line	EasyCrypt source	Mathematical reading
2536	+ move=> b_wf2 row_rng2.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2537	rewrite /haetae_unpack_poly_q_ref_at /.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2538	apply/(eq_from_nth 0).	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
2539	+ rewrite size_mkseq.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2540	move: b_wf2; rewrite /haetae_polyveck_polyq_coeffs_wf	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2541	/polyveck_wf => -[[b_sz _] b_rng].	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2542	have row_p_wf := b_rng row _.	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2543	+ by smt().	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2544	move: row_p_wf; rewrite /haetae_polyq_coeffs_wf /poly_wf => -[row_sz _].	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2545	by rewrite row_sz.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2546	move=> i i_rng.	Introduces universally quantified variables or hypotheses into the proof context.
2547	rewrite size_mkseq in i_rng.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2548	rewrite nth_mkseq; first by smt().	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2549	by apply haetae_unpack_polyveck_body_ref_pack_mode23_coeff.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2550	+ move=> b_wf3 row_rng3.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2551	rewrite /haetae_unpack_poly_q_ref_at /.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2552	apply/(eq_from_nth 0).	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
2553	+ rewrite size_mkseq.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2554	move: b_wf3; rewrite /haetae_polyveck_polyq_coeffs_wf	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2555	/polyveck_wf => -[[b_sz _] b_rng].	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2556	have row_p_wf := b_rng row _.	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2557	+ by smt().	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2558	move: row_p_wf; rewrite /haetae_polyq_coeffs_wf /poly_wf => -[row_sz _].	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2559	by rewrite row_sz.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2560	move=> i i_rng.	Introduces universally quantified variables or hypotheses into the proof context.
2561	rewrite size_mkseq in i_rng.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2562	rewrite nth_mkseq; first by smt().	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2563	by apply haetae_unpack_polyveck_body_ref_pack_mode23_coeff.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2564	by smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2565	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2566	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2567	lemma haetae_pack_vk_ref_roundtrip_mode23 md sd b :	States checked lemma haetae_pack_vk_ref_roundtrip_mode23. The following proof script must derive this proposition from prior definitions and lemmas.
2568	(md = Mode2 \ / md = Mode3) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.

Line	EasyCrypt source	Mathematical reading
2569	size sd = seedbytes =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2570	haetae_polyveck_polyq_coeffs_wf md b =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2571	haetae_unpack_vk_public_key_ref md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2572	(haetae_pack_vk_ref md b sd) = (sd, b).	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2573	proof.	Begins the proof script for the preceding proposition.
2574	move=> md23 sd_sz b_wf.	Introduces universally quantified variables or hypotheses into the proof context.
2575	rewrite /haetae_unpack_vk_public_key_ref /haetae_pack_vk_ref /.=.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2576	rewrite (haetae_unpack_seed_pack_cat sd	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2577	(haetae_pack_polyveck_body_ref md b) sd_sz).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2578	have seed_sz : seedbytes = size (haetae_pack_seed sd).	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2579	+ by rewrite haetae_pack_seed_size.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2580	rewrite seed_sz.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2581	by rewrite (haetae_unpack_polyveck_body_ref_pack_mode23	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2582	md (haetae_pack_seed sd) b md23 b_wf).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2583	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2584	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2585	lemma haetae_unpack_poly_q_ref_pack_mode5_coeff p i :	States checked lemma <code>haetae_unpack_poly_q_ref_pack_mode5_coeff</code> . The following proof script must derive this proposition from prior definitions and lemmas.
2586	haetae_polyq_coeffs_wf Mode5 p =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2587	0 <= i < n =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2588	haetae_unpack_poly_q_mode5_coeff_at	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2589	(haetae_pack_poly_q_ref Mode5 p) 0 i = poly_coeff p i.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2590	proof.	Begins the proof script for the preceding proposition.
2591	move=> p_wf i_rng.	Introduces universally quantified variables or hypotheses into the proof context.
2592	rewrite /haetae_unpack_poly_q_mode5_coeff_at	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2593	/haetae_pack_poly_q_ref /= /haetae_pack_poly_q_mode5_byte	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2594	/haetae_byte_norm /haetae_byte_modulus.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2595	move: p_wf; rewrite /haetae_polyq_coeffs_wf	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2596	/haetae_polyq_coeff_bound	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2597	/haetae_polyq_mode5_bound => -[p_sz p_rng].	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2598	have ci_rng :	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2599	0 <= poly_coeff p i < 65536 by smt().	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2600	by smt(nth_mkseq size_mkseq).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2601	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2602	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
2603	lemma haetae_unpack_poly_q_ref_pack_mode5_at prefix p :	States checked lemma <code>haetae_unpack_poly_q_ref_pack_mode5_at</code> . The following proof script must derive this proposition from prior definitions and lemmas.
2604	haetae_polyq_coeffs_wf Mode5 p =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2605	haetae_unpack_poly_q_ref_at Mode5	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2606	(prefix ++ haetae_pack_poly_q_ref Mode5 p) (size prefix) = p.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2607	proof.	Begins the proof script for the preceding proposition.
2608	move=> p_wf.	Introduces universally quantified variables or hypotheses into the proof context.
2609	rewrite /haetae_unpack_poly_q_ref_at /=.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2610	apply/(eq_from_nth 0).	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
2611	+ rewrite size_mkseq.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2612	move: p_wf; rewrite /haetae_polyq_coeffs_wf /poly_wf => -[p_sz _].	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2613	by rewrite p_sz.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2614	move=> i i_rng.	Introduces universally quantified variables or hypotheses into the proof context.
2615	rewrite size_mkseq in i_rng.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2616	rewrite nth_mkseq; first by smt().	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2617	rewrite /haetae_unpack_poly_q_mode5_coeff_at	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2618	/haetae_pack_poly_q_ref /= /haetae_pack_poly_q_mode5_byte	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2619	/haetae_byte_norm /haetae_byte_modulus.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2620	move: p_wf; rewrite /haetae_polyq_coeffs_wf	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2621	/haetae_polyq_coeff_bound	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2622	/haetae_polyq_mode5_bound => -[p_sz p_rng].	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2623	have ci_rng :	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2624	0 <= poly_coeff p i < 65536 by smt().	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2625	by smt(nth_cat nth_mkseq size_mkseq).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2626	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2627	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2628	lemma haetae_unpack_polyveck_body_ref_pack_mode5_coeff prefix b row i :	States checked lemma <code>haetae_unpack_polyveck_body_ref_pack_mode5_coeff</code> . The following proof script must derive this proposition from prior definitions and lemmas.
2629	haetae_polyveck_polyq_coeffs_wf Mode5 b =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2630	0 <= row < mode_k Mode5 =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2631	0 <= i < n =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2632	haetae_unpack_poly_q_mode5_coeff_at	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2633	(prefix ++ haetae_pack_polyveck_body_ref Mode5 b)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2634	(size prefix + row * mode_polyq_packedbytes Mode5) i =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2635	poly_coeff (nth poly_zero b row) i.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
2636	proof.	Begins the proof script for the preceding proposition.
2637	move=> b_wf row_rng i_rng.	Introduces universally quantified variables or hypotheses into the proof context.
2638	have row_wf :	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2639	haetae_polyq_coefs_wf Mode5 (nth poly_zero b row).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2640	+ move: b_wf; rewrite /haetae_polyveck_polyq_coefs_wf => -[_ b_rng].	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2641	by apply b_rng.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2642	rewrite /haetae_unpack_poly_q_mode5_coeff_at.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2643	have lo :	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2644	nth 0 (prefix ++ haetae_pack_polyveck_body_ref Mode5 b)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2645	(size prefix + row * mode_polyq_packedbytes Mode5 + 2 * i) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2646	nth 0 (haetae_pack_poly_q_ref Mode5 (nth poly_zero b row))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2647	(2 * i).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2648	+ have k_rng : 0 <= 2 * i < mode_polyq_packedbytes Mode5 by smt().	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2649	have idxE :	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2650	size prefix + row * mode_polyq_packedbytes Mode5 + 2 * i =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2651	size prefix + (row * mode_polyq_packedbytes Mode5 + 2 * i) by smt().	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2652	have catE :	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2653	nth 0 (prefix ++ haetae_pack_polyveck_body_ref Mode5 b)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2654	(size prefix + (row * mode_polyq_packedbytes Mode5 + 2 * i)) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2655	nth 0 (haetae_pack_polyveck_body_ref Mode5 b)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2656	(row * mode_polyq_packedbytes Mode5 + 2 * i).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2657	+ by apply nth_cat_size_plus; smt().	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2658	rewrite idxE catE.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2659	by apply haetae_pack_polyveck_body_ref_mode5_nth.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2660	have hi :	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2661	nth 0 (prefix ++ haetae_pack_polyveck_body_ref Mode5 b)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2662	(size prefix + row * mode_polyq_packedbytes Mode5 + 2 * i + 1) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2663	nth 0 (haetae_pack_poly_q_ref Mode5 (nth poly_zero b row))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2664	(2 * i + 1).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2665	+ have k_rng : 0 <= 2 * i + 1 < mode_polyq_packedbytes Mode5 by smt().	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2666	have idxE :	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2667	size prefix + row * mode_polyq_packedbytes Mode5 + 2 * i + 1 =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2668	size prefix + (row * mode_polyq_packedbytes Mode5 + (2 * i + 1))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2669	by smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.

Line	EasyCrypt source	Mathematical reading
2670	have catE :	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2671	nth 0 (prefix ++ haetae_pack_polyveck_body_ref Mode5 b)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2672	(size prefix + (row * mode_polyq_packedbytes Mode5 + (2 * i + 1))) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2673	nth 0 (haetae_pack_polyveck_body_ref Mode5 b)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2674	(row * mode_polyq_packedbytes Mode5 + (2 * i + 1)).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2675	+ by apply nth_cat_size_plus; smt().	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2676	rewrite idxE catE.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2677	by apply haetae_pack_polyveck_body_ref_mode5_nth.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2678	rewrite lo hi.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2679	rewrite /(haetae_unpack_poly_q_mode5_coeff_at	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2680	(haetae_pack_poly_q_ref Mode5 (nth poly_zero b row)) 0 i).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2681	by apply haetae_unpack_poly_q_ref_pack_mode5_coeff.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2682	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2683	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2684	lemma haetae_unpack_polyveck_body_ref_pack_mode5 prefix b :	States checked lemma <code>haetae_unpack_polyveck_body_ref_pack_mode5</code> . The following proof script must derive this proposition from prior definitions and lemmas.
2685	haetae_polyveck_polyq_coeffs_wf Mode5 b =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2686	haetae_unpack_polyveck_body_ref Mode5	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2687	(prefix ++ haetae_pack_polyveck_body_ref Mode5 b)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2688	(size prefix) = b.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2689	proof.	Begins the proof script for the preceding proposition.
2690	move=> b_wf.	Introduces universally quantified variables or hypotheses into the proof context.
2691	rewrite /haetae_unpack_polyveck_body_ref /.=.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2692	apply/(eq_from_nth poly_zero).	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
2693	+ rewrite size_mkseq.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2694	move: b_wf; rewrite /haetae_polyveck_polyq_coeffs_wf	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2695	/polyveck_wf => -[[b_sz _] _].	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2696	by rewrite b_sz.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2697	move=> row row_rng.	Introduces universally quantified variables or hypotheses into the proof context.
2698	rewrite size_mkseq in row_rng.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2699	rewrite nth_mkseq; first by smt().	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2700	rewrite /haetae_unpack_poly_q_ref_at /.=.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2701	apply/(eq_from_nth 0).	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
2702	+ rewrite size_mkseq.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2703	move: b_wf; rewrite /haetae_polyveck_polyq_coeffs_wf	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
2704	/polyveck_wf => -[[b_sz _] b_rng].	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2705	have row_p_wf := b_rng row _.	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2706	+ by smt().	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2707	move: row_p_wf; rewrite /haetae_polyq_coeffs_wf /poly_wf => -[row_sz _].	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2708	by rewrite row_sz.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2709	move=> i i_rng.	Introduces universally quantified variables or hypotheses into the proof context.
2710	rewrite size_mkseq in i_rng.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2711	rewrite nth_mkseq; first by smt().	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2712	by apply haetae_unpack_polyveck_body_ref_pack_mode5_coeff.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2713	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2714	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2715	lemma haetae_pack_vk_ref_roundtrip_mode5 sd b :	States checked lemma <code>haetae_pack_vk_ref_roundtrip_mode5</code> . The following proof script must derive this proposition from prior definitions and lemmas.
2716	size sd = seedbytes =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2717	haetae_polyveck_polyq_coeffs_wf Mode5 b =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2718	haetae_unpack_vk_public_key_ref Mode5	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2719	(haetae_pack_vk_ref Mode5 b sd) = (sd, b).	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2720	proof.	Begins the proof script for the preceding proposition.
2721	move=> sd_sz b_wf.	Introduces universally quantified variables or hypotheses into the proof context.
2722	rewrite /haetae_unpack_vk_public_key_ref /haetae_pack_vk_ref /.=.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2723	rewrite (haetae_unpack_seed_pack_cat sd	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2724	(haetae_pack_polyveck_body_ref Mode5 b) sd_sz).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2725	have seed_sz : seedbytes = size (haetae_pack_seed sd).	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2726	+ by rewrite haetae_pack_seed_size.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2727	rewrite seed_sz.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2728	by rewrite (haetae_unpack_polyveck_body_ref_pack_mode5	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2729	(haetae_pack_seed sd) b b_wf).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2730	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2731	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2732	lemma haetae_public_key_ref_roundtrip md (pk : pkey) :	States checked lemma <code>haetae_public_key_ref_roundtrip</code> . The following proof script must derive this proposition from prior definitions and lemmas.
2733	size pk.'1 = seedbytes =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2734	haetae_polyveck_polyq_coeffs_wf md pk.'2 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2735	haetae_unpack_public_key_bytes_ref md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2736	(haetae_pack_public_key_bytes_ref md pk) = pk.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2737	proof.	Begins the proof script for the preceding proposition.
2738	case: pk => sd b /.=.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.

Line	EasyCrypt source	Mathematical reading
2739	<code>move=> sd_sz b_wf.</code>	Introduces universally quantified variables or hypotheses into the proof context.
2740	<code>rewrite /haetae_unpack_public_key_bytes_ref</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2741	<code>/haetae_pack_public_key_bytes_ref.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2742	<code>case: md b_wf => /= b_wf.</code>	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
2743	<code>+ apply haetae_pack_vk_ref_roundtrip_mode23.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2744	<code>+ by left.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2745	<code>+ by apply sd_sz.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2746	<code>by apply b_wf.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2747	<code>+ apply haetae_pack_vk_ref_roundtrip_mode23.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2748	<code>+ by right.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2749	<code>+ by apply sd_sz.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2750	<code>by apply b_wf.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2751	<code>by apply haetae_pack_vk_ref_roundtrip_mode5;</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2752	<code>[apply sd_sz apply b_wf].</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2753	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
2754	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2755	<code>lemma haetae_keygen_public_key_ref_roundtrip md sd :</code>	States checked lemma <code>haetae_keygen_public_key_ref_roundtrip</code> . The following proof script must derive this proposition from prior definitions and lemmas.
2756	<code>size sd = seedbytes =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2757	<code>haetae_unpack_vk_public_key_ref md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2758	<code>(haetae_pack_vk_ref md (haetae_keygen_public_vector md sd) sd) =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2759	<code>packed_haetae_public_key md sd.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2760	<code>proof.</code>	Begins the proof script for the preceding proposition.
2761	<code>move=> sd_sz.</code>	Introduces universally quantified variables or hypotheses into the proof context.
2762	<code>rewrite /packed_haetae_public_key.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2763	<code>have h :=</code>	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2764	<code>haetae_public_key_ref_roundtrip md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2765	<code>(sd, haetae_keygen_public_vector md sd)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2766	<code>sd_sz (haetae_keygen_public_vector_polyq_wf md sd).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2767	<code>by move: h; rewrite /haetae_unpack_public_key_bytes_ref</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2768	<code>/haetae_pack_public_key_bytes_ref.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2769	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
2770	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2771	<code>lemma haetae_pack_polyveck_body_size md b :</code>	States checked lemma <code>haetae_pack_polyveck_body_size</code> . The following proof script must derive this proposition from prior definitions and lemmas.
2772	<code>size (haetae_pack_polyveck_body md b) = haetae_public_key_body_bytes md.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.

Line	EasyCrypt source	Mathematical reading
2773	proof.	Begins the proof script for the preceding proposition.
2774	by rewrite /haetae_pack_polyveck_body /haetae_public_key_body_bytes	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2775	size_mkseq; case md.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2776	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2777	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2778	lemma haetae_pack_poly_q_size md p :	States checked lemma haetae_pack_poly_q_size. The following proof script must derive this proposition from prior definitions and lemmas.
2779	size (haetae_pack_poly_q md p) = mode_polyq_packedbytes md.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2780	proof. by rewrite /haetae_pack_poly_q_size_mkseq; case md. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2781	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2782	lemma haetae_unpack_poly_q_at_wf md enc offset :	States checked lemma haetae_unpack_poly_q_at_wf. The following proof script must derive this proposition from prior definitions and lemmas.
2783	poly_wf (haetae_unpack_poly_q_at md enc offset).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2784	proof. by rewrite /poly_wf /haetae_unpack_poly_q_at size_mkseq /n. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2785	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2786	lemma haetae_unpack_poly_q_pack_at md prefix p :	States checked lemma haetae_unpack_poly_q_pack_at. The following proof script must derive this proposition from prior definitions and lemmas.
2787	poly_wf p =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2788	haetae_unpack_poly_q_at md (prefix ++ haetae_pack_poly_q md p)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2789	(size prefix) = p.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2790	proof.	Begins the proof script for the preceding proposition.
2791	move=> p_wf.	Introduces universally quantified variables or hypotheses into the proof context.
2792	rewrite /haetae_unpack_poly_q_at.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2793	apply/(eq_from_nth 0).	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
2794	+ rewrite size_mkseq.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2795	by move: p_wf; rewrite /poly_wf => ->.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2796	move=> i i_rng.	Introduces universally quantified variables or hypotheses into the proof context.
2797	rewrite size_mkseq in i_rng.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2798	rewrite nth_mkseq; first by smt().	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2799	rewrite /haetae_pack_poly_q /poly_coeff.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2800	by smt(nth_cat nth_mkseq n_le_mode_polyq_packedbytes).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2801	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2802	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2803	lemma haetae_unpack_polyveck_body_wf md enc offset :	States checked lemma haetae_unpack_polyveck_body_wf. The following proof script must derive this proposition from prior definitions and lemmas.
2804	polyveck_wf md (haetae_unpack_polyveck_body md enc offset).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2805	proof.	Begins the proof script for the preceding proposition.
2806	rewrite /polyveck_wf /haetae_unpack_polyveck_body size_mkseq.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2807	split.	Splits a conjunction or structured goal into independent proof obligations.
2808	+ by case md.	Solves a proof branch produced by a previous split, transitivity, or case analysis.

Line	EasyCrypt source	Mathematical reading
2809	<code>apply/allP=> p /mkseqP [i [_ ->]].</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
2810	<code>by rewrite /poly_wf size_mkseq /n.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2811	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
2812	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2813	<code>lemma haetae_body_index_div md row col :</code>	States checked lemma <code>haetae_body_index_div</code> . The following proof script must derive this proposition from prior definitions and lemmas.
2814	<code>0 <= row =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2815	<code>0 <= col < n =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2816	<code>(row * mode_polyq_packedbytes md + col)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2817	<code>%/ mode_polyq_packedbytes md = row.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2818	<code>proof. by case md; rewrite /mode_polyq_packedbytes /n; smt(). qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2819	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2820	<code>lemma haetae_body_index_mod md row col :</code>	States checked lemma <code>haetae_body_index_mod</code> . The following proof script must derive this proposition from prior definitions and lemmas.
2821	<code>0 <= row =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2822	<code>0 <= col < n =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2823	<code>(row * mode_polyq_packedbytes md + col)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2824	<code>%% mode_polyq_packedbytes md = col.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2825	<code>proof. by case md; rewrite /mode_polyq_packedbytes /n; smt(). qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2826	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2827	<code>lemma haetae_body_index_lt md row col :</code>	States checked lemma <code>haetae_body_index_lt</code> . The following proof script must derive this proposition from prior definitions and lemmas.
2828	<code>0 <= row < mode_k md =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2829	<code>0 <= col < n =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2830	<code>row * mode_polyq_packedbytes md + col <</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2831	<code>haetae_public_key_body_bytes md.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2832	<code>proof.</code>	Begins the proof script for the preceding proposition.
2833	<code>rewrite /haetae_public_key_body_bytes.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2834	<code>by smt(n_le_mode_polyq_packedbytes mode_polyq_packedbytes_gt0).</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2835	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
2836	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2837	<code>lemma haetae_unpack_polyveck_body_pack_at md prefix b :</code>	States checked lemma <code>haetae_unpack_polyveck_body_pack_at</code> . The following proof script must derive this proposition from prior definitions and lemmas.
2838	<code>polyveck_wf md b =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2839	<code>haetae_unpack_polyveck_body md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2840	<code>(prefix ++ haetae_pack_polyveck_body md b) (size prefix) = b.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2841	<code>proof.</code>	Begins the proof script for the preceding proposition.
2842	<code>move=> b_wf.</code>	Introduces universally quantified variables or hypotheses into the proof context.
2843	<code>rewrite /haetae_unpack_polyveck_body.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.

Line	EasyCrypt source	Mathematical reading
2844	<code>apply/(eq_from_nth poly_zero).</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
2845	<code>+ by rewrite size_mkseq; smt().</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2846	<code>move=> row row_rng.</code>	Introduces universally quantified variables or hypotheses into the proof context.
2847	<code>rewrite size_mkseq in row_rng.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2848	<code>rewrite nth_mkseq; first by smt().</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2849	<code>apply/(eq_from_nth 0).</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
2850	<code>+ by rewrite size_mkseq; smt(polyveck_wf_nth).</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2851	<code>move=> col col_rng.</code>	Introduces universally quantified variables or hypotheses into the proof context.
2852	<code>rewrite size_mkseq in col_rng.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2853	<code>rewrite nth_mkseq; first by smt().</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2854	<code>rewrite /haetae_pack_polyveck_body /poly_coeff.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2855	<code>by smt(nth_cat nth_mkseq haetae_body_index_div haetae_body_index_mod</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2856	<code>haetae_body_index_lt n_le_mode_polyq_packedbytes</code>	Continuation line in the current declaration or proof. Read it with
2857	<code>mode_polyq_packedbytes_gt0).</code>	surrounding lines as part of the same mathematical object.
2858	<code>qed.</code>	Continuation line in the current declaration or proof. Read it with
2859	<code>blank</code>	surrounding lines as part of the same mathematical object.
2860	<code>lemma haetae_unpack_polyveck_body_pack md sd b :</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
2861	<code>polyveck_wf md b =></code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2862	<code>haetae_unpack_polyveck_body md</code>	States checked lemma <code>haetae_unpack_polyveck_body_pack</code> . The following proof script must derive this proposition from prior definitions and lemmas.
2863	<code>(haetae_pack_seed sd ++ haetae_pack_polyveck_body md b)</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2864	<code>seedbytes = b.</code>	Continuation line in the current declaration or proof. Read it with
2865	<code>proof.</code>	surrounding lines as part of the same mathematical object.
2866	<code>move=> b_wf.</code>	Continuation line in the current declaration or proof. Read it with
2867	<code>have seed_sz := haetae_pack_seed_size sd.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2868	<code>rewrite -seed_sz.</code>	Begins the proof script for the preceding proposition.
2869	<code>by apply haetae_unpack_polyveck_body_pack_at.</code>	Introduces universally quantified variables or hypotheses into the proof context.
2870	<code>qed.</code>	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2871	<code>blank</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2872	<code>lemma haetae_public_key_roundtrip md pk :</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2873	<code>haetae_public_key_unpacked_wf md pk =></code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
2874	<code>haetae_public_key_roundtrip_ok md pk.</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2875	<code>proof.</code>	States checked lemma <code>haetae_public_key_roundtrip</code> . The following proof script must derive this proposition from prior definitions and lemmas.
2876	<code>case: pk => sd b /=.</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2877	<code>rewrite /haetae_public_key_unpacked_wf /haetae_public_key_roundtrip_ok</code>	Continuation line in the current declaration or proof. Read it with
2878	<code>/haetae_unpack_public_key_bytes /haetae_pack_public_key_bytes</code>	surrounding lines as part of the same mathematical object.
		Begins the proof script for the preceding proposition.
		Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
		Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
		Continuation line in the current declaration or proof. Read it with
		surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
2879	/haetae_unpack_vk_public_key /haetae_pack_vk /=.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2880	move=> [sd_sz b_wf].	Introduces universally quantified variables or hypotheses into the proof context.
2881	by rewrite (haetae_unpack_seed_pack_cat sd	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2882	(haetae_pack_polyveck_body md b) sd_sz)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2883	(haetae_unpack_polyveck_body_pack md sd b b_wf).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2884	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2885	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2886	lemma haetae_pack_public_key_bytes_size md pk :	States checked lemma haetae_pack_public_key_bytes_size. The following proof script must derive this proposition from prior definitions and lemmas.
2887	size (haetae_pack_public_key_bytes md pk) = haetae_public_key_bytes md.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2888	proof.	Begins the proof script for the preceding proposition.
2889	rewrite /haetae_pack_public_key_bytes /haetae_pack_vk	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2890	/haetae_public_key_bytes /mode_publickeybytes.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2891	by rewrite size_cat haetae_pack_seed_size haetae_pack_polyveck_body_size.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2892	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2893	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2894	lemma haetae_pack_public_key_bytes_wf md pk :	States checked lemma haetae_pack_public_key_bytes_wf. The following proof script must derive this proposition from prior definitions and lemmas.
2895	haetae_public_key_encoding_wf md (haetae_pack_public_key_bytes md pk).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2896	proof.	Begins the proof script for the preceding proposition.
2897	by rewrite /haetae_public_key_encoding_wf haetae_pack_public_key_bytes_size.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2898	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2899	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2900	lemma haetae_pack_public_key_bytes_ref_size md pk :	States checked lemma haetae_pack_public_key_bytes_ref_size. The following proof script must derive this proposition from prior definitions and lemmas.
2901	size (haetae_pack_public_key_bytes_ref md pk) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2902	haetae_public_key_bytes md.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2903	proof.	Begins the proof script for the preceding proposition.
2904	by rewrite /haetae_pack_public_key_bytes_ref haetae_pack_vk_ref_size.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2905	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2906	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2907	lemma haetae_pack_public_key_bytes_ref_wf md pk :	States checked lemma haetae_pack_public_key_bytes_ref_wf. The following proof script must derive this proposition from prior definitions and lemmas.
2908	haetae_public_key_encoding_wf md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2909	(haetae_pack_public_key_bytes_ref md pk).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2910	proof.	Begins the proof script for the preceding proposition.
2911	by rewrite /haetae_public_key_encoding_wf	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2912	haetae_pack_public_key_bytes_ref_size.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2913	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2914	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2915	lemma haetae_unpack_public_key_bytes_wf md enc :	States checked lemma haetae_unpack_public_key_bytes_wf. The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
2916	haetae_public_key_unpacked_wf md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2917	(haetae_unpack_public_key_bytes md enc).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2918	proof.	Begins the proof script for the preceding proposition.
2919	rewrite /haetae_public_key_unpacked_wf /haetae_unpack_public_key_bytes	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2920	/haetae_unpack_vk_public_key /=.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2921	have seed_sz := haetae_unpack_seed_size enc.	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2922	have body_wf := haetae_unpack_polyveck_body_wf md enc seedbytes.	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2923	by rewrite seed_sz body_wf.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2924	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2925	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2926	lemma haetae_unpack_public_key_bytes_ref_wf md enc :	States checked lemma haetae_unpack_public_key_bytes_ref_wf. The following proof script must derive this proposition from prior definitions and lemmas.
2927	haetae_public_key_unpacked_wf md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2928	(haetae_unpack_public_key_bytes_ref md enc).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2929	proof.	Begins the proof script for the preceding proposition.
2930	by rewrite /haetae_unpack_public_key_bytes_ref;	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2931	apply haetae_unpack_vk_public_key_ref_wf.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
2932	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2933	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2934	lemma concrete_haetae_keygen_packed_public_key_wf md sd :	States checked lemma concrete_haetae_keygen_packed_public_key_wf. The following proof script must derive this proposition from prior definitions and lemmas.
2935	haetae_public_key_encoding_wf md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2936	(concrete_haetae_keygen_packed_public_key md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2937	proof.	Begins the proof script for the preceding proposition.
2938	by rewrite /concrete_haetae_keygen_packed_public_key	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2939	haetae_pack_public_key_bytes_wf.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2940	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2941	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2942	lemma concrete_haetae_keygen_unpacked_public_key_wf md sd :	States checked lemma concrete_haetae_keygen_unpacked_public_key_wf. The following proof script must derive this proposition from prior definitions and lemmas.
2943	haetae_public_key_unpacked_wf md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2944	(concrete_haetae_keygen_unpacked_public_key md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2945	proof.	Begins the proof script for the preceding proposition.
2946	by rewrite /concrete_haetae_keygen_unpacked_public_key;	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2947	apply haetae_unpack_public_key_bytes_wf.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
2948	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2949	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2950	lemma concrete_haetae_keygen_packed_public_key_ref_wf md sd :	States checked lemma concrete_haetae_keygen_packed_public_key_ref_wf. The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
2951	haetae_public_key_encoding_wf md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2952	(concrete_haetae_keygen_packed_public_key_ref md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2953	proof.	Begins the proof script for the preceding proposition.
2954	by rewrite /concrete_haetae_keygen_packed_public_key_ref	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2955	haetae_pack_public_key_bytes_ref_wf.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2956	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2957	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2958	lemma concrete_haetae_keygen_unpacked_public_key_ref_wf md sd :	States checked lemma concrete_haetae_keygen_unpacked_public_key_ref_wf. The following proof script must derive this proposition from prior definitions and lemmas.
2959	haetae_public_key_unpacked_wf md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2960	(concrete_haetae_keygen_unpacked_public_key_ref md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2961	proof.	Begins the proof script for the preceding proposition.
2962	by rewrite /concrete_haetae_keygen_unpacked_public_key_ref;	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2963	apply haetae_unpack_public_key_bytes_ref_wf.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
2964	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2965	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2966	lemma packed_haetae_reference_public_key_wf md sd :	States checked lemma packed_haetae_reference_public_key_wf. The following proof script must derive this proposition from prior definitions and lemmas.
2967	polyveck_wf md (packed_haetae_reference_public_key md sd).‘2.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2968	proof.	Begins the proof script for the preceding proposition.
2969	have h := haetae_reference_keygen_public_vector_polyq_wf md sd.	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2970	by move: h; rewrite /haetae_polyveck_polyq_coeffs_wf	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2971	/packed_haetae_reference_public_key.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2972	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2973	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2974	lemma packed_haetae_reference_public_key_polyq_wf md sd :	States checked lemma packed_haetae_reference_public_key_polyq_wf. The following proof script must derive this proposition from prior definitions and lemmas.
2975	haetae_polyveck_polyq_coeffs_wf md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2976	(packed_haetae_reference_public_key md sd).‘2.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2977	proof.	Begins the proof script for the preceding proposition.
2978	by rewrite /packed_haetae_reference_public_key /:=;	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2979	apply haetae_reference_keygen_public_vector_polyq_wf.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
2980	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2981	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2982	lemma packed_haetae_reference_public_key_unpacked_wf md sd :	States checked lemma packed_haetae_reference_public_key_unpacked_wf. The following proof script must derive this proposition from prior definitions and lemmas.
2983	haetae_public_key_unpacked_wf md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2984	(packed_haetae_reference_public_key md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2985	proof.	Begins the proof script for the preceding proposition.
2986	rewrite /haetae_public_key_unpacked_wf	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.

Line	EasyCrypt source	Mathematical reading
2987	/packed_haetae_reference_public_key /=.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2988	split.	Splits a conjunction or structured goal into independent proof obligations.
2989	+ by apply haetae_keygen_rho_prime_size.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2990	have h := haetae_reference_keygen_public_vector_polyq_wf md sd.	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
2991	by move: h; rewrite /haetae_polyveck_polyq_coeffs_wf.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2992	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2993	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2994	lemma concrete_haetae_reference_keygen_packed_public_key_ref_wf md sd :	States checked lemma concrete_haetae_reference_keygen_packed_public_key_ref_wf. The following proof script must derive this proposition from prior definitions and lemmas.
2995	haetae_public_key_encoding_wf md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2996	(concrete_haetae_reference_keygen_packed_public_key_ref md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2997	proof.	Begins the proof script for the preceding proposition.
2998	by rewrite /concrete_haetae_reference_keygen_packed_public_key_ref	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2999	haetae_pack_public_key_bytes_ref_wf.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3000	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3001	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3002	lemma concrete_haetae_reference_keygen_unpacked_public_key_ref_wf md sd :	States checked lemma concrete_haetae_reference_keygen_unpacked_public_key_ref_wf. The following proof script must derive this proposition from prior definitions and lemmas.
3003	haetae_public_key_unpacked_wf md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3004	(concrete_haetae_reference_keygen_unpacked_public_key_ref md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3005	proof.	Begins the proof script for the preceding proposition.
3006	by rewrite /concrete_haetae_reference_keygen_unpacked_public_key_ref;	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3007	apply haetae_unpack_public_key_bytes_ref_wf.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
3008	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3009	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3010	lemma haetae_reference_keygen_public_key_ref_roundtrip md sd :	States checked lemma haetae_reference_keygen_public_key_ref_roundtrip. The following proof script must derive this proposition from prior definitions and lemmas.
3011	haetae_unpack_public_key_bytes_ref md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3012	(haetae_pack_public_key_bytes_ref md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3013	(packed_haetae_reference_public_key md sd)) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3014	packed_haetae_reference_public_key md sd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3015	proof.	Begins the proof script for the preceding proposition.
3016	apply haetae_public_key_ref_roundtrip.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
3017	+ by rewrite /packed_haetae_reference_public_key /=;	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3018	apply haetae_keygen_rho_prime_size.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
3019	by apply packed_haetae_reference_public_key_polyq_wf.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3020	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3021	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
3022	lemma public_key_of_secret_relation md sk :	States checked lemma public_key_of_secret_relation. The following proof script must derive this proposition from prior definitions and lemmas.
3023	public_key_relation md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3024	(public_key_of_secret md sk)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3025	(public_secret_vector_seed md sk.'1)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3026	(public_error_vector_seed md sk.'1).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3027	proof.	Begins the proof script for the preceding proposition.
3028	by rewrite /public_key_relation /public_key_of_secret	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3029	/public_key_vector_seed /haetae_keygen_public_vector	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3030	/haetae_keygen_secret_vector /haetae_keygen_error_vector.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3031	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3032	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3033	lemma public_key_of_secret_equation_holds md sk :	States checked lemma public_key_of_secret_equation_holds. The following proof script must derive this proposition from prior definitions and lemmas.
3034	public_key_equation_holds md (public_key_of_secret md sk).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3035	proof.	Begins the proof script for the preceding proposition.
3036	by rewrite /public_key_equation_holds public_key_of_secret_relation.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3037	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3038	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3039	op valid_mode : mode -> bool = fun _ => true.	Defines operation valid_mode, a total mathematical function or abbreviation used by later declarations.
3040	op valid_keypair (md : mode) (pk : pkey) (sk : skey) : bool =	Defines predicate valid_keypair, an event or well-formedness condition used in later implications and game proofs.
3041	pk = public_key_of_secret md sk.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3042	op valid_signature (md : mode) (sig : signature) : bool =	Defines predicate valid_signature, an event or well-formedness condition used in later implications and game proofs.
3043	polyveck_wf md sig.'1 /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3044	poly_wf sig.'2 /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3045	crh_wf sig.'3 /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3046	challenge_wf sig.'4 /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3047	polyvecl_wf md (sig.'5).'1 /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3048	polyveck_wf md (sig.'5).'2 /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3049	polyveck_wf md (sig.'5).'3.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3050	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3051	op sig_commitment_highbits (sig : signature) : polyveck = sig.'1.	Defines operation sig_commitment_highbits, a total mathematical function or abbreviation used by later declarations.
3052	op sig_commitment_lowbits (sig : signature) : poly = sig.'2.	Defines operation sig_commitment_lowbits, a total mathematical function or abbreviation used by later declarations.
3053	op sig_message_hash (sig : signature) : crh = sig.'3.	Defines operation sig_message_hash, a total mathematical function or abbreviation used by later declarations.
3054	op sig_challenge (sig : signature) : challenge = sig.'4.	Defines operation sig_challenge, a total mathematical function or abbreviation used by later declarations.
3055	op sig_token_value (sig : signature) : sig_token = sig.'5.	Defines operation sig_token_value, a total mathematical function or abbreviation used by later declarations.
3056	op sig_response (sig : signature) : polyvecl = (sig_token_value sig).'1.	Defines operation sig_response, a total mathematical function or abbreviation used by later declarations.

Line	EasyCrypt source	Mathematical reading
3057	<code>op sig_response_aux (sig : signature) : polyveck = (sig_token_value sig).‘2.</code>	Defines operation <code>sig_response_aux</code> , a total mathematical function or abbreviation used by later declarations.
3058	<code>op sig_hint (sig : signature) : hint_t = (sig_token_value sig).‘3.</code>	Defines operation <code>sig_hint</code> , a total mathematical function or abbreviation used by later declarations.
3059	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3060	<code>op encode_poly (p : poly) : byte list = p.</code>	Defines operation <code>encode_poly</code> , a total mathematical function or abbreviation used by later declarations.
3061	<code>op encode_polyveck (xs : polyveck) : byte list = flatten xs.</code>	Defines operation <code>encode_polyveck</code> , a total mathematical function or abbreviation used by later declarations.
3062	<code>op encode_pkey (pk : pkey) : byte list = pk.‘1 ++ encode_polyveck pk.‘2.</code>	Defines operation <code>encode_pkey</code> , a total mathematical function or abbreviation used by later declarations.
3063	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3064	<code>op deterministic_poly (tag : int) (src : byte list) : poly =</code>	Defines integer-valued parameter <code>deterministic_poly</code> , used for dimensions, budgets, support sizes, or bounds.
3065	<code>mkseq (fun i => coeff_mod (tag + i + nth 0 src i)) n.</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3066	<code>op deterministic_polyveck (md : mode) (src : byte list) : polyveck =</code>	Defines operation <code>deterministic_polyveck</code> , a total mathematical function or abbreviation used by later declarations.
3067	<code>mkseq</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3068	<code>(fun i => deterministic_poly i (i :: src))</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3069	<code>(mode_k md).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3070	<code>op deterministic_unit_coeff (tag : int) (src : byte list) (i : int) :</code>	Defines integer-valued parameter <code>deterministic_unit_coeff</code> , used for dimensions, budgets, support sizes, or bounds.
3071	<code>coeff =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3072	<code>if (tag + i + nth 0 src i) %% 2 = 0 then 1 else -1.</code>	Splits proof obligations according to a conditional branch in the program or formula.
3073	<code>op deterministic_unit_poly (tag : int) (src : byte list) : poly =</code>	Defines integer-valued parameter <code>deterministic_unit_poly</code> , used for dimensions, budgets, support sizes, or bounds.
3074	<code>mkseq (fun i => deterministic_unit_coeff tag src i) n.</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3075	<code>op deterministic_unit_polyveck</code>	Defines operation <code>deterministic_unit_polyveck</code> , a total mathematical function or abbreviation used by later declarations.
3076	<code>(md : mode) (tag : int) (src : byte list) : polyveck =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3077	<code>mkseq</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3078	<code>(fun i => deterministic_unit_poly (tag + i) (i :: src))</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3079	<code>(mode_k md).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3080	<code>op deterministic_unit_polyvecl</code>	Defines operation <code>deterministic_unit_polyvecl</code> , a total mathematical function or abbreviation used by later declarations.
3081	<code>(md : mode) (tag : int) (src : byte list) : polyvecl =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3082	<code>mkseq</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3083	<code>(fun i => deterministic_unit_poly (tag + i) (i :: src))</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3084	<code>(mode_l md).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3085	<code>op commitment_source</code>	Defines operation <code>commitment_source</code> , a total mathematical function or abbreviation used by later declarations.
3086	<code>(sk : skey) (m : message) (ctx : context)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3087	<code>(coins : random_coins) : byte list =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3088	<code>coins ++ sk.‘1 ++ ctx ++ m.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3089	<code>op commitment_raw :</code>	Defines operation <code>commitment_raw</code> , a total mathematical function or abbreviation used by later declarations.

Line	EasyCrypt source	Mathematical reading
3090	mode -> skey -> message -> context -> random_coins -> polyveck =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3091	fun md sk m ctx coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3092	deterministic_polyveck md (commitment_source sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3093	op response_source	Defines operation response_source, a total mathematical function or abbreviation used by later declarations.
3094	(sk : skey) (m : message) (ctx : context)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3095	(coins : random_coins) : byte list =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3096	coins ++ sk.'1 ++ ctx ++ m.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3097	op response_aux_vector :	Defines operation response_aux_vector, a total mathematical function or abbreviation used by later declarations.
3098	mode -> skey -> message -> context -> random_coins -> polyveck =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3099	fun md sk m ctx coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3100	deterministic_unit_polyveck md 29 (response_source sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3101	op signing_sample_y1 :	Defines operation signing_sample_y1, a total mathematical function or abbreviation used by later declarations.
3102	mode -> skey -> message -> context -> random_coins -> polyveckl =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3103	fun md sk m ctx coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3104	deterministic_unit_polyveckl md 73 (response_source sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3105	op signing_sample_y2 :	Defines operation signing_sample_y2, a total mathematical function or abbreviation used by later declarations.
3106	mode -> skey -> message -> context -> random_coins -> polyveck =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3107	fun md sk m ctx coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3108	deterministic_unit_polyveck md 79 (response_source sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3109	op signing_entropy_token_of_coins (coins : random_coins) : int =	Defines integer-valued parameter signing_entropy_token_of_coins, used for dimensions, budgets, support sizes, or bounds.
3110	nth 0 coins 0 +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3111	256 * nth 0 coins 1 +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3112	65536 * nth 0 coins 2 +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3113	16777216 * nth 0 coins 3 +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3114	4294967296 * nth 0 coins 4 +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3115	1099511627776 * nth 0 coins 5 +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3116	281474976710656 * nth 0 coins 6 +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3117	72057594037927936 * nth 0 coins 7.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3118	op commitment_lowbits :	Defines operation commitment_lowbits, a total mathematical function or abbreviation used by later declarations.
3119	mode -> skey -> message -> context -> random_coins -> poly =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3120	fun md sk m ctx coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3121	let raw = nth poly_zero (commitment_raw md sk m ctx coins) 0 in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3122	mkseq	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
3123	(fun i =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3124	if i = 0 then signing_entropy_token_of_coins coins	Splits proof obligations according to a conditional branch in the program or formula.
3125	else poly_coeff raw i)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3126	n.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3127	op message_hash : pkey -> context -> message -> crh =	Defines operation message_hash, a total mathematical function or abbreviation used by later declarations.
3128	fun pk ctx m => mkseq (fun i => nth 0 (encode_pkey pk ++ ctx ++ m) i) crhbytes.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3129	op challenge_source (highbits : polyveck) (lowbits : poly) (mu : crh) :	Defines operation challenge_source, a total mathematical function or abbreviation used by later declarations.
3130	byte list =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3131	encode_poly lowbits ++ encode_polyveck highbits ++ mu.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3132	op challenge_from_seed (md : mode) (src : byte list) : challenge =	Defines operation challenge_from_seed, a total mathematical function or abbreviation used by later declarations.
3133	mkseq	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3134	(fun i =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3135	if i < mode_tau md	Splits proof obligations according to a conditional branch in the program or formula.
3136	then if nth 0 src i %% 2 = 0 then 1 else -1	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3137	else 0) n.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3138	op challenge_hash : mode -> polyveck -> poly -> crh -> challenge =	Defines operation challenge_hash, a total mathematical function or abbreviation used by later declarations.
3139	fun md (_ : polyveck) lowbits mu =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3140	challenge_from_seed md (encode_poly lowbits ++ mu).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3141	op commitment_challenge :	Defines operation commitment_challenge, a total mathematical function or abbreviation used by later declarations.
3142	mode -> skey -> message -> context -> random_coins -> challenge =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3143	fun md sk m ctx coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3144	challenge_hash md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3145	(response_aux_vector md sk m ctx coins)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3146	(commitment_lowbits md sk m ctx coins)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3147	(message_hash (public_key_of_secret md sk) ctx m).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3148	op commitment_highbits :	Defines operation commitment_highbits, a total mathematical function or abbreviation used by later declarations.
3149	mode -> skey -> message -> context -> random_coins -> polyveck =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3150	fun md sk m ctx coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3151	let pk = public_key_of_secret md sk in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3152	let ch = commitment_challenge md sk m ctx coins in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3153	reconstructed_highbits md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3154	(response_aux_vector md sk m ctx coins)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3155	(public_key_challenge_term md pk ch).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
3156	<code>op response_vector :</code>	Defines operation <code>response_vector</code> , a total mathematical function or abbreviation used by later declarations.
3157	<code>mode -> skey -> message -> context -> random_coins -> polyvecl =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3158	<code>fun md sk m ctx coins =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3159	<code>deterministic_unit_polyvecl md 17 (response_source sk m ctx coins).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3160	<code>op hint_vector :</code>	Defines operation <code>hint_vector</code> , a total mathematical function or abbreviation used by later declarations.
3161	<code>mode -> skey -> message -> context -> random_coins -> hint_t =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3162	<code>fun md sk m ctx coins =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3163	<code>deterministic_unit_polyveck md 41 (response_source sk m ctx coins).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3164	<code>op signature_token :</code>	Defines operation <code>signature_token</code> , a total mathematical function or abbreviation used by later declarations.
3165	<code>mode -> skey -> message -> context -> random_coins -> sig_token =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3166	<code>fun md sk m ctx coins =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3167	<code>(response_vector md sk m ctx coins,</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3168	<code>response_aux_vector md sk m ctx coins,</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3169	<code>hint_vector md sk m ctx coins).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3170	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3171	<code>lemma deterministic_poly_wf tag src :</code>	States checked lemma <code>deterministic_poly_wf</code> . The following proof script must derive this proposition from prior definitions and lemmas.
3172	<code>poly_wf (deterministic_poly tag src).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3173	<code>proof. by rewrite /poly_wf /deterministic_poly size_mkseq /n. qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3174	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3175	<code>lemma deterministic_polyveck_wf md src :</code>	States checked lemma <code>deterministic_polyveck_wf</code> . The following proof script must derive this proposition from prior definitions and lemmas.
3176	<code>polyveck_wf md (deterministic_polyveck md src).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3177	<code>proof.</code>	Begins the proof script for the preceding proposition.
3178	<code>rewrite /polyveck_wf /deterministic_polyveck size_mkseq.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3179	<code>split.</code>	Splits a conjunction or structured goal into independent proof obligations.
3180	<code>+ by case md.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3181	<code>apply/allP=> p /mkseqP [i [_ ->]].</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
3182	<code>by apply deterministic_poly_wf.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3183	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
3184	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3185	<code>lemma deterministic_unit_poly_wf tag src :</code>	States checked lemma <code>deterministic_unit_poly_wf</code> . The following proof script must derive this proposition from prior definitions and lemmas.
3186	<code>poly_wf (deterministic_unit_poly tag src).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3187	<code>proof. by rewrite /poly_wf /deterministic_unit_poly size_mkseq /n. qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3188	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3189	<code>lemma deterministic_unit_poly_unit tag src :</code>	States checked lemma <code>deterministic_unit_poly_unit</code> . The following proof script must derive this proposition from prior definitions and lemmas.
3190	<code>poly_unit (deterministic_unit_poly tag src).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
3191	<code>proof.</code>	Begins the proof script for the preceding proposition.
3192	<code>rewrite /poly_unit.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3193	<code>split.</code>	Splits a conjunction or structured goal into independent proof obligations.
3194	<code>+ by apply deterministic_unit_poly_wf.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3195	<code>apply/allP=> x /mkseqP [i [_ ->]].</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
3196	<code>rewrite /deterministic_unit_poly /deterministic_unit_coeff /unit_coeff_ok.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3197	<code>by case: ((tag + i + nth 0 src i) %% 2 = 0).</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3198	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
3199	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3200	<code>lemma deterministic_unit_polyveck_wf md tag src :</code>	States checked lemma <code>deterministic_unit_polyveck_wf</code> . The following proof script must derive this proposition from prior definitions and lemmas.
3201	<code>polyveck_wf md (deterministic_unit_polyveck md tag src).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3202	<code>proof.</code>	Begins the proof script for the preceding proposition.
3203	<code>rewrite /polyveck_wf /deterministic_unit_polyveck size_mkseq.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3204	<code>split.</code>	Splits a conjunction or structured goal into independent proof obligations.
3205	<code>+ by case md.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3206	<code>apply/allP=> p /mkseqP [i [_ ->]].</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
3207	<code>by apply deterministic_unit_poly_wf.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3208	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
3209	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3210	<code>lemma deterministic_unit_polyveck_unit md tag src :</code>	States checked lemma <code>deterministic_unit_polyveck_unit</code> . The following proof script must derive this proposition from prior definitions and lemmas.
3211	<code>polyveck_unit md (deterministic_unit_polyveck md tag src).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3212	<code>proof.</code>	Begins the proof script for the preceding proposition.
3213	<code>rewrite /polyveck_unit.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3214	<code>split.</code>	Splits a conjunction or structured goal into independent proof obligations.
3215	<code>+ by apply deterministic_unit_polyveck_wf.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3216	<code>apply/allP=> p /mkseqP [i [_ ->]].</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
3217	<code>by apply deterministic_unit_poly_unit.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3218	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
3219	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3220	<code>lemma deterministic_unit_polyvecl_wf md tag src :</code>	States checked lemma <code>deterministic_unit_polyvecl_wf</code> . The following proof script must derive this proposition from prior definitions and lemmas.
3221	<code>polyvecl_wf md (deterministic_unit_polyvecl md tag src).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3222	<code>proof.</code>	Begins the proof script for the preceding proposition.
3223	<code>rewrite /polyvecl_wf /deterministic_unit_polyvecl size_mkseq.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3224	<code>split.</code>	Splits a conjunction or structured goal into independent proof obligations.
3225	<code>+ by case md.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3226	<code>apply/allP=> p /mkseqP [i [_ ->]].</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
3227	<code>by apply deterministic_unit_poly_wf.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3228	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
3229	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
3230	lemma deterministic_unit_polyvecl_unit md tag src :	States checked lemma deterministic_unit_polyvecl_unit. The following proof script must derive this proposition from prior definitions and lemmas.
3231	polyvecl_unit md (deterministic_unit_polyvecl md tag src).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3232	proof.	Begins the proof script for the preceding proposition.
3233	rewrite /polyvecl_unit.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3234	split.	Splits a conjunction or structured goal into independent proof obligations.
3235	+ by apply deterministic_unit_polyvecl_wf.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3236	apply/allP=> p /mkseqP [i [_ ->]].	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
3237	by apply deterministic_unit_poly_unit.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3238	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3239	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3240	lemma commitment_raw_wf md sk m ctx coins :	States checked lemma commitment_raw_wf. The following proof script must derive this proposition from prior definitions and lemmas.
3241	polyveck_wf md (commitment_raw md sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3242	proof.	Begins the proof script for the preceding proposition.
3243	by rewrite /commitment_raw; apply deterministic_polyveck_wf.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3244	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3245	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3246	lemma commitment_highbits_wf md sk m ctx coins :	States checked lemma commitment_highbits_wf. The following proof script must derive this proposition from prior definitions and lemmas.
3247	polyveck_wf md (commitment_highbits md sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3248	proof.	Begins the proof script for the preceding proposition.
3249	by rewrite /commitment_highbits /reconstructed_highbits /;	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3250	apply polyveck_add_wf.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
3251	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3252	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3253	lemma commitment_lowbits_wf md sk m ctx coins :	States checked lemma commitment_lowbits_wf. The following proof script must derive this proposition from prior definitions and lemmas.
3254	poly_wf (commitment_lowbits md sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3255	proof.	Begins the proof script for the preceding proposition.
3256	rewrite /commitment_lowbits /poly_wf.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3257	by smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3258	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3259	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3260	lemma commitment_lowbits_entropy_token md sk m ctx coins :	States checked lemma commitment_lowbits_entropy_token. The following proof script must derive this proposition from prior definitions and lemmas.
3261	nth 0 (commitment_lowbits md sk m ctx coins) 0 =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3262	signing_entropy_token_of_coins coins.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3263	proof.	Begins the proof script for the preceding proposition.
3264	rewrite /commitment_lowbits /= nth_mkseq; first by smt(n_gt0).	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3265	case: (0 = 0) => z.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
3266	+ by smt.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3267	by smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.

Line	EasyCrypt source	Mathematical reading
3268	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3269	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3270	lemma challenge_from_seed_wf md src :	States checked lemma challenge_from_seed_wf. The following proof script must derive this proposition from prior definitions and lemmas.
3271	challenge_wf (challenge_from_seed md src).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3272	proof.	Begins the proof script for the preceding proposition.
3273	rewrite /challenge_wf.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3274	split.	Splits a conjunction or structured goal into independent proof obligations.
3275	+ by rewrite /poly_wf /challenge_from_seed size_mkseq /n.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3276	apply/allP=> x /mkseqP [i [_ ->]].	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
3277	rewrite /challenge_coeff_ok /challenge_from_seed.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3278	case: (i < mode_tau md) => /=.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
3279	by case: (nth 0 src i %% 2 = 0).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3280	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3281	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3282	lemma challenge_from_seed_sparse md src :	States checked lemma challenge_from_seed_sparse. The following proof script must derive this proposition from prior definitions and lemmas.
3283	challenge_sparse md (challenge_from_seed md src).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3284	proof.	Begins the proof script for the preceding proposition.
3285	rewrite /challenge_sparse.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3286	split.	Splits a conjunction or structured goal into independent proof obligations.
3287	+ by rewrite /challenge_from_seed size_mkseq /n.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3288	move=> i [ge0_i lt_i].	Introduces universally quantified variables or hypotheses into the proof context.
3289	rewrite /challenge_sparse_at /challenge_from_seed nth_mkseq.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3290	+ by split.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3291	case: (i < mode_tau md) => h_tau /=.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
3292	+ rewrite h_tau /unit_coeff_ok.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3293	by case: (nth 0 src i %% 2 = 0).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3294	by rewrite h_tau.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3295	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3296	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3297	lemma challenge_hash_wf md w1 w0 mu :	States checked lemma challenge_hash_wf. The following proof script must derive this proposition from prior definitions and lemmas.
3298	challenge_wf (challenge_hash md w1 w0 mu).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3299	proof. by rewrite /challenge_hash; apply challenge_from_seed_wf. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3300	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3301	lemma challenge_hash_sparse md w1 w0 mu :	States checked lemma challenge_hash_sparse. The following proof script must derive this proposition from prior definitions and lemmas.
3302	challenge_sparse md (challenge_hash md w1 w0 mu).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3303	proof. by rewrite /challenge_hash; apply challenge_from_seed_sparse. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3304	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
3305	lemma response_vector_wf md sk m ctx coins :	States checked lemma response_vector_wf. The following proof script must derive this proposition from prior definitions and lemmas.
3306	polyvecl_wf md (response_vector md sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3307	proof.	Begins the proof script for the preceding proposition.
3308	by rewrite /response_vector; apply deterministic_unit_polyvecl_wf.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3309	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3310	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3311	lemma response_vector_unit md sk m ctx coins :	States checked lemma response_vector_unit. The following proof script must derive this proposition from prior definitions and lemmas.
3312	polyvecl_unit md (response_vector md sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3313	proof.	Begins the proof script for the preceding proposition.
3314	by rewrite /response_vector; apply deterministic_unit_polyvecl_unit.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3315	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3316	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3317	lemma response_aux_vector_wf md sk m ctx coins :	States checked lemma response_aux_vector_wf. The following proof script must derive this proposition from prior definitions and lemmas.
3318	polyveck_wf md (response_aux_vector md sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3319	proof.	Begins the proof script for the preceding proposition.
3320	by rewrite /response_aux_vector; apply deterministic_unit_polyveck_wf.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3321	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3322	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3323	lemma response_aux_vector_unit md sk m ctx coins :	States checked lemma response_aux_vector_unit. The following proof script must derive this proposition from prior definitions and lemmas.
3324	polyveck_unit md (response_aux_vector md sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3325	proof.	Begins the proof script for the preceding proposition.
3326	by rewrite /response_aux_vector; apply deterministic_unit_polyveck_unit.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3327	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3328	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3329	lemma signing_sample_y1_wf md sk m ctx coins :	States checked lemma signing_sample_y1_wf. The following proof script must derive this proposition from prior definitions and lemmas.
3330	polyvecl_wf md (signing_sample_y1 md sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3331	proof.	Begins the proof script for the preceding proposition.
3332	by rewrite /signing_sample_y1; apply deterministic_unit_polyvecl_wf.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3333	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3334	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3335	lemma signing_sample_y1_unit md sk m ctx coins :	States checked lemma signing_sample_y1_unit. The following proof script must derive this proposition from prior definitions and lemmas.
3336	polyvecl_unit md (signing_sample_y1 md sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3337	proof.	Begins the proof script for the preceding proposition.
3338	by rewrite /signing_sample_y1; apply deterministic_unit_polyvecl_unit.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3339	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3340	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3341	lemma signing_sample_y2_wf md sk m ctx coins :	States checked lemma signing_sample_y2_wf. The following proof script must derive this proposition from prior definitions and lemmas.
3342	polyveck_wf md (signing_sample_y2 md sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3343	proof.	Begins the proof script for the preceding proposition.
3344	by rewrite /signing_sample_y2; apply deterministic_unit_polyveck_wf.	Discharges the current goal in one step using the cited simplification, lemma, or automation.

Line	EasyCrypt source	Mathematical reading
3345	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3346	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3347	lemma signing_sample_y2_unit md sk m ctx coins :	States checked lemma signing_sample_y2_unit. The following proof script must derive this proposition from prior definitions and lemmas.
3348	polyveck_unit md (signing_sample_y2 md sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3349	proof.	Begins the proof script for the preceding proposition.
3350	by rewrite /signing_sample_y2; apply deterministic_unit_polyveck_unit.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3351	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3352	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3353	lemma hint_vector_wf md sk m ctx coins :	States checked lemma hint_vector_wf. The following proof script must derive this proposition from prior definitions and lemmas.
3354	polyveck_wf md (hint_vector md sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3355	proof.	Begins the proof script for the preceding proposition.
3356	by rewrite /hint_vector; apply deterministic_unit_polyveck_wf.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3357	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3358	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3359	lemma hint_vector_unit md sk m ctx coins :	States checked lemma hint_vector_unit. The following proof script must derive this proposition from prior definitions and lemmas.
3360	polyveck_unit md (hint_vector md sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3361	proof.	Begins the proof script for the preceding proposition.
3362	by rewrite /hint_vector; apply deterministic_unit_polyveck_unit.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3363	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3364	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3365	lemma message_hash_size pk ctx m :	States checked lemma message_hash_size. The following proof script must derive this proposition from prior definitions and lemmas.
3366	size (message_hash pk ctx m) = crhbytes.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3367	proof. by rewrite /message_hash size_mkseq /crhbytes. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3368	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3369	lemma challenge_hash_size md w1 w0 mu :	States checked lemma challenge_hash_size. The following proof script must derive this proposition from prior definitions and lemmas.
3370	size (challenge_hash md w1 w0 mu) = n.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3371	proof. by rewrite /challenge_hash /challenge_from_seed size_mkseq /n. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3372	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3373	op keygen_internal (md : mode) (sd : seed) : pkey * skey =	Defines operation keygen_internal, a total mathematical function or abbreviation used by later declarations.
3374	let sk = secret_key_of_seed md sd in (public_key_of_secret md sk, sk).	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3375	op sign_internal :	Defines operation sign_internal, a total mathematical function or abbreviation used by later declarations.
3376	mode -> skey -> message -> context -> random_coins -> signature =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3377	fun (md : mode) (sk : skey) (m : message) (ctx : context)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3378	(coins : random_coins) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3379	let pk = public_key_of_secret md sk in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3380	let highbits = commitment_highbits md sk m ctx coins in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3381	let lowbits = commitment_lowbits md sk m ctx coins in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.

Line	EasyCrypt source	Mathematical reading
3382	let mu = message_hash pk ctx m in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3383	let ch = commitment_challenge md sk m ctx coins in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3384	(highbits, lowbits, mu, ch, signature_token md sk m ctx coins, 0, 0).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3385	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3386	op verify_challenge_consistent	Defines operation <code>verify_challenge_consistent</code> , a total mathematical function or abbreviation used by later declarations.
3387	(md : mode) (pk : pkey) (m : message) (ctx : context)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3388	(sig : signature) : bool =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3389	let mu = message_hash pk ctx m in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3390	sig_message_hash sig = mu /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3391	sig_challenge sig =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3392	challenge_hash md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3393	(sig_commitment_highbits sig)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3394	(sig_commitment_lowbits sig)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3395	mu.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3396	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3397	op verify_reconstructed_highbits	Defines operation <code>verify_reconstructed_highbits</code> , a total mathematical function or abbreviation used by later declarations.
3398	(md : mode) (pk : pkey) (sig : signature) : polyveck =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3399	let pk_ch = public_key_challenge_term md pk (sig_challenge sig) in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3400	reconstructed_highbits md (sig_response_aux sig) pk_ch.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3401	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3402	op verify_public_equation	Defines operation <code>verify_public_equation</code> , a total mathematical function or abbreviation used by later declarations.
3403	(md : mode) (pk : pkey) (sig : signature) : bool =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3404	sig_commitment_highbits sig =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3405	verify_reconstructed_highbits md pk sig.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3406	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3407	lemma keygen_internal_valid md sd :	States checked lemma <code>keygen_internal_valid</code> . The following proof script must derive this proposition from prior definitions and lemmas.
3408	valid_keypair md (keygen_internal md sd).‘1 (keygen_internal md sd).‘2.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3409	proof. by rewrite /valid_keypair /keygen_internal. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3410	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3411	lemma public_key_of_keygen md sd :	States checked lemma <code>public_key_of_keygen</code> . The following proof script must derive this proposition from prior definitions and lemmas.
3412	public_key_of_secret md (keygen_internal md sd).‘2 =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3413	(keygen_internal md sd).‘1.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3414	proof. by rewrite /keygen_internal. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
3415	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3416	lemma public_key_vector_seed_keygen_equation md sd :	States checked lemma public_key_vector_seed_keygen_equation. The following proof script must derive this proposition from prior definitions and lemmas.
3417	public_key_vector_seed md sd =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3418	haetae_public_key_vector_from_relation md sd	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3419	(haetae_keygen_secret_vector md sd)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3420	(haetae_keygen_error_vector md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3421	proof. by rewrite /public_key_vector_seed /haetae_keygen_public_vector. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3422	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3423	lemma public_key_vector_seed_mode23_keygen_equation md sd :	States checked lemma public_key_vector_seed_mode23_keygen_equation. The following proof script must derive this proposition from prior definitions and lemmas.
3424	(md = Mode2 \ / md = Mode3) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3425	public_key_vector_seed md sd =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3426	polyveck_vk_highbits md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3427	(polyveck_add md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3428	(public_rounding_vector_seed md sd)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3429	(polyveck_add md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3430	(matrix_vec_mul md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3431	(haetae_keygen_a0_matrix md sd)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3432	(haetae_keygen_secret_vector md sd))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3433	(haetae_keygen_error_vector md sd))).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3434	proof.	Begins the proof script for the preceding proposition.
3435	move=> md23.	Introduces universally quantified variables or hypotheses into the proof context.
3436	rewrite /public_key_vector_seed /haetae_keygen_public_vector	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3437	/haetae_public_key_vector_from_relation	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3438	/haetae_keygen_a0_matrix /haetae_keygen_secret_vector	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3439	/haetae_keygen_error_vector.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3440	by case: md md23 => //.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3441	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3442	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3443	lemma public_key_vector_seed_mode5_keygen_equation sd :	States checked lemma public_key_vector_seed_mode5_keygen_equation. The following proof script must derive this proposition from prior definitions and lemmas.
3444	public_key_vector_seed Mode5 sd =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3445	polyveck_neg Mode5	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3446	(polyveck_double Mode5	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3447	(polyveck_add Mode5	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
3448	(matrix_vec_mul Mode5	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3449	(haetae_keygen_a0_matrix Mode5 sd)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3450	(haetae_keygen_secret_vector Mode5 sd))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3451	(haetae_keygen_error_vector Mode5 sd))).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3452	proof.	Begins the proof script for the preceding proposition.
3453	by rewrite /public_key_vector_seed /haetae_keygen_public_vector	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3454	/haetae_public_key_vector_from_relation	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3455	/haetae_keygen_a0_matrix /haetae_keygen_secret_vector	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3456	/haetae_keygen_error_vector.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3457	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3458	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3459	lemma haetae_keygen_raw_public_vector_keygen_equation md sd :	States checked lemma haetae_keygen_raw_public_vector_keygen_equation. The following proof script must derive this proposition from prior definitions and lemmas.
3460	haetae_keygen_raw_public_vector md sd =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3461	polyveck_add md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3462	(matrix_vec_mul md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3463	(haetae_keygen_a0_matrix md sd)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3464	(haetae_keygen_secret_vector md sd))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3465	(haetae_keygen_error_vector md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3466	proof. by rewrite /haetae_keygen_raw_public_vector. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3467	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3468	lemma haetae_reference_keygen_raw_public_vector_equation md sd :	States checked lemma haetae_reference_keygen_raw_public_vector_equation. The following proof script must derive this proposition from prior definitions and lemmas.
3469	haetae_reference_keygen_raw_public_vector md sd =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3470	polyveck_add md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3471	(matrix_vec_mul md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3472	(haetae_reference_polymatkm_expand_matA md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3473	(haetae_keygen_rho_prime sd))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3474	(haetae_reference_keygen_secret_vector md sd))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3475	(haetae_reference_keygen_error_vector md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3476	proof.	Begins the proof script for the preceding proposition.
3477	by rewrite /haetae_reference_keygen_raw_public_vector	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3478	/haetae_reference_keygen_a0_matrix.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3479	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3480	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3481	lemma haetae_reference_keygen_mode23_public_vector_equation md sd :	States checked lemma haetae_reference_keygen_mode23_public_vector_equation. The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
3482	(md = Mode2 \ / md = Mode3) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3483	haetae_reference_keygen_public_vector md sd =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3484	polyveck_vk_highbits md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3485	(polyveck_add md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3486	(haetae_reference_polyveck_expand_vecA md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3487	(haetae_keygen_rhoprime sd))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3488	(polyveck_add md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3489	(matrix_vec_mul md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3490	(haetae_reference_polymatkm_expand_matA md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3491	(haetae_keygen_rhoprime sd))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3492	(haetae_reference_keygen_secret_vector md sd))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3493	(haetae_reference_keygen_error_vector md sd))).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3494	proof.	Begins the proof script for the preceding proposition.
3495	move=> md23.	Introduces universally quantified variables or hypotheses into the proof context.
3496	rewrite /haetae_reference_keygen_public_vector	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3497	/haetae_public_key_vector_from_relation	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3498	/haetae_reference_polyveck_expand_vecA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3499	/haetae_reference_polymatkm_expand_matA.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3500	by case: md md23 => //=.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3501	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3502	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3503	lemma haetae_reference_keygen_mode23_adjusted_error_equation md sd :	States checked lemma
3504	(md = Mode2 \ / md = Mode3) =>	haetae_reference_keygen_mode23_adjusted_error_equation. The following proof script must derive this proposition from prior definitions and lemmas.
3505	haetae_reference_keygen_mode23_adjusted_error_vector md sd =	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3506	polyveck_sub md	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3507	(haetae_reference_keygen_error_vector md sd)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3508	(polyveck_vk_lowbits md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3509	(polyveck_add md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3510	(haetae_reference_polyveck_expand_vecA md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3511	(haetae_keygen_rhoprime sd))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3512	(haetae_reference_keygen_raw_public_vector md sd))).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3513	proof.	Begins the proof script for the preceding proposition.
3514	move=> _.	Introduces universally quantified variables or hypotheses into the proof context.
3515	by rewrite /haetae_reference_keygen_mode23_adjusted_error_vector	Discharges the current goal in one step using the cited simplification, lemma, or automation.

Line	EasyCrypt source	Mathematical reading
3516	<code>/haetae_reference_keygen_mode23_rounding_lowbits</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3517	<code>/haetae_reference_keygen_mode23_predecompose_vector.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3518	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
3519	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3520	<code>lemma haetae_reference_keygen_mode5_public_vector_equation sd :</code>	States checked lemma <code>haetae_reference_keygen_mode5_public_vector_equation</code> . The following proof script must derive this proposition from prior definitions and lemmas.
3521	<code>haetae_reference_keygen_public_vector Mode5 sd =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3522	<code>polyveck_neg Mode5</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3523	<code>(polyveck_double Mode5</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3524	<code>(polyveck_add Mode5</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3525	<code>(matrix_vec_mul Mode5</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3526	<code>(haetae_reference_polymatkm_expand_matA Mode5</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3527	<code>(haetae_keygen_rho_prime sd))</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3528	<code>(haetae_reference_keygen_secret_vector Mode5 sd))</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3529	<code>(haetae_reference_keygen_error_vector Mode5 sd)))</code> .	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3530	<code>proof.</code>	Begins the proof script for the preceding proposition.
3531	<code>by rewrite /haetae_reference_keygen_public_vector</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3532	<code>/haetae_public_key_vector_from_relation</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3533	<code>/haetae_reference_polymatkm_expand_matA.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3534	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
3535	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3536	<code>lemma packed_haetae_reference_public_key_seedE md sd :</code>	States checked lemma <code>packed_haetae_reference_public_key_seedE</code> . The following proof script must derive this proposition from prior definitions and lemmas.
3537	<code>(packed_haetae_reference_public_key md sd). '1 =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3538	<code>haetae_keygen_rho_prime sd.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3539	<code>proof. by rewrite /packed_haetae_reference_public_key. qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3540	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3541	<code>lemma packed_haetae_reference_public_key_bodyE md sd :</code>	States checked lemma <code>packed_haetae_reference_public_key_bodyE</code> . The following proof script must derive this proposition from prior definitions and lemmas.
3542	<code>(packed_haetae_reference_public_key md sd). '2 =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3543	<code>haetae_reference_keygen_public_vector md sd.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3544	<code>proof. by rewrite /packed_haetae_reference_public_key. qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3545	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3546	<code>lemma haetae_reference_keygen_seed_flow_wf_ok md sd :</code>	States checked lemma <code>haetae_reference_keygen_seed_flow_wf_ok</code> . The following proof script must derive this proposition from prior definitions and lemmas.
3547	<code>haetae_reference_keygen_seed_flow_wf md sd.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3548	<code>proof.</code>	Begins the proof script for the preceding proposition.

Line	EasyCrypt source	Mathematical reading
3549	by rewrite /haetae_reference_keygen_seed_flow_wf	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3550	/haetae_reference_keygen_a0_matrix	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3551	/haetae_reference_keygen_secret_vector	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3552	/haetae_reference_keygen_error_vector	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3553	/haetae_reference_keygen_public_vector	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3554	/packed_haetae_reference_public_key.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3555	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3556	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3557	lemma packed_haetae_public_key_wf md sd :	States checked lemma packed_haetae_public_key_wf. The following proof script must derive this proposition from prior definitions and lemmas.
3558	polyveck_wf md (packed_haetae_public_key md sd).‘2.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3559	proof.	Begins the proof script for the preceding proposition.
3560	have h := haetae_keygen_public_vector_polyq_wf md sd.	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
3561	by move: h; rewrite /haetae_polyveck_polyq_coeffs_wf /packed_haetae_public_key.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3562	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3563	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3564	lemma packed_haetae_public_key_polyq_wf md sd :	States checked lemma packed_haetae_public_key_polyq_wf. The following proof script must derive this proposition from prior definitions and lemmas.
3565	haetae_polyveck_polyq_coeffs_wf md (packed_haetae_public_key md sd).‘2.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3566	proof.	Begins the proof script for the preceding proposition.
3567	by rewrite /packed_haetae_public_key /:=;	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3568	apply haetae_keygen_public_vector_polyq_wf.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
3569	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3570	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3571	lemma packed_haetae_public_key_unpacked_wf md sd :	States checked lemma packed_haetae_public_key_unpacked_wf. The following proof script must derive this proposition from prior definitions and lemmas.
3572	size sd = seedbytes =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3573	haetae_public_key_unpacked_wf md (packed_haetae_public_key md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3574	proof.	Begins the proof script for the preceding proposition.
3575	move=> sd_sz.	Introduces universally quantified variables or hypotheses into the proof context.
3576	by rewrite /haetae_public_key_unpacked_wf /= sd_sz packed_haetae_public_key_wf.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3577	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3578	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3579	lemma haetae_keygen_public_key_roundtrip md sd :	States checked lemma haetae_keygen_public_key_roundtrip. The following proof script must derive this proposition from prior definitions and lemmas.
3580	size sd = seedbytes =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3581	haetae_keygen_public_key_roundtrip_ok md sd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3582	proof.	Begins the proof script for the preceding proposition.
3583	move=> sd_sz.	Introduces universally quantified variables or hypotheses into the proof context.
3584	rewrite /haetae_keygen_public_key_roundtrip_ok.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3585	apply haetae_public_key_roundtrip.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.

Line	EasyCrypt source	Mathematical reading
3586	by apply packed_haetae_public_key_unpacked_wf.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3587	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3588	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3589	lemma public_key_of_secret_packed md sk :	States checked lemma public_key_of_secret_packed. The following proof script must derive this proposition from prior definitions and lemmas.
3590	public_key_of_secret md sk = packed_haetae_public_key md sk.'1.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3591	proof.	Begins the proof script for the preceding proposition.
3592	by rewrite /public_key_of_secret /packed_haetae_public_key	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3593	/public_key_vector_seed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3594	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3595	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3596	lemma keygen_public_key_packed md sd :	States checked lemma keygen_public_key_packed. The following proof script must derive this proposition from prior definitions and lemmas.
3597	(keygen_internal md sd).'1 = packed_haetae_public_key md sd.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3598	proof.	Begins the proof script for the preceding proposition.
3599	by rewrite /keygen_internal /public_key_of_secret /secret_key_of_seed	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3600	/packed_haetae_public_key /public_key_vector_seed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3601	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3602	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3603	lemma public_verification_matrix_packedE md pk :	States checked lemma public_verification_matrix_packedE. The following proof script must derive this proposition from prior definitions and lemmas.
3604	public_verification_matrix md pk = packed_haetae_verification_matrix md pk.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3605	proof. by rewrite /public_verification_matrix. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3606	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3607	lemma packed_haetae_keygen_verification_matrixE md sd :	States checked lemma packed_haetae_keygen_verification_matrixE. The following proof script must derive this proposition from prior definitions and lemmas.
3608	packed_haetae_keygen_verification_matrix md sd =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3609	haetae_verification_matrix_from_unpacked md sd	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3610	(haetae_keygen_public_vector md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3611	proof.	Begins the proof script for the preceding proposition.
3612	by rewrite /packed_haetae_keygen_verification_matrix	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3613	/packed_haetae_verification_matrix	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3614	/packed_haetae_public_key.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3615	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3616	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3617	lemma honest_public_verification_matrix_eq_packed_keygen md sd :	States checked lemma honest_public_verification_matrix_eq_packed_keygen. The following proof script must derive this proposition from prior definitions and lemmas.
3618	public_verification_matrix md (keygen_internal md sd).'1 =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3619	packed_haetae_keygen_verification_matrix md sd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3620	proof.	Begins the proof script for the preceding proposition.
3621	by rewrite /keygen_internal /public_key_of_secret /secret_key_of_seed	Discharges the current goal in one step using the cited simplification, lemma, or automation.

Line	EasyCrypt source	Mathematical reading
3622	<code>/public_verification_matrix /packed_haetae_keygen_verification_matrix</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3623	<code>/packed_haetae_verification_matrix /packed_haetae_public_key</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3624	<code>/public_key_vector_seed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3625	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
3626	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3627	<code>lemma concrete_keygen_public_verification_matrix_correct md sd :</code>	States checked lemma <code>concrete_keygen_public_verification_matrix_correct</code> . The following proof script must derive this proposition from prior definitions and lemmas.
3628	<code>size sd = seedbytes =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3629	<code>public_verification_matrix md (keygen_internal md sd). '1 =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3630	<code>concrete_haetae_keygen_verification_matrix md sd.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3631	<code>proof.</code>	Begins the proof script for the preceding proposition.
3632	<code>move=> sd_sz.</code>	Introduces universally quantified variables or hypotheses into the proof context.
3633	<code>have rt := haetae_keygen_public_key_roundtrip md sd sd_sz.</code>	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
3634	<code>rewrite honest_public_verification_matrix_eq_packed_keygen.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3635	<code>rewrite /packed_haetae_keygen_verification_matrix</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3636	<code>/concrete_haetae_keygen_verification_matrix</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3637	<code>/concrete_haetae_keygen_unpacked_public_key</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3638	<code>/concrete_haetae_keygen_packed_public_key.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3639	<code>rewrite /haetae_keygen_public_key_roundtrip_ok</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3640	<code>/haetae_public_key_roundtrip_ok in rt.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3641	<code>by rewrite rt.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3642	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
3643	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3644	<code>lemma concrete_keygen_public_verification_matrix_ref_correct md sd :</code>	States checked lemma <code>concrete_keygen_public_verification_matrix_ref_correct</code> . The following proof script must derive this proposition from prior definitions and lemmas.
3645	<code>size sd = seedbytes =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3646	<code>public_verification_matrix md (keygen_internal md sd). '1 =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3647	<code>concrete_haetae_keygen_verification_matrix_ref md sd.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3648	<code>proof.</code>	Begins the proof script for the preceding proposition.
3649	<code>move=> sd_sz.</code>	Introduces universally quantified variables or hypotheses into the proof context.
3650	<code>have rt := haetae_keygen_public_key_ref_roundtrip md sd sd_sz.</code>	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
3651	<code>rewrite honest_public_verification_matrix_eq_packed_keygen.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3652	<code>rewrite /packed_haetae_keygen_verification_matrix</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3653	<code>/concrete_haetae_keygen_verification_matrix_ref</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3654	<code>/concrete_haetae_keygen_unpacked_public_key_ref</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3655	<code>/concrete_haetae_keygen_packed_public_key_ref</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
3656	/haetae_unpack_public_key_bytes_ref	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3657	/haetae_pack_public_key_bytes_ref.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3658	by rewrite rt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3659	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3660	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3661	lemma concrete_reference_keygen_public_verification_matrix_ref_correct md sd :	States checked lemma concrete_reference_keygen_public_verification_matrix_ref_correct. The following proof script must derive this proposition from prior definitions and lemmas.
3662	packed_haetae_verification_matrix md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3663	(packed_haetae_reference_public_key md sd) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3664	concrete_haetae_reference_keygen_verification_matrix_ref md sd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3665	proof.	Begins the proof script for the preceding proposition.
3666	have rt := haetae_reference_keygen_public_key_ref_roundtrip md sd.	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
3667	rewrite /concrete_haetae_reference_keygen_verification_matrix_ref	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3668	/concrete_haetae_reference_keygen_unpacked_public_key_ref	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3669	/concrete_haetae_reference_keygen_packed_public_key_ref.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3670	by rewrite rt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3671	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3672	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3673	lemma concrete_keygen_public_verification_matrix_size md sd :	States checked lemma concrete_keygen_public_verification_matrix_size. The following proof script must derive this proposition from prior definitions and lemmas.
3674	size (concrete_haetae_keygen_verification_matrix md sd) = mode_k md.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3675	proof.	Begins the proof script for the preceding proposition.
3676	by rewrite /concrete_haetae_keygen_verification_matrix	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3677	/packed_haetae_verification_matrix	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3678	/haetae_verification_matrix_from_unpacked size_mkseq; case md.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3679	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3680	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3681	lemma concrete_keygen_public_verification_matrix_ref_size md sd :	States checked lemma concrete_keygen_public_verification_matrix_ref_size. The following proof script must derive this proposition from prior definitions and lemmas.
3682	size (concrete_haetae_keygen_verification_matrix_ref md sd) = mode_k md.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3683	proof.	Begins the proof script for the preceding proposition.
3684	by rewrite /concrete_haetae_keygen_verification_matrix_ref	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3685	/packed_haetae_verification_matrix	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3686	/haetae_verification_matrix_from_unpacked size_mkseq; case md.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3687	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3688	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3689	lemma concrete_reference_keygen_public_verification_matrix_ref_size md sd :	States checked lemma concrete_reference_keygen_public_verification_matrix_ref_size. The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
3690	size (concrete_haetae_reference_keygen_verification_matrix_ref md sd) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3691	mode_k md.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3692	proof.	Begins the proof script for the preceding proposition.
3693	by rewrite /concrete_haetae_reference_keygen_verification_matrix_ref	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3694	/packed_haetae_verification_matrix	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3695	/haetae_verification_matrix_from_unpacked size_mkseq; case md.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3696	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3697	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3698	lemma concrete_keygen_public_verification_matrix_rows_wf md sd :	States checked lemma concrete_keygen_public_verification_matrix_rows_wf. The following proof script must derive this proposition from prior definitions and lemmas.
3699	all (polyvecl_wf md) (concrete_haetae_keygen_verification_matrix md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3700	proof.	Begins the proof script for the preceding proposition.
3701	rewrite /concrete_haetae_keygen_verification_matrix	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3702	/packed_haetae_verification_matrix	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3703	/haetae_verification_matrix_from_unpacked.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3704	apply/allP=> row /mkseqP [i [i_rng ->]].	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
3705	rewrite /polyvecl_wf size_mkseq.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3706	split.	Splits a conjunction or structured goal into independent proof obligations.
3707	+ by smt(mode_l_gt0).	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3708	apply/allP=> p /mkseqP [j [_ ->]].	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
3709	case: (j = 0) => j0; rewrite j0 /.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
3710	+ have col_wf :=	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3711	haetae_verification_column_from_unpacked_wf md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3712	(concrete_haetae_keygen_unpacked_public_key md sd).‘1	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3713	(concrete_haetae_keygen_unpacked_public_key md sd).‘2.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3714	smt(polyveck_wf_nth).	Calls SMT automation for first-order arithmetic, list, or algebraic side conditions.
3715	by apply poly_double_wf.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3716	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3717	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3718	lemma concrete_keygen_public_verification_matrix_ref_rows_wf md sd :	States checked lemma concrete_keygen_public_verification_matrix_ref_rows_wf. The following proof script must derive this proposition from prior definitions and lemmas.
3719	all (polyvecl_wf md)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3720	(concrete_haetae_keygen_verification_matrix_ref md sd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3721	proof.	Begins the proof script for the preceding proposition.
3722	rewrite /concrete_haetae_keygen_verification_matrix_ref	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3723	/packed_haetae_verification_matrix	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3724	/haetae_verification_matrix_from_unpacked.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
3725	<code>apply/allP=> row /mkseqP [i [i_rng ->]].</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
3726	<code>rewrite /polyvecl_wf size_mkseq.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3727	<code>split.</code>	Splits a conjunction or structured goal into independent proof obligations.
3728	<code>+ by smt(mode_l_gt0).</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3729	<code>apply/allP=> p /mkseqP [j [_ ->]].</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
3730	<code>case: (j = 0) => j0; rewrite j0 /.=.</code>	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
3731	<code>+ have col_wf :=</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3732	<code>haetae_verification_column_from_unpacked_wf md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3733	<code>(concrete_haetae_keygen_unpacked_public_key_ref md sd).‘1</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3734	<code>(concrete_haetae_keygen_unpacked_public_key_ref md sd).‘2.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3735	<code>smt(polyveck_wf_nth).</code>	Calls SMT automation for first-order arithmetic, list, or algebraic side conditions.
3736	<code>by apply poly_double_wf.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3737	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
3738	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3739	<code>lemma concrete_reference_keygen_public_verification_matrix_ref_rows_wf md sd :</code>	States checked lemma
		<code>concrete_reference_keygen_public_verification_matrix_ref_rows_wf</code> . The following proof script must derive this proposition from prior definitions and lemmas.
3740	<code>all (polyvecl_wf md)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3741	<code>(concrete_haetae_reference_keygen_verification_matrix_ref md sd).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3742	<code>proof.</code>	Begins the proof script for the preceding proposition.
3743	<code>rewrite /concrete_haetae_reference_keygen_verification_matrix_ref</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3744	<code>/packed_haetae_verification_matrix</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3745	<code>/haetae_verification_matrix_from_unpacked.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3746	<code>apply/allP=> row /mkseqP [i [i_rng ->]].</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
3747	<code>rewrite /polyvecl_wf size_mkseq.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3748	<code>split.</code>	Splits a conjunction or structured goal into independent proof obligations.
3749	<code>+ by smt(mode_l_gt0).</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3750	<code>apply/allP=> p /mkseqP [j [_ ->]].</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
3751	<code>case: (j = 0) => j0; rewrite j0 /.=.</code>	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
3752	<code>+ have col_wf :=</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3753	<code>haetae_verification_column_from_unpacked_wf md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3754	<code>(concrete_haetae_reference_keygen_unpacked_public_key_ref md sd).‘1</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3755	<code>(concrete_haetae_reference_keygen_unpacked_public_key_ref md sd).‘2.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3756	<code>smt(polyveck_wf_nth).</code>	Calls SMT automation for first-order arithmetic, list, or algebraic side conditions.
3757	<code>by apply poly_double_wf.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3758	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
3759	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
3760	lemma valid_signature_sign_internal md sk m ctx coins :	States checked lemma valid_signature_sign_internal. The following proof script must derive this proposition from prior definitions and lemmas.
3761	valid_signature md (sign_internal md sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3762	proof.	Begins the proof script for the preceding proposition.
3763	rewrite /valid_signature /sign_internal /signature_token /=.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3764	rewrite commitment_highbits_wf commitment_lowbits_wf /crh_wf message_hash_size	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3765	/commitment_challenge challenge_hash_wf response_vector_wf.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3766	by rewrite response_aux_vector_wf hint_vector_wf.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3767	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3768	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3769	lemma verify_challenge_consistent_sign_internal md sk m ctx coins :	States checked lemma verify_challenge_consistent_sign_internal. The following proof script must derive this proposition from prior definitions and lemmas.
3770	verify_challenge_consistent md (public_key_of_secret md sk) m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3771	(sign_internal md sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3772	proof.	Begins the proof script for the preceding proposition.
3773	by rewrite /verify_challenge_consistent /sign_internal	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3774	/commitment_challenge /challenge_hash /=.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3775	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3776	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3777	op coeff_norm_sq (x : coeff) : int = x * x.	Defines integer-valued parameter coeff_norm_sq, used for dimensions, budgets, support sizes, or bounds.
3778	op poly_norm_sq (p : poly) : int =	Defines integer-valued parameter poly_norm_sq, used for dimensions, budgets, support sizes, or bounds.
3779	if poly_unit p then size p	Splits proof obligations according to a conditional branch in the program or formula.
3780	else if p = poly_zero then 0	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3781	else sumz (map coeff_norm_sq p).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3782	op polyvecl_norm_sq (md : mode) (xs : polyvecl) : int =	Defines integer-valued parameter polyvecl_norm_sq, used for dimensions, budgets, support sizes, or bounds.
3783	if polyvecl_unit md xs then mode_l md * n	Splits proof obligations according to a conditional branch in the program or formula.
3784	else if xs = polyvecl_zero md then 0	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3785	else sumz (map poly_norm_sq xs).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3786	op polyveck_norm_sq (md : mode) (xs : polyveck) : int =	Defines integer-valued parameter polyveck_norm_sq, used for dimensions, budgets, support sizes, or bounds.
3787	if polyveck_unit md xs then mode_k md * n	Splits proof obligations according to a conditional branch in the program or formula.
3788	else if xs = polyveck_zero md then 0	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3789	else sumz (map poly_norm_sq xs).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3790	op signature_response_norm_sq (md : mode) (sig : signature) : int =	Defines integer-valued parameter signature_response_norm_sq, used for dimensions, budgets, support sizes, or bounds.
3791	polyvecl_norm_sq md (sig_response sig) +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3792	polyveck_norm_sq md (sig_response_aux sig).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3793	op signature_verify_norm_sq (md : mode) (sig : signature) : int =	Defines integer-valued parameter signature_verify_norm_sq, used for dimensions, budgets, support sizes, or bounds.
3794	signature_response_norm_sq md sig +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
3795	<code>polyveck_norm_sq md (sig_hint sig).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3796	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3797	<code>op secret_norm_sq (sk : skey) : int = sk.'2.</code>	Defines integer-valued parameter <code>secret_norm_sq</code> , used for dimensions, budgets, support sizes, or bounds.
3798	<code>op response_norm_sq (md : mode) (sig : signature) : int =</code>	Defines integer-valued parameter <code>response_norm_sq</code> , used for dimensions, budgets, support sizes, or bounds.
3799	<code>signature_response_norm_sq md sig.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3800	<code>op verify_norm_sq (md : mode) (sig : signature) : int =</code>	Defines integer-valued parameter <code>verify_norm_sq</code> , used for dimensions, budgets, support sizes, or bounds.
3801	<code>signature_verify_norm_sq md sig.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3802	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3803	<code>op secret_norm_ok (md : mode) (sk : skey) : bool =</code>	Defines predicate <code>secret_norm_ok</code> , an event or well-formedness condition used in later implications and game proofs.
3804	<code>0 <= secret_norm_sq sk /\ secret_norm_sq sk <= mode_b0sq md.</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
3805	<code>op response_norm_ok (md : mode) (sig : signature) : bool =</code>	Defines predicate <code>response_norm_ok</code> , an event or well-formedness condition used in later implications and game proofs.
3806	<code>0 <= response_norm_sq md sig /\ response_norm_sq md sig <= mode_b1sq md.</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
3807	<code>op verify_norm_ok (md : mode) (sig : signature) : bool =</code>	Defines predicate <code>verify_norm_ok</code> , an event or well-formedness condition used in later implications and game proofs.
3808	<code>0 <= verify_norm_sq md sig /\ verify_norm_sq md sig <= mode_b2sq md.</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
3809	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3810	<code>op verify_internal :</code>	Defines operation <code>verify_internal</code> , a total mathematical function or abbreviation used by later declarations.
3811	<code>mode -> pkey -> message -> context -> signature -> bool =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3812	<code>fun (md : mode) (pk : pkey) (m : message) (ctx : context)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3813	<code>(sig : signature) =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3814	<code>valid_signature md sig /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3815	<code>verify_challenge_consistent md pk m ctx sig /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3816	<code>verify_public_equation md pk sig /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3817	<code>verify_norm_ok md sig.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3818	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3819	<code>lemma secret_norm_ok_current md sd :</code>	States checked lemma <code>secret_norm_ok_current</code> . The following proof script must derive this proposition from prior definitions and lemmas.
3820	<code>secret_norm_ok md (secret_key_of_seed md sd).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3821	<code>proof. by rewrite /secret_norm_ok /secret_norm_sq /secret_key_of_seed; case md. qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3822	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3823	<code>lemma response_norm_ok_current md sk m ctx coins :</code>	States checked lemma <code>response_norm_ok_current</code> . The following proof script must derive this proposition from prior definitions and lemmas.
3824	<code>response_norm_ok md (sign_internal md sk m ctx coins).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3825	<code>proof.</code>	Begins the proof script for the preceding proposition.
3826	<code>rewrite /response_norm_ok /response_norm_sq /signature_response_norm_sq</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3827	<code>/sign_internal /signature_token /sig_response /sig_response_aux</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3828	<code>/sig_token_value /polyvecl_norm_sq /polyveck_norm_sq</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
3829	<code>/response_vector /response_aux_vector.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3830	<code>have hresp :</code>	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
3831	<code>polyvecl_unit md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3832	<code>(deterministic_unit_polyvecl md 17 (response_source sk m ctx coins)).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3833	<code>+ by apply deterministic_unit_polyvecl_unit.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3834	<code>have haux :</code>	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
3835	<code>polyveck_unit md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3836	<code>(deterministic_unit_polyveck md 29 (response_source sk m ctx coins)).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3837	<code>+ by apply deterministic_unit_polyveck_unit.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3838	<code>rewrite hresp haux.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3839	<code>clear hresp haux.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3840	<code>by case md.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3841	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
3842	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3843	<code>lemma verify_norm_ok_current md sk m ctx coins :</code>	States checked lemma <code>verify_norm_ok_current</code> . The following proof script must derive this proposition from prior definitions and lemmas.
3844	<code>verify_norm_ok md (sign_internal md sk m ctx coins).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3845	<code>proof.</code>	Begins the proof script for the preceding proposition.
3846	<code>rewrite /verify_norm_ok /verify_norm_sq /signature_verify_norm_sq</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3847	<code>/signature_response_norm_sq /sign_internal /signature_token</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3848	<code>/sig_response /sig_response_aux /sig_hint /sig_token_value</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3849	<code>/polyvecl_norm_sq /polyveck_norm_sq</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3850	<code>/response_vector /response_aux_vector /hint_vector.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3851	<code>have hresp :</code>	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
3852	<code>polyvecl_unit md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3853	<code>(deterministic_unit_polyvecl md 17 (response_source sk m ctx coins)).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3854	<code>+ by apply deterministic_unit_polyvecl_unit.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3855	<code>have haux :</code>	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
3856	<code>polyveck_unit md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3857	<code>(deterministic_unit_polyveck md 29 (response_source sk m ctx coins)).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3858	<code>+ by apply deterministic_unit_polyveck_unit.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3859	<code>have hhint :</code>	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
3860	<code>polyveck_unit md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3861	<code>(deterministic_unit_polyveck md 41 (response_source sk m ctx coins)).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3862	<code>+ by apply deterministic_unit_polyveck_unit.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.

Line	EasyCrypt source	Mathematical reading
3863	<code>rewrite hresp haux hhint.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3864	<code>clear hresp haux hhint.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3865	<code>by case md.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3866	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
3867	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3868	<code>lemma verify_internal_sign_internal md sk m ctx coins :</code>	States checked lemma <code>verify_internal_sign_internal</code> . The following proof script must derive this proposition from prior definitions and lemmas.
3869	<code>verify_internal md (public_key_of_secret md sk) m ctx</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3870	<code>(sign_internal md sk m ctx coins).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3871	<code>proof.</code>	Begins the proof script for the preceding proposition.
3872	<code>rewrite /verify_internal.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3873	<code>split.</code>	Splits a conjunction or structured goal into independent proof obligations.
3874	<code>+ by apply valid_signature_sign_internal.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3875	<code>split.</code>	Splits a conjunction or structured goal into independent proof obligations.
3876	<code>+ by apply verify_challenge_consistent_sign_internal.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3877	<code>split.</code>	Splits a conjunction or structured goal into independent proof obligations.
3878	<code>+ rewrite /verify_public_equation /verify_reconstructed_highbits</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3879	<code>/sign_internal /commitment_highbits</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3880	<code>/commitment_challenge /=.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3881	<code>by rewrite /signature_token /sig_response_aux /sig_token_value.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3882	<code>by apply verify_norm_ok_current.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3883	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
3884	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3885	<code>end HAETAE_Algebra.</code>	Closes the current theory or module block.